

An Open-Source Dual-Camera Edge AI Runtime for Vision-Guided Drone Autonomy on an MCU-EdgeTPU Platform

Author Name ^{1,*}

¹ Affiliation, City, Country; email@example.com

* Correspondence: email@example.com

Highlights

What are the main findings?

- Our MCU-EdgeTPU board extends the original Coral Micro board from which the design started with dual-camera sensing, onboard IMU support, MicroPython control, and both EdgeTPU-backed and MCU-side inference paths for compact autonomous-drone experiments.
- The implementation indicates that sustained onboard perception on this class of hardware depends strongly on RTOS scheduling, memory placement, and hardware-assisted image preparation rather than on accelerator execution alone.

What are the implications of the main findings?

- Lightweight UAV platforms can support richer onboard perception when sensing, memory movement, inference placement, and telemetry are co-designed as one embedded system.
- A compact open-source platform can serve as a reproducible testbed for studying onboard inference, runtime adaptation, and bounded high-level tool use in drone applications.

Abstract

This paper presents our MCU-EdgeTPU board, an open-source inference platform derived from the original Coral Micro board from which the design started and extended with two 5 Mpx cameras, an onboard IMU, and a single MicroPython runtime. The platform is intended to make onboard inference accessible from scripts, not only by exposing accelerator access, but by integrating sensing, model execution, communication, and runtime utilities into a single embedded environment. The runtime supports both EdgeTPU-backed execution for quantized models compiled for accelerator use and CPU-side TensorFlow Lite Micro execution for auxiliary workloads that remain on the MCU. The platform targets two complementary UAV workflows: Bitcraze Crazyflie for agile indoor experimentation and PX4/MAVLink-oriented autopilot integration for more transferable field deployment. At the perception level, the runtime provides a dual-camera, multi-model execution path with camera switching, per-camera geometry, lightweight tracking, motion compensation, and ground-aware spatial reasoning. A central contribution is the systems-level engineering of the visual path from sensor capture to EdgeTPU input, including DMA-backed acquisition, hardware-assisted image preparation, memory placement across internal RAM and SDRAM, and pipelined FreeRTOS scheduling that overlaps capture, preprocessing, and inference. The implementation therefore treats throughput, buffer ownership, and accelerator feeding as first-class design constraints rather than secondary optimization details. Beyond the runtime itself, the work is accompanied by an open synthetic dataset and a custom YOLO model intended for low-altitude drone navigation in inhabited environments. Taken

Received:

Accepted:

Published:

Copyright: © 2026 by the authors.

Submitted to *Drones* for possible open access publication under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](#) license.

together, these elements define an open embedded platform for onboard inference experiments on a compact airborne system: integrated sensing, accelerator-backed and MCU-side inference paths, and a substantial set of embedded-ML functions available directly from MicroPython. Together with the accompanying synthetic aerial dataset and custom YOLO model, the work extends the original Coral Micro board from which the design started into a more complete environment for onboard inference, adaptive embedded ML, and lightweight autonomous-drone research.

Keywords: embedded AI; EdgeTPU; MicroPython; drone autonomy; MAVLink; Crazyflie; dual camera; FreeRTOS; embedded inference platform

1. Introduction

Autonomous visual navigation for small drones sits at the intersection of several conflicting constraints: low mass, low power, real-time perception, onboard decision making, and enough software flexibility to support rapid iteration. In practice, many platforms solve only part of this problem. Very small aerial vehicles are easy to fly indoors and safe to test in tight environments, but they often lack the compute and memory budget required by modern object detection pipelines. At the opposite end, larger embedded AI platforms such as Coral Dev Board or NVIDIA Jetson-class systems provide significantly more resources, but they also bring higher current consumption, larger batteries, more weight, and a form factor that is difficult to justify for lightweight micro-UAV missions.

This work starts from the original Coral Micro board from which the design started and treats it not as a fixed product, but as a compact open platform that can be extended through systems engineering. The goal is not to add isolated features, but to assemble our MCU-EdgeTPU board as an embedded-AI stack with improved sensing, more predictable runtime behavior, and a clearer control and telemetry model.

The starting point of this work was the need for a platform that remains physically lightweight and energy-efficient, while still exploiting as much as possible the inference capability of the EdgeTPU and the real-time performance of the MCU. The original minimalist EdgeTPU-based board configuration was attractive from the perspective of size and power, but its standard low-resolution camera workflow was not sufficient for the class of missions targeted here. In particular, a 320x240 sensing regime is too restrictive when the goal is to support richer object detection, stricter scene understanding, and higher-confidence flight decisions in inhabited or cluttered spaces. This became even more limiting when considering modern YOLO-style detectors, which benefit from larger spatial inputs and from a sensor pipeline capable of preserving more detail before resizing and quantization.

The objective is therefore to define a different point in the design space: a board-level and software-level architecture that preserves the low-weight, low-power character of minimalist embedded AI hardware, but extends it toward an open inference platform for autonomous-drone operation. The platform couples an MCU and an EdgeTPU with two 5 Mpx cameras and an onboard IMU, and exposes sensing, inference, communication, and control through MicroPython. High-level mission logic remains scriptable, while timing-critical operations stay in the RTOS and accelerator path. The runtime is also no longer limited to a single inference route: it includes both EdgeTPU-backed execution for quantized models compiled for accelerator execution and CPU-side TensorFlow Lite Micro execution, together with a growing family of embedded-ML utilities intended to be used directly from scripts.

The work explicitly targets two flight platforms. The first is Bitcraze Crazyflie, used as an indoor experimental platform because it is agile, lightweight, easy to maneuver, and well suited for repeated safety-conscious testing in the laboratory. The second is a PX4-based autopilot stack accessed through MAVLink, representing the path toward a more professional and transferable deployment model. The intention is to provide a common high-level autonomy layer across both targets, so that mission scripts, perception logic, and communication flows can be developed once and adapted across indoor research flights and more operational MAVLink-centered scenarios. This broader PX4-facing integration matters because the runtime is designed not only to emit textual or backend-oriented telemetry, but also to provide structured obstacle information that can be consumed directly by autopilot-side collision-prevention logic.

From a vision-systems perspective, the work is driven by the observation that model accuracy alone is not enough. Once YOLO-class detectors are pushed toward larger inputs, for example 640-class or even workflows derived from native 1280-wide camera imagery, the dominant issues become memory layout, transfer scheduling, tensor preparation cost, and buffer ownership between the camera subsystem, the MCU, and the EdgeTPU. For this reason, the contribution is as much about systems engineering as it is about AI. The implementation analyzes the full visual path from image acquisition at the sensor, through DMA-backed capture into RAM, through hardware pixel conversion and scaling, and finally into the EdgeTPU input memory. These stages are pipelined in parallel under the RTOS so that camera capture and image preparation can overlap with inference, thereby maximizing frame rate while minimizing idle time and unnecessary copies.

An important architectural extension is the addition of a dual-camera subsystem. Instead of treating perception as a single fixed viewpoint problem, the platform supports two 5 Mpx sensors that may be mounted in parallel or perpendicular configurations. This enables distinct operational modes: forward-looking navigation together with nadir inspection, redundant views for indoor localization, or viewpoint switching according to the phase of the mission. The camera-switching mechanism is designed as a first-class runtime concept, not as an afterthought, and is supported by explicit per-camera geometry, frame sequencing, and clean post-switch acquisition logic. Together with IMU-aware tracking and ground-plane projection, this creates the basis for drone-centric perception that is aware of mounting geometry and flight context rather than only raw image content.

The work also extends beyond the runtime itself. In addition to dynamic model loading and multi-model switching, the project assumes the availability of a custom YOLO detector trained on an open synthetic dataset built for low-altitude drone navigation in inhabited spaces. The dataset contains views captured at nadir and at approximately 45 degrees, and is intended as a reusable pre-training or adaptation resource for future systems that must obey stricter navigation and avoidance logic in populated environments, including scenarios informed by European drone categories such as A0-A3 and A2. This dataset-and-model layer complements the runtime by providing a path from open training assets to experimentally reproducible, EdgeTPU-oriented inference models.

The present work should therefore be read as an integration effort. Embedded Python, edge inference, object detection, MAVLink communication, visual tracking, and lightweight learning primitives are all known in isolation. The difficulty is making them coexist on a highly constrained airborne platform without losing throughput, determinism, or usability. The practical bottlenecks addressed here include RAM placement across internal memory and SDRAM, large tensor staging, RTOS synchronization, camera-switching latency, conversion of tracked detections into obstacle maps, and the need to sustain high throughput without giving up the low-power character of the platform.

The main contributions of the work can be summarized as follows:

1. An open-source MicroPython runtime for a lightweight EdgeTPU-enabled MCU platform intended for high-level autonomous drone control, with both EdgeTPU and CPU-side TensorFlow Lite Micro inference paths.
2. A dual-target flight-control concept spanning Bitcraze Crazyflie for indoor experimentation and PX4/MAVLink for professional autopilot integration.
3. A dual-camera 5 Mpx sensing architecture with onboard IMU support, runtime camera switching, per-camera geometry, and viewpoint-aware operation.
4. A real-time perception pipeline that studies and optimizes the end-to-end path from image sensor to MCU RAM to EdgeTPU memory, using parallel RTOS execution and double-buffered staging to maximize frame rate.
5. A memory-aware deployment strategy for YOLO-class detectors and other larger vision models on an MCU-constrained platform.
6. A multi-model workflow with dynamic model loading, context-dependent switching, tracking, projection to the ground plane, obstacle reporting to MAVLink autopilots, and telemetry integration.
7. An extended embedded-ML runtime surface that includes on-device learning, clustering, anomaly detection, sequence modeling, reinforcement learning, and lightweight mapping primitives as integrated MicroPython-accessible components alongside the vision stack.
8. An open synthetic aerial dataset and a custom YOLO model intended as a starting point for regulation-aware drone navigation in inhabited environments.

The rest of the paper is organized as follows. The next section reviews related work in autonomous nano-UAV navigation, embedded deep learning, YOLO-based detection, MicroPython-enabled embedded control, and accelerator-centric edge AI. The following sections present the hardware and software architecture, the real-time memory and processing pipeline, the communication and flight-control abstractions, and the experimental perspective enabled by the platform.

2. Related Work

The narrow related-work discussion for the **dual-sensor switching and MCU camera pipelines** contribution of this paper is in [related_embedded_inference.md](#). The sections below position the broader platform and its components in the wider nano-UAV, edge-AI, YOLO, MicroPython and SLAM literatures.

2.1. Autonomous visual navigation for nano-UAVs

An important reference point for this work is the line of research that demonstrates fully onboard visual autonomy on extremely small flying robots. A particularly relevant example is *An Open Source and Open Hardware Deep Learning-powered Visual Navigation Engine for Autonomous Nano-UAVs*, which shows that a nano-UAV can execute a closed-loop visual navigation pipeline onboard under severe power and mass constraints. That work is important not only because it proves feasibility, but because it frames the design problem correctly: perception, inference, control, and hardware architecture cannot be treated independently when the air vehicle is very small. The platform developed here follows this systems-level viewpoint, but shifts the focus from a custom navigation engine toward a more general-purpose embedded AI runtime that combines high-level scripting, object detection, multi-view sensing, and compatibility with both experimental and professional drone-control ecosystems.

The broader nano-UAV literature also distinguishes between systems that are truly autonomous onboard and systems that depend on offboard computing, external localization infrastructure, or heavy companion computers. This distinction remains essential. For

indoor research, offboard support is useful during development, but it does not solve the core embedded autonomy problem. Our MCU-EdgeTPU board therefore sits within the class of systems that aim to keep perception and high-level decision making onboard, even when this requires substantial engineering effort in memory organization, RTOS scheduling, and sensor integration.

2.2. Embedded AI accelerators and systolic-array inference

The use of an EdgeTPU-class accelerator places this work within the wider context of edge inference on specialized matrix-processing hardware. Systolic-array accelerators changed the practical landscape of embedded AI because they made it possible to execute neural inference at high throughput and low energy relative to CPU-only or even general GPU-centric approaches. In this broader context, the TPU family established a strong reference model for efficient tensor computation through structured dataflow and regular multiply-accumulate pipelines. This work is not a study of accelerator microarchitecture by itself, but it depends critically on the availability of this kind of accelerator model and on the practical challenge of feeding it efficiently from a constrained MCU environment.

This distinction matters. On larger boards, accelerator throughput is often discussed independently from sensor transfer and host memory layout, because the host has enough resources to hide inefficiencies. On small MCU-based systems, however, the effective performance of the accelerator is inseparable from the cost of moving images from the camera to RAM, rearranging tensors, and scheduling inference without starving the rest of the application. In that sense, this work contributes to the embedded AI literature by focusing on the host-side industrialization problem around the accelerator, not only on the accelerator itself.

2.3. YOLO-family object detectors and compact detection models

The object detection component of this work is aligned with the large body of literature around single-stage detectors and especially the YOLO family. YOLO-style architectures are attractive in airborne robotics because they offer a practical balance between accuracy, latency, and deployment simplicity. They are also natural candidates for quantized execution and edge deployment. At the same time, deploying YOLO-class models on resource-constrained embedded platforms introduces a very different set of concerns from those seen on desktop GPUs: input resolution becomes a memory-placement problem, post-processing cost becomes part of the frame budget, and intermediate copies become unacceptable at scale.

Within this design space, compact variants such as YOLO26-style models are especially relevant because they seek to preserve detection usefulness while reducing computational and memory pressure. This makes them well matched to embedded AI pipelines in which the native camera imagery may be relatively large, but the runtime still needs to fit within tight SRAM and SDRAM limits. Our platform sits precisely at this interface between compact model design and deployment-aware systems engineering. Rather than treating the detector as an isolated black box, it studies how detector choice interacts with camera resolution, quantization, memory layout, and real-time scheduling.

2.4. MicroPython and high-level programmability on embedded systems

MicroPython has long been valued in embedded systems for shortening the iteration cycle between algorithm design, hardware interaction, and system testing. In robotics, this is particularly attractive because state machines, mission logic, communication rules, and experimental procedures change quickly, often faster than low-level firmware can be safely recompiled and reflashed. However, the majority of MicroPython deployments

emphasize convenience over hard real-time performance and are usually not coupled to a high-throughput edge-AI pipeline.

The present platform uses MicroPython differently: not as a replacement for the real-time system, but as a high-level orchestration layer placed above it. Time-critical operations such as frame capture, scaling, quantization, inference, and buffering remain implemented in compiled code under the RTOS, while Python exposes them as programmable mission primitives. This approach is relevant to previous work on embedded scripting because it shows how an interpreted environment can be integrated into an accelerator-centric, real-time drone application without turning the interpreter into the performance bottleneck.

2.5. *Lightweight embedded learning, sequence modeling, and SLAM primitives*

The newer `sentai` namespaces also connect the platform to a broader line of work in statistical learning and robotics. The `kmeans` and `pca` modules draw on widely used clustering and dimensionality-reduction methods that remain attractive on microcontroller-class systems because they are compact, interpretable, and easy to integrate into larger pipelines [1,2]. The `anomaly` module fits the same pattern: it relies on lightweight statistical monitoring techniques that are practical when memory and compute budgets do not permit heavier end-to-end methods [3,4].

The sequence-oriented modules map just as cleanly onto established post-1990 literature. Dynamic time warping remains a practical baseline for elastic matching of short temporal patterns [5], while hidden Markov models provide a compact probabilistic framework for temporal-state inference [6]. The reinforcement-learning surface draws from tabular Q-learning, bandit strategies such as UCB and Thompson sampling, and later neural approximations exemplified by DQN [7–10]. The `slam` namespace likewise sits within the modern SLAM literature on probabilistic mapping and localization [11,12]. The contribution here is not novelty in these algorithms themselves, but their exposure as compact components inside the same embedded runtime that already handles sensing, inference, and flight interoperability.

2.6. *Vision pipelines for drone perception and viewpoint-aware sensing*

Most embedded drone-vision pipelines still assume a single active camera and a relatively static geometry. This is reasonable in many systems, but it limits mission flexibility. Drone operation frequently benefits from multiple sensing attitudes: forward views for obstacle awareness and navigation, nadir views for inspection and target localization, and oblique views that balance situational context with ground visibility. This work extends that discussion by treating dual-camera operation and runtime camera switching as core design elements. The system therefore aligns with research on viewpoint-aware aerial perception, but approaches the problem from the perspective of deployment on a compact embedded AI stack rather than from the perspective of large-scale perception benchmarks alone.

This also connects with work on projection-aware tracking and sensor fusion. Once IMU measurements, camera geometry, and ground-plane assumptions are available, detections can be represented not only in image coordinates but also in more operationally meaningful spatial coordinates. The implementation builds on this idea by combining tracking, IMU-aware motion compensation, per-camera geometry, and projection to the ground plane so that perception can be consumed more directly by flight logic and telemetry systems. In tracking terms, the design sits between classical SORT [13] and later ByteTrack-style association [14]: it preserves a simple Kalman-based online core and the recovery of lower-confidence detections, but replaces heavier desktop-oriented assumptions with

MCU-suitable mechanisms such as static memory, compact histogram cues, and IMU-based motion compensation.

2.7. Open datasets, synthetic data, and regulation-aware navigation

Synthetic data has become increasingly important in aerial robotics because collecting and annotating large real-world datasets from drones is expensive, repetitive, and often constrained by safety or regulation. For applications involving inhabited spaces, this difficulty is even greater: the scenes are operationally sensitive, the failure modes are costly, and the data should ideally cover multiple viewing angles and flight contexts. An open synthetic dataset that explicitly contains nadir and oblique perspectives is therefore consistent with current trends in simulation-assisted pre-training and domain adaptation.

The relevance of this direction becomes stronger when drone navigation is discussed not only as a perception problem, but also as a compliance problem. European operation categories such as A0-A3 and A2 motivate systems that are more conservative, more aware of surrounding objects and persons, and easier to validate under explicit operational constraints. While regulation-aware autonomy is still an emerging topic, it provides an important framing device for this work: the detector and dataset are not meant only to improve benchmark accuracy, but to support future high-level navigation policies that must operate under stricter safety and legal constraints.

2.8. Positioning of this work

Relative to prior work, our MCU-EdgeTPU board occupies a middle ground that is not addressed well by any single existing direction. It is lighter and more power-conscious than larger embedded AI computers, more programmable than fixed-function onboard vision stacks, more perception-rich than minimal MCU-only controllers, and more systems-oriented than many standalone model papers. Its contribution is therefore not a new detector alone, not a new autopilot alone, and not a new board alone, but the integration of these concerns into an embedded autonomy platform for small drones.

Indicative references to include in the final bibliography are the following:

1. *An Open Source and Open Hardware Deep Learning-powered Visual Navigation Engine for Autonomous Nano-UAVs.*
2. Core YOLO-family papers and the compact YOLO26 line relevant to efficient embedded detection.
3. Foundational work on TPU and systolic-array-based neural accelerators.
4. MicroPython project and related work on embedded scripting for robotics.
5. Prior work on aerial perception, synthetic datasets for UAV vision, and regulation-aware autonomous flight.

2.9. Dual-Sensor Switching and MCU Camera Pipelines

A focused related-work chapter for the specific contributions of this study: glitch-free dual-camera switching on a shared MIPI-CSI lane, per-iteration measurement of switch cost on MCU-class hardware, and the platform-level design choices that made those measurements possible. Complements the broader survey in [relatedwork.md](#), which covers nano-UAV autonomy, YOLO-family detectors, and MicroPython-in-robotics context; this chapter is narrower and deeper on the switching problem.

3. 1. Hardware multiplexing of multiple cameras onto one CSI receiver318

3.1. 1.1 Analogue-switch MUX (our topology)319

Multiplexing two camera modules onto a single MIPI-CSI2 lane via an analogue switch is the default approach on small-form-factor evaluation boards where the SoC has only one CSI receiver and board area is scarce. The TS5MP645 family from Texas Instruments is a representative device; in the NXP i.MX ecosystem, the same pattern appears on the i.MX8M Mini where the OV5640 community thread (1064767) documents two-camera operation through a single CSI controller with GPIO-selected mux. The analogue-switch approach is attractive because the MUX is cheap, passive, and transparent to the D-PHY receiver once the line settles, but it carries two known limitations:

- **No virtual-channel separation.** Both sensors’ MIPI short packets (FS, LS) arrive on the same virtual channel (VC=0) from the receiver’s perspective; there is no hardware disambiguation between them. This forces the platform to ensure only one sensor is driving the line at a time, which is exactly what the GPIO MUX guarantees.
- **Switch timing is software’s responsibility.** The receiver has no notion of “sensor change”; if the MUX flips mid-line, the DMA buffer being written at that moment silently contains pixels from both sensors. Our measurement of this failure mode is in [cam_switch.md](#) §“Visual evidence — seam before the fix” and is the ground truth that motivates flip-on-EOF.

The TI E2E thread on the TS5MP645 MIPI-CSI2 MUX (874026) identifies D-PHY re-training after each switch as an optional but expensive mitigation; we instead keep the MUX transition inside the MIPI blanking window so no re-training is needed.

3.2. 1.2 Virtual-channel-separated dual-stream receivers340

Hardware that supports MIPI-CSI2 virtual channels natively — e.g. systems using MIPI-bridge chips such as the Lontium LT9611 or Toshiba TC358748 — can run two sensors concurrently on separate VCs and demultiplex at the receiver side. On such a topology the “switch cost” problem disappears entirely because both streams are live. NXP’s MIPI-CSI2 peripheral on the RT1170/1176 supports four virtual channels on the receive side, but the OV5640 cannot natively remap its output VC, and we do not have a bridge chip on the SentAI board. This positions our measurements as a lower-bound study: any platform that upgrades to a VC-split topology should expect *better* dual- camera throughput than we report, not worse.

3.3. 1.3 Dual CSI controllers350

SoCs with multiple independent CSI receivers (e.g. the i.MX8MP family, Jetson Xavier/Orin line) are a third option. Each sensor has its own receiver and its own DMA pipeline; there is no shared lane to multiplex. The community threads for the i.MX8MM dual- OV5640 case (NXP Community posts 1064767 and 1435731) discuss a variety of configurations, ranging from single-VC with GPIO mux (analogous to our board) to dual-CSI with fully independent streams. The trade-off is not framed in the literature as “software switch cost” but as “how many lanes the application of interest can afford”; our measurement of what the single-lane topology costs is, to our knowledge, quantitative evidence that was previously only inferred.

3.4. 1.4 Summary of topology options360

Table 1. Comparative positioning of dual-camera topologies, with per-topology cost of a camera switch.

Topology	Shared CSI?	Switch cost per flip	Hardware complexity
Analogue MUX + single CSI receiver (this work)	yes	~83 ms (evaluation.md Table 3)	minimum — one analogue switch IC
MIPI-bridge with VC split + single CSI receiver	yes, but demultiplexed in hardware	0 ms (both live)	moderate — bridge chip, extra board area
Two independent CSI receivers on SoC	no	0 ms (software “select” is a buffer choice)	high — SoC dependency, routing

4. 2. VSYNC-synchronised switching and frame-boundary techniques

The pattern of “change hardware state only during the blanking interval” is old in display systems and has well-known analogues in capture. The camera-capture literature (RidgeRun wiki on Camera Sensor Basics; Raspberry Pi forum thread 48238 on frame synchronisation; Jetson developer forum 252751 on CSI-2 sync packet shifts) uses the term “synchronisation to the frame boundary” in two distinct ways:

1. **Sync two cameras to each other** (using FSIN on the OV5640, master/slave wiring). This is the approach taken when the system needs two *simultaneously captured* frames, typically for stereo or for HDR with two different exposures. It does not solve the switching problem — both sensors still stream, and if only one receiver is available, the second is lost unless there is a MUX.
2. **Sync the MUX flip to the frame boundary** of the current sensor. This is what our flip-on-EOF primitive does. The literature for this specific pattern in embedded camera pipelines is sparse — most industrial and NVIDIA Jetson-side systems assume dual receivers — so our contribution is a concrete implementation and a measurement of the residual cost.

Framebuffer-replacement techniques in display drivers (double buffering, page flipping, Vsync-triggered DMA update) are the architectural ancestor of flip-on-EOF. The analogue is not exact because we are flipping an input multiplexer rather than swapping an output buffer, but the principle — *do the hardware state change during the interval when no data is flowing* — is identical.

5. 3. Sensor-side effects after a switch

Once the mid-buffer seam is eliminated (§2), the residual visual artefact observed at `switch_drain=1` in our E17 data ([threats_to_validity.md](#) §2.5) is consistent with known OV5640 post-stream-resume behaviour: AEC/AGC loops take several frames to converge, and the first one or two frames after a line becomes active again can be mildly over- or under-exposed relative to the sensor’s final state. The Armbian forum thread (armbian.com topic 4688) on OV5640 color-space configuration, and the ESP32-camera GitHub issue 201 on frame-rate regressions, both touch on this effect obliquely. We did not find a published quantitative characterisation of how many frames are needed for AEC/AGC convergence on this specific sensor; our choice of `switch_drain=2` as the production default is an engineering conservatism grounded in the observation that three separate sessions ([s039](#),

s041, the user-visual review) all showed some residual artefact at drain=1. A follow-up study could instrument the sensor's AEC/AGC status register to tie the observed pixel artefact to the sensor's own convergence counter.

6. 4. ISR-context GPIO and NASA/JPL-style real-time discipline

The implementation choice to do the MUX flip inside the CSI EOF ISR is small in code volume but significant in real-time terms. Three strands of prior work inform this choice:

- **AUTOSAR / industrial real-time** — the principle that ISRs should be minimal and defer work to task context is codified in AUTOSAR methodology and widely applied in automotive MCU firmware. Our ISR does not defer — it performs the GPIO write directly — but it is still compliant with the “minimum, bounded, deterministic” rule because the operation is a single atomic-write to DR_SET / DR_CLEAR (see [libs/base/gpio.cc GpioSetFromIsr](#)) with no mutex, no loop, no scheduler interaction.
- **NASA/JPL flight-software rules** — the discipline referenced in [agent/embedded.md](#) (bounded loops, bounded ISRs, explicit fault containment, error-code taxonomy). Our implementation explicitly applies the rules: ISR body is branch- and-store only; ratio scheduling is stateless modulo; task-side timeout uses delta arithmetic surviving a tick-counter wrap; every degraded path has its own error code and counter.
- **RT-MCU embedded patterns** — the broader body of MCU-camera literature routinely uses GPIO writes in ISRs for pin-state changes that must race against hardware timing (e.g. exposure triggering, flash-strobe). What is less common, and is our contribution, is wiring the ISR write to a measured MIPI-CSI timing event (end-of-frame) rather than to a free-running timer.

The end result is that the timing of the MUX flip is no longer software-controlled in the sense of “when does the task happen to run”; it is hardware-event-controlled. That is the transition from best-effort to deterministic that this work makes concrete for the dual-sensor case.

7. 5. Cross-platform framing

MCU-class camera-inference pipelines on NXP i.MX RT family (Zephyr support for the VMU RT1170 board, NXP application notes AN13116, AN13264, AN5305) focus on single-sensor throughput and power optimisation; dual-sensor switching is occasionally mentioned as a feature of the platform but not, to our knowledge, measured to the per-stage granularity we present. Our E18 head-to-tail benchmark is portable — any platform implementing the same diag session model can run the same three-sweep A/B/C benchmark against any MUX topology and any sensor pair — and would be useful as a comparison point for future platforms claiming lower switch cost.

OpenMV's OV5640 breakout, Adafruit's OV5640 module, and the broader Arduino-ecosystem OV5640 boards expose the same sensor family but do not target the dual-sensor use case we focus on. Their configurations are single-camera and their performance envelopes are therefore not directly comparable.

8. 6. References to gather for the bibliography

Indicative external sources identified during this study; to be folded into the paper’s references.bib. Each is cited in this or the companion chapters at the point where its content is used.

Source	Cited for
NXP Community post 1064767 — “Dual camera(ov5640) in iMx8 M mini”	dual-OV5640 configuration on shared CSI with GPIO mux
NXP Community post 1992533 — “OV5640 and OV7251 with IMX8MM over MIPI-CSI”	mixed-sensor dual-camera topology
NXP Community post 825091 — “two OV5640 mipi csi cameras”	single-VC, single-receiver dual-OV5640
NXP AN5305 — “MIPI-CSI2 Peripheral on i.MX6 MPUs”	MIPI-CSI2 packet structure, FS/LS semantics
TI E2E forum 874026 — “TS5MP645: MIPI CSI-2 multiplexer between camera and processor”	impedance and timing implications of analogue MUX on a D-PHY
RidgeRun wiki — “Camera Sensor Basics”	VSYNC / HSYNC / FS / LS vocabulary and sync-packet behaviour
NXP IMXRT1170 reference manual	CSI peripheral EOF / SOF interrupt capability
OmniVision OV5640 datasheet	720p@60fps via 2×2 binning; available modes per resolution
Armbian forum topic 4688 — “OV5640 on OPI0+: How to reach higher fps and color spaces”	anecdotal evidence on OV5640 AEC/AGC post-resume behaviour
Adafruit OV5640 breakout documentation	FSIN slave-sync pin availability
Bewley et al., <i>SORT</i> , 2016	tracking-by-detection association baseline, referenced also in relatedwork.md
Zhang et al., <i>ByteTrack</i> , 2022	low-score detection recovery in tracking

The full BibTeX entries live in the existing [references.bib](#).

9. Experimental Setup

This chapter describes once, canonically, the hardware, firmware, software and measurement tooling used by every experiment reported in this paper. Chapters that present a specific optimisation — the SDRAM-to-tensor eDMA memcpy ([memcpy.md](#)) and the glitch-free dual-camera switch ([cam_switch.md](#)) — refer back to this description rather than repeating it. The same configuration is used end-to-end across all reported sessions; departures are flagged explicitly in the individual result tables.

10. 1. Hardware platform

The device under test is a Coral Dev Board Micro carrying the custom-designed **SentAI** daughter-board revision v1.0. The relevant subsystems of this stack are summarised below.

Subsystem	Component	Parameters
MCU	NXP i.MX RT1176 crossover	Cortex-M7 @ 800 MHz + Cortex-M4 co-processor (M4 is idle throughout our measurements)
On-chip AI accelerator	Google EdgeTPU	internal USB 2.0 bus, Mode 3 (kMax) requested by <code>sentai.tpu.load()</code>
External SDRAM	On-module SEMC SDR-SDRAM	16 MB @ 166 MHz, accessible via the SEMC controller
On-chip memory	OCRAM / DTCM / ITCM	1.25 MB / 256 KB / 256 KB
External flash	Octal-SPI NAND + NOR	NAND hosts the LittleFS user partition (firmware + /diags/ data + /lib/diag/ package)
Camera 0 (front)	OV5640-based coralmicro module	1280×720 native, 30 fps streaming (see §3)
Camera 1 (back)	OV5640-based coralmicro module	1280×720 native, 30 fps streaming
Camera MUX	Analogue switch on the shared MIPI-CSI2 lane	GPIO-controlled (kCamMux); GPIO polarity canonicalised in cam_mux.h
Host link	USB-C to Linux workstation	CDC-ACM (REPL) + CDC-NCM (IP 10.0.0.1) simultaneously; see usb.md
Power	USB-C bus-powered	no separate supply; thermal envelope within the 3-second experiment windows

The only dual-sensor hardware constraint that matters for interpretation: **the two OV5640 sensors share a single MIPI-CSI2 lane through an analogue MUX**. There is no hardware sync pin wired between them (no FSIN master/slave), so their internal frame clocks drift independently. Every measurement of “switch cost” in this work is a measurement against *that* topology, not against a two-receiver SoC.

11. 2. Firmware stack

Component	Version / parameters
Firmware project	examples/sentai_runtime/, build #640+ at the time of the final post-refactor session (s045_e18_post_refactor)
Linker script	MIMXRT1176xxxxx_cm7_ram_mp.ld (text and rodata in flash, working set in SDRAM)
RTOS	FreeRTOS 10.x (CMSIS M7 build, 1 ms tick)

Component	Version / parameters
USB stack	NXP USB device driver, CDC-ACM + CDC-NCM simultaneously (MSC only in “storage mode”; see usb.md)
HTTP server	lwIP httpd with custom <code>FsOpenCustom / FsCloseCustom</code> in sentai_httpd.cc ; LS requests route through a dedicated <code>lfs_task</code> — see lfs.md §4.2 build #633 for the rationale
MicroPython runtime	stable embed port under <code>third_party/micropython/ports/embed</code> ; GC heap 512 KB in <code>.sdram_bss</code>
EdgeTPU driver	Coral EdgeTpuManager with per-model package cache
Camera driver	NXP <code>fs1_ov5640.c</code> + <code>camera.cc</code> , with the custom additions documented in cam_switch.md (flip-on-EOF ISR, ratio scheduler)
Watchdog	WDOG1 hardware @ 30 s; software activity watchdog with 60 s warn / 120 s dead thresholds — watchdog.md
Persistent diagnostic store	LittleFS user partition, <code>/diags/</code> and <code>/lib/diag/</code> subtrees

The firmware image for a reported result is identified by `build_version.h` and reflected in every session’s `summary.txt` (implicit via build date) and in the session name suffix we chose at the time (e.g. `_post_refactor`). Major build transitions called out in the results tables:

Transition	Build	Effect
Pre-eDMA CPU memcpy	#584 or earlier	baseline used as the left-hand side of memcpy.md tables
eDMA memcpy merged	#622	<code>sentai.pipeline.dma_memcpy(0/1)</code> A/B flag available at runtime
Fix A (atomic snapshot)	#632	cam_switch.md §“Fix A”
Fix B (flip-on-EOF, 30 fps, ratio scheduler)	#635	cam_switch.md §“Fix B”
LS slow-path (reset-loop fix)	#633	lfs.md §4.2
NASA/JPL review refactor (A1-A7, B1-B4)	#640	cam_switch.md §“Fix B post-review”

12. 3. Sensor configuration

12.1. 3.1 Native mode

Both OV5640s are driven at **1280×720 @ 30 fps** in MIPI-CSI2 mode, with the PLL lookup at `s_ov5640MipiClockConfigs[resolution=720P, framePerSec=30]`:

```
pllCtrl1 = 0x21    // SYSTEM_CLK_DIV = 2
```



```
pllCtrl12 = 0x54    // PLL multiplier = 84
vfifoCtrl10C = 0x20
pclkDiv = 0x04
pclkPeriod = 0x0a
```

 tHsSettle in the CSI2RX is set to 0x12 — the NXP-recommended value for this (resolution, fps) pair. Attempts to push to 45 fps (not in the NXP driver lookup) or 60 fps (PLL accepted, CSI2RX did not lock) were reverted and are documented in the comment block of `libs/camera/camera_support.h`.

12.2. 3.2 Logical resolution

 The host application requests a logical resolution via `sentai.camera.set_resolution(w_h) + sentai.camera.init(1)`. The firmware reconfigures the CSI receiver accordingly. Supported pairs are the union of the NXP driver lookup and the `sentai` wrapper: 720p, 1080p (limited mode), 640×480 (VGA), 320×240 (QVGA), plus the custom 512×512 target used by the TPU model. Every experiment's `.txt` descriptor records the resolution active during that run.

12.3. 3.3 Rotation

`cam0` is configured with a 180° rotation (`sentai.camera.rotate(0, 180)`) via OV5640 MIRROR H/V registers at sensor-init time. `cam1` is unrotated. Rotation does not change per-frame cost.

13. 4. Vision model under test

 The single-class 512×512 model used for every E15-E18 session is:

Property	Value
File	<code>/yolo_1_class_512_1_upsample_512_inloc_de_1</code>
On-flash size	5 591 680 B (5.33 MB)
Input tensor	<code>uint8[1, 512, 512, 3]</code> — 786 432 B, the buffer at the centre of the eDMA memcpy optimisation
Input quantisation	scale = 1/255, zero_point = 0
Output tensor	<code>uint8[1, 1344, 6]</code> — 8 064 B — YOLOv5-enhanced anchor format
Output quantisation	scale = 1/255, zero_point = 0
Output row layout	<code>[cx, cy, w, h, obj_conf, class_conf]</code> (normalised 0..1)
Architecture	YOLOv5 enhanced, single-upsample at P5/32, 1-class head
TFLite arena	799 KB used / 8 192 KB available
EdgeTPU mode	3 (kMax), fully compiled for the accelerator (no CPU fallback)

 The earlier pipeline-bring-up sessions (E1-E14 groundwork, listed in Appendix A of [experiments/README.md](#)) used `yolo26n.edgetpu_1.tflite` — an 80-class model at 320×320 that partially falls back to CPU. Those sessions are baseline evidence for the ~6.4 FPS floor; they are not presented as final performance numbers.

14. 5. Scene

All camera-related measurements were captured with a static scene: two lilac flower arrangements against a pale wall, with a teal ceramic mug in the upper-right of cam0's field of view. The scene did not contain any object class present in the 1-class YOLOv5-enhanced model, so every frame's `num_detections` is zero. That is deliberate: NMS runs its candidate scan over all 1344 anchors in both the zero- detection and the non-zero case; reporting a zero-detection run isolates the timing contribution of the post-processing pipeline from the confound of variable detection counts.

Every session captured before/after JPEGs from BOTH cameras via `diag.snapshot_both_cams` (see [experiments/methodology.md](#) §5.1), so a reviewer can verify offline that the scene did not drift across a multi-sweep session. Figure~3 shows the canonical scene captured at the start of the final post-refactor session.

[t]0.44

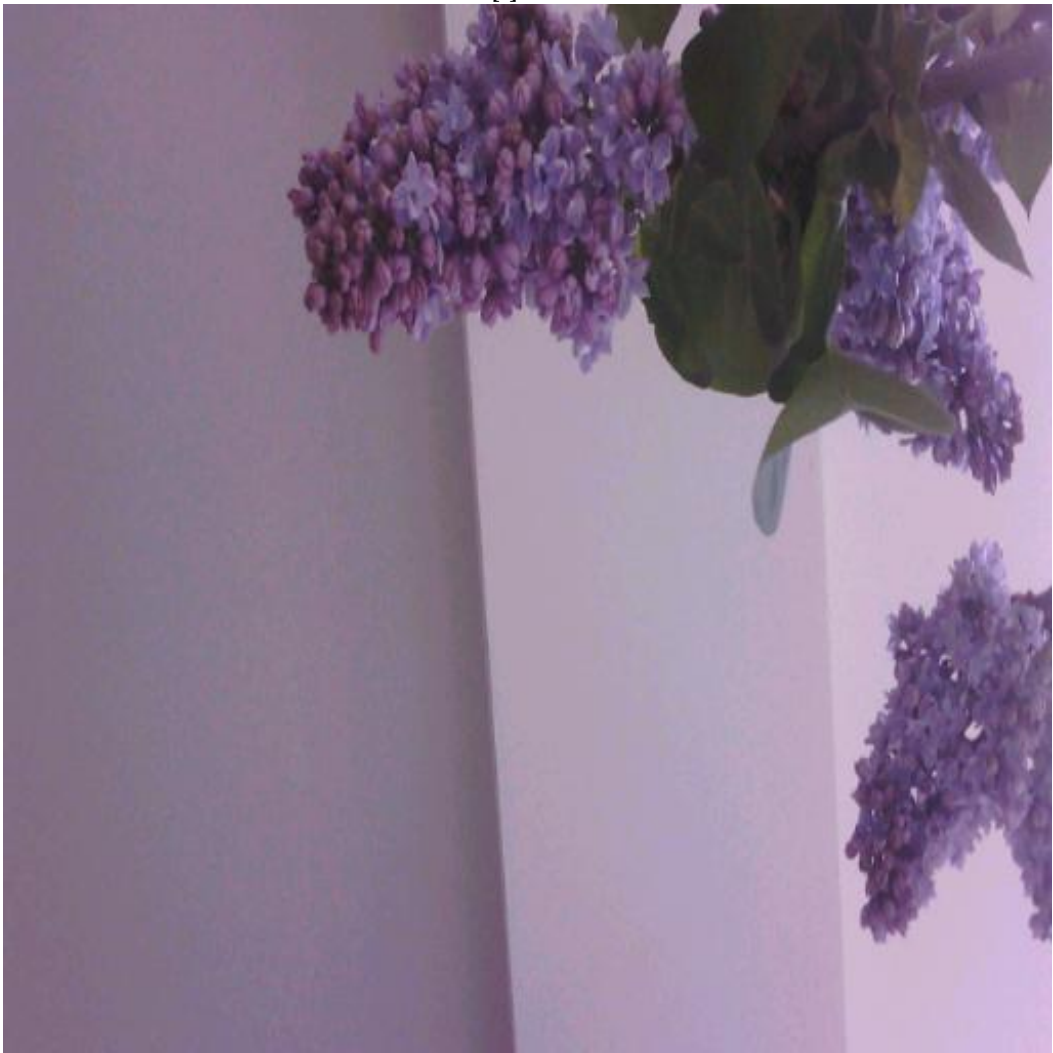
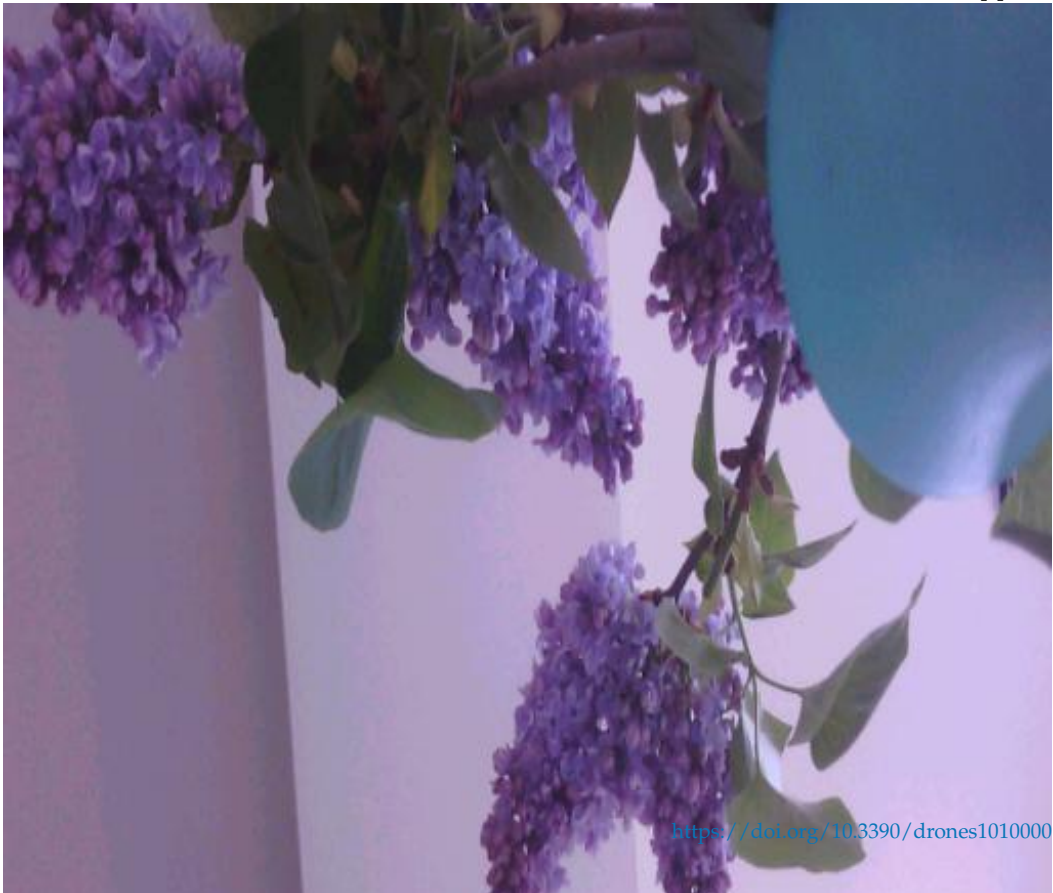


Figure 1. cam0 (front, rotated 180°)

[t]0.44



511

15. 6. Measurement tooling

512

513

15.1. 6.1 On-device: the *diag* package

Experiments are implemented as Python functions in `examples/sentai_runtime/diag/`, invoked from the REPL as `diag.eN_xxx(...)`. Each function:

514
515

- Opens (or inherits) a session
 - Sets a known firmware state (`verbose(0)`, `ratio(0,0)`, `switch_drain(2)`, `pipeline_stopped`, `gc.collect()`)
 - Runs a warm-up block
 - Executes N timed iterations, dropping the first
 - Writes `NNN_<name>.csv` + `NNN_<name>.txt` + records in `manifest.csv`
 - Returns a summary dict
- 516
517
518
519
520
521
522

A list of experiment functions relevant to the performance story:

523

ID	Function	What it measures
E13	<code>e13_pipeline_full</code>	Sequential pipeline at 320×320, per-stage timing
E14	<code>e14_pipeline_parallel</code>	<code>sentai.pipeline.start/get/stop</code> throughput (parallel PrepTask + InferTask)
E15	<code>e15_pipeline_parallel</code>	E14 wrapper for the 512×512 1-class model
E16	<code>e16_camera_switch</code>	Alternating cam0cam1 sequential, per-stage timing
E17	<code>e17_switch_drain_visual</code>	Per-switch JPEG capture (in-RAM buffer; LFS write deferred so timing is clean)
E18	<code>e18_camera_switch_headtail</code>	Three-sweep benchmark in one session: fixed cam0, fixed cam1, alternating

524

15.2. 6.2 Time source

`sentai.rtos.ticks_ms()` — FreeRTOS tick counter exposed to MicroPython. Resolution 1 ms, wrap at 49.7 days (handled by delta subtraction). All stage timings are either this counter (host- visible) or the firmware’s own wrapper around its invoke duration (returned as the integer ms value of `sentai.tpu.invoke()`).

525
526
527
528

529

15.3. 6.3 Statistical tools

Means, stdev (Bessel-corrected), min/max, median are computed offline from the raw CSVs by the appendix generator [experiments/_build_appendix.py](#). The device never reports a summary statistic except the short "mean=X fps=Y" string in each manifest row, and that is informational only — the numbers that appear in paper tables are always re-derived from the raw per-iteration CSVs.

530
531
532
533
534

See [experiments/methodology.md](#) for the full protocol (warm-up drop, repetition count rationale, reproducibility criteria, noise budget). The short form is: every reported number is the mean of n 19 independent iterations with the first sample dropped; sessions are self-contained; the same scene, model, and firmware build underlie any cross-session comparison.

535
536
537
538
539

540

541

16. 7. Host environment

Component	Value
OS	Linux 6.17 (kernel), Ubuntu-derived userland
USB stack	cdc_acm + cdc_ncm kernel modules, MSC via littlefs-fuse
IP	USB-NCM: host 10.0.0.N/24, device 10.0.0.1
Python	3.12, pyserial for REPL-driven uploads
Build	CMake 3.x + Ninja/Make, arm-none-eabi-gcc 10+
Firmware flash	python3 scripts/flashtool.py -e sentai_runtime
Result fetch	HTTP GET from http://10.0.0.1/api/raw/diags/... with lfs_busy retry

The exact host details matter only insofar as they influence measurement access (e.g. the REPL-chunked uploader at `diag/_host_upload_repl.py` exists because HTTP POST to `/api/write/` hangs on this firmware). None of the performance numbers reported in this paper depend on host scheduling or host I/O; they are all measured on-device and fetched post-hoc.

547

548

17. 8. Summary of what changes between experiments, and what does not

A reader comparing two sessions should know that the following are **held constant** across every E15-E18 result in this paper unless a section explicitly notes a departure:

- Hardware (same SentAI board unit, no swap of camera modules, no physical rework)
- OV5640 PLL and sensor-init register sequences (same NXP driver lookup)
- Scene (static lilac arrangement, no lighting changes)
- Model (`yolo_1_class_512_..._P5_32.tflite`, compiled for EdgeTPU Mode 3)
- Logical resolution requested (512×512 unless the section says VGA or QVGA)
- Warm-up protocol (one full cycle per camera before the measured loop)
- Warm-up sample drop (first measurement sample discarded from stats)
- Fault-counter check (post-run `sentai.diag.cam_stats()` values are recorded; a non-zero degraded-path counter invalidates a session)

What *does* change between the sessions the paper compares:

- The firmware build (documented per-section in the cross-session comparison tables — e.g. `cam_switch.md` §“Cross- session comparison”)
- The A/B runtime flag under test (`dma_memcpy(0/1)`, `switch_drain(1/2)`, `ratio(0,0) / (3,1) / (9,1)`)
- The chosen alternation pattern (fixed cam0, fixed cam1, or alternating — the three sweeps of E18)

A reviewer verifying a specific claim can thus navigate directly: the numeric claim → the section that presents it → the session name → the folder under `../experiments/` with the raw CSV. The claim-to-file mapping is tabulated in [artifact.md](#).

18. Materials and Methods

This chapter describes the implementation of the platform as a research system positioned at the intersection of embedded AI acceleration, real-time systems, and autonomous drone control. The central thesis of the implementation is that an EdgeTPU-class accelerator is simultaneously low-power and computationally powerful enough to justify a much tighter integration with an MCU than is common in larger companion-computer architectures. Rather than treating the accelerator as a peripheral attached to a heavy host, the design treats the MCU and EdgeTPU as a compact co-processing pair: the MCU handles sensing, scheduling, memory orchestration, communication, and high-level control, while the EdgeTPU is driven as the main execution substrate for neural inference.

An equally important framing point is that the implementation starts from the original Coral Micro board from which the design started and deliberately pushes it toward industrialization. In other words, the work is not only about extending that open-source base with additional capabilities, but about transforming it into our MCU-EdgeTPU board, a system that exhibits more production-oriented behavior: higher sustained throughput, better control of memory and latency, richer sensor integration, clearer runtime abstractions, and a software structure that can support repeated deployment rather than isolated demonstrations.

The implementation presented in this work differs substantially from the initial fork baseline by transforming a board-oriented demo environment into a programmable autonomous-flight runtime. The main engineering question is not only whether inference can run onboard, but how to maximize accelerator utilization, sustain real-time responsiveness, and preserve a lightweight and low-power hardware profile suitable for micro-UAV operation. This leads to a design in which memory placement, transfer scheduling, camera integration, scripting boundaries, and telemetry abstractions become first-order implementation concerns. In this sense, the chapter documents a move from open-source proof-of-potential to an embedded system engineered with production-performance objectives in mind.

The chapter is organized into the following subchapters:

1. [implementation_01_system_architecture.md](#), which explains the hardware-software co-design and the motivation for pairing an MCU with an EdgeTPU on a lightweight drone-oriented board.
2. [implementation_02_runtime_and_memory.md](#), which analyzes the RTOS integration, linker-level memory layout, staging buffers, and the host-side logic required to keep the accelerator fed efficiently.
3. [implementation_03_vision_pipeline.md](#), which details the dual-camera subsystem, the real-time image path, tracking, projection, and the deployment of YOLO-class detectors.
4. [implementation_04_flight_and_telemetry.md](#), which presents the dual-target control strategy spanning Crazyflie and PX4/MAVLink, together with telemetry and backend integration.
5. [implementation_05_programming_model.md](#), which studies the MicroPython layer and uses the platform help system as evidence of how the embedded capabilities are exposed as a coherent experimental interface.

Taken together, these sections argue that the implementation is best understood as an exercise in practical embedded-AI industrialization: the work does not merely

deploy neural inference on a small board, but systematically resolves the hardware and software bottlenecks that arise when such inference must coexist with flight control, camera switching, runtime programmability, and constrained memory resources. The overall contribution is therefore to show how an open-source platform can be matured toward a more production-capable embedded autonomy stack without abandoning its accessibility, compactness, or low-power design philosophy.

18.1. System Architecture and Design Rationale

Figure 4 summarizes the layered view of the platform, from sensing hardware and RTOS orchestration to EdgeTPU inference and the MicroPython experimentation surface.

System Architecture and Design Rationale

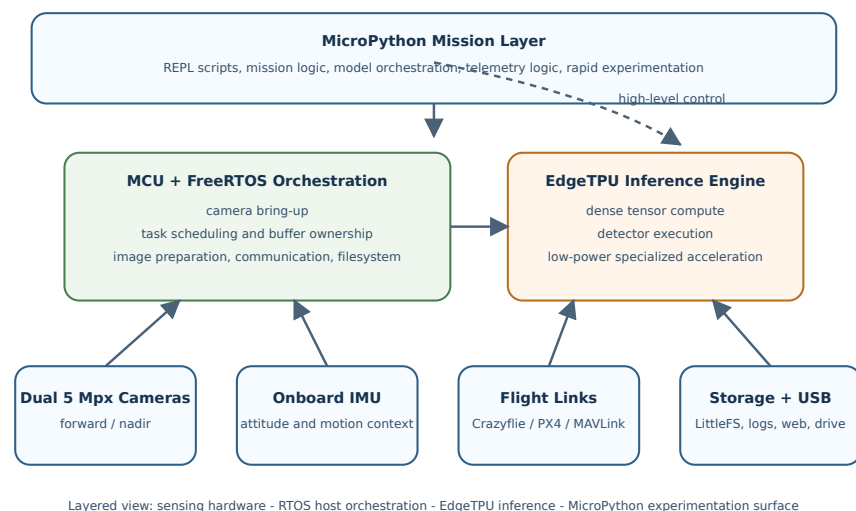


Figure 4. Block-level view of the platform architecture, highlighting the split between sensing hardware, MCU orchestration, EdgeTPU inference, and the script-oriented MicroPython layer.

The implementation starts from a strong architectural hypothesis: for lightweight autonomous drones, the most useful computing arrangement is not a large companion computer with a small flight controller attached to it, but a tightly coupled MCU-plus-accelerator system in which the accelerator performs the dense tensor computation and the MCU orchestrates everything else. This view is consistent with broader trends in domain-specific architectures for neural networks, especially the systolic-array lineage popularized by TPU-style accelerators, where high arithmetic density and low energy per operation make specialized inference hardware much more attractive than general-purpose processing for continuously running perception tasks.

In this work, the EdgeTPU is treated as the primary neural execution engine and the MCU as the real-time systems controller that makes this engine usable in practice. The MCU is responsible for sensor bring-up, camera switching, image scaling and quantization setup, task scheduling under FreeRTOS, telemetry transport, filesystem access, and the high-level control hooks exposed to the drone. The choice of a real-time operating system, rather than a Linux-based software stack, is deliberate. For this class of airborne system, FreeRTOS offers lower software overhead, tighter control over scheduling and memory, faster boot behavior, simpler timing analysis, and a substantially smaller energy and storage footprint than a full operating-system distribution. This division of responsibility is central to the implementation strategy. It avoids the weight and current draw associated with

larger embedded AI platforms while still enabling onboard deep-learning inference for navigation and scene understanding.

This design choice is particularly important in the drone context. Platforms such as Coral Dev Board or Jetson-class systems are powerful, but their mass, thermal requirements, battery burden, and system complexity make them difficult to justify for small indoor drones or for compact airframes where endurance and safety matter. Their software model is also usually tied to a Linux-class environment, which is convenient for development but less attractive when deterministic timing, reduced boot latency, and strict power discipline are required onboard a lightweight UAV. By contrast, a minimalist EdgeTPU-enabled board can remain lightweight and power-conscious, yet still sustain meaningful onboard perception if the software stack is built around the accelerator and an RTOS-controlled MCU rather than around the comfort assumptions of a desktop-like host. The implementation therefore argues that the EdgeTPU is not merely a convenient inference add-on, but a sufficiently strong low-power compute primitive to be integrated beside virtually any capable MCU, provided that memory and data movement are engineered with care.

The concrete system extends the minimal board profile with two 5 Mpx cameras and an onboard IMU. This is a deliberate departure from lower-resolution single-camera operating modes commonly found in embedded vision examples. The intent is to preserve more visual detail at the sensing stage, even when the final detector input is smaller, because high-resolution acquisition creates more room for strategic cropping, scaling, model switching, and camera-role specialization. In practical terms, this matters for YOLO-class detectors, which are sensitive to input detail and often benefit from larger or better-preserved spatial structure before quantization.

The two-camera design also reflects a broader systems argument. A single fixed camera is often enough for a benchmark, but not always enough for a drone mission. Indoor navigation, target localization, regulatory awareness, and scene understanding may require forward-looking and nadir-looking views, or rapid switching between complementary perspectives. The implementation supports this by treating the dual-camera subsystem as part of the platform architecture, not as an optional accessory. Camera identity, mount geometry, and active-view selection are propagated all the way to the tracking and projection layers.

At the software level, the architecture is also shaped by programmability. Rather than exposing the platform only through C++ firmware entry points, the system embeds a full MicroPython runtime and binds the major capabilities of the board into a structured `sentai` module. This follows the intuition that autonomous flight research evolves too quickly for every experimental change to require a full firmware rewrite. High-level mission logic, model orchestration, telemetry handling, and flight behaviors are therefore scripted, while the timing-critical path remains implemented in compiled code.

This leads to a deliberate split between two programming models. At the lower level, the firmware is task-oriented in C++: camera acquisition, preprocessing, communication, tracking, and control services are organized as explicit FreeRTOS tasks, queues, semaphores, and singleton runtime components with well-defined ownership over buffers and peripherals. At the upper level, the platform becomes script-oriented in MicroPython: the user composes these services through short scripts, REPL commands, and boot-time logic without needing to manage the underlying task graph directly. The purpose of this split is not only convenience, but architectural clarity. C++ remains responsible for determinism, concurrency, and performance-critical resource control, while MicroPython remains responsible for orchestration, experimentation, and rapid reconfiguration of behavior.

In this way, the implementation combines the rapid iteration benefits associated with embedded scripting with the deterministic execution required by real-time perception.

The resulting system should thus be understood as a co-designed stack with four tightly linked layers: hardware sensing and capture, real-time host orchestration on the MCU, neural inference on the EdgeTPU, and a high-level control and experimentation surface in MicroPython. This layered organization is what makes the platform useful as a research artifact rather than merely as a single-purpose demo.

18.2. RTOS Integration, Memory Layout, and Accelerator Feeding

Figure 5 shows the runtime pipeline used to keep the accelerator fed, with explicit staging, task handoff, and memory placement.

RTOS Integration, Memory Layout, and Accelerator Feeding

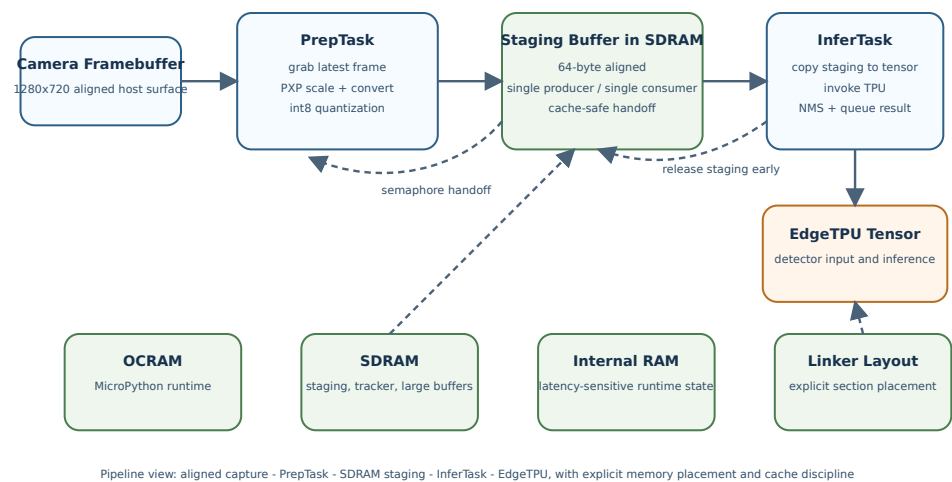


Figure 5. Block diagram of the RTOS and memory pipeline, showing aligned capture buffers, PrepTask, SDRAM staging, InferTask, and the EdgeTPU tensor path.

One of the main implementation claims of this work is that achieving strong onboard vision performance on a compact MCU-plus-EdgeTPU platform depends less on raw inference speed alone and more on how efficiently the host prepares, places, and transfers data. This is particularly visible in the code added relative to the initial fork, where the runtime evolves from a board example into a carefully staged real-time system.

The core execution environment is FreeRTOS. Rather than running perception as a single sequential loop, the implementation decomposes the workload into cooperating tasks with explicit ownership boundaries. The most relevant example is the continuous detection pipeline, where the code in `detection_task.cc` organizes the system around a preparation task and an inference task. The preparation task acquires the latest frame, scales and converts it through the hardware pixel pipeline, applies quantization when needed, and writes the result into a staging buffer. The inference task copies the prepared staging contents into the EdgeTPU input tensor, releases the staging buffer immediately, and then executes inference and post-processing while preparation for the next frame can already proceed. This design is not simply an optimization; it is the mechanism through which the implementation turns limited memory and compute resources into sustained throughput.

An important low-level detail is that the runtime does not feed the detector directly from the raw camera bus format. The active camera configuration captures native 1280x720 frames into 64-byte-aligned framebuffers with a 4-byte-per-pixel host representation. In practice, the software treats these buffers as XRGB8888-like surfaces because this layout is

easier to move through CSI DMA, cache maintenance, and the RT1176 pixel pipeline than a tightly packed 3-byte RGB tensor. This is precisely where the PXP block becomes essential rather than optional: it converts the aligned capture surface into RGB888 output at the detector input size, eliminating expensive per-pixel software shuffles on the Cortex-M7.

This organization also clarifies the broader software split introduced in the architecture chapter. The lower implementation layer is task-oriented in C++, meaning that concurrency is expressed directly through FreeRTOS tasks, semaphores, queues, and explicit buffer handoff between producer and consumer stages. By contrast, the upper user layer remains script-oriented in MicroPython, where these same services are invoked as high-level operations without exposing the user to the internal synchronization structure. The runtime and memory subsystem is precisely where this separation becomes most meaningful: scripts can request model loading, camera capture, or continuous detection, but the correctness and performance of those operations depend on the task-oriented C++ layer maintaining deterministic ownership over memory, timing, and accelerator access.

The use of double-buffered staging is especially important because it enforces a clean ownership discipline between producer and consumer stages. The staging buffer is written by the preparation side and read by the inference side under semaphore-controlled handoff, while the TPU tensor is written only when the staging contents are ready. This minimizes race conditions, reduces needless synchronization complexity, and preserves the ability to overlap work. In a larger system with abundant memory, one could compensate for poor orchestration by brute force. In the present implementation, orchestration is the only realistic way to maintain high frame rate while preserving the low-power character of the platform.

The same section of code also shows why byte alignment becomes a first-order systems concern. The staging area in SDRAM is explicitly aligned to cache boundaries, while the PXP conversion path performs cache clean-and-invalidate operations before DMA writes and invalidation after completion so that stale boundary lines do not overwrite fresh image data. This is needed because the TFLite arena is not naturally aligned to the same cache-line granularity as the DMA engine. Without that discipline, a seemingly minor mismatch between 16-byte tensor allocation and 32-byte cache-line behavior would produce intermittent corruption exactly at the stage where the detector input is assembled.

The memory layout reflects the same philosophy. The linker script introduces a deliberate partitioning across internal memory, OCRAM, heap, SDRAM, and a dedicated region for camera-related traffic. MicroPython is placed explicitly in OCRAM rather than in the most latency-sensitive memory regions, while large buffers, tracker state, and staging areas are pushed into SDRAM where capacity matters more than absolute access latency. This is visible both in the custom linker script and in the pervasive use of explicit section placement such as `.sdram_bss`. The implementation thus accepts that not all memory is equal and uses the linker as part of the optimization strategy rather than as a passive build artifact.

This memory strategy also explains a design choice that would otherwise look surprising from a purely model-centric viewpoint: the system first tolerates a larger, alignment-friendly framebuffer representation and only later collapses it into the compact RGB tensor expected by the model. In other words, the runtime pays for structure before it pays for compactness. That tradeoff is justified because aligned DMA and deterministic hardware conversion are cheaper on this platform than repeatedly manipulating tightly packed pixels in software.

This issue becomes more acute once YOLO-class detectors are considered. Larger detector inputs significantly increase the pressure on staging buffers, copied tensors, post-processing data, and runtime metadata. In a conventional desktop pipeline this is almost

invisible. On an MCU platform it becomes decisive. The code therefore embodies a memory-rearrangement strategy aimed at fitting the MicroPython runtime, the real-time tasks, the tracker structures, and the camera-to-TPU pipeline into a workable whole while minimizing inefficient copies. This is one of the places where the implementation differs most strongly from a conceptual prototype: the problem is no longer simply to run a model, but to run it without collapsing the rest of the system.

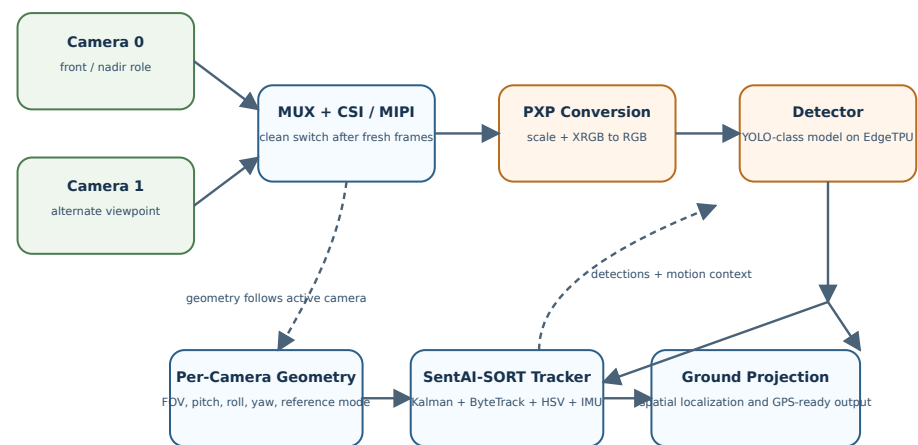
The same logic informs the build system. The custom CMake configuration aggregates a full embedded MicroPython build, nanopb-generated protocol code, additional audio and communication components, and the custom runtime sources into a single executable with a purpose-built linker script. This makes the firmware itself part of the experimental method. The implementation is not a loose collection of demos, but a custom-built runtime whose compilation model, memory map, and dependencies are aligned with the targeted deployment conditions.

From a research perspective, this subsection supports a broader claim: on compact embedded AI platforms, the practical efficiency of a specialized accelerator depends on the host system's ability to keep the accelerator supplied with valid input at the right pace. The implementation therefore treats memory organization and RTOS scheduling as intrinsic parts of accelerator utilization rather than as secondary engineering details.

18.3. Dual-Camera Vision Pipeline and Perception Stack

Figure 6 illustrates how the two cameras, preprocessing path, detector, tracker, and geometry-aware projection stages interact inside the perception stack.

Dual-Camera Vision Pipeline and Perception Stack



Perception view: dual sensors - clean camera switching - PXP preprocessing - EdgeTPU detection - tracking - geometry-aware projection

Figure 6. Dual-camera perception pipeline with camera switching, PXP preprocessing, EdgeTPU detection, tracking, and ground-aware projection.

The perception implementation is built around the idea that high-quality airborne vision is constrained as much by sensing geometry and data movement as by the neural model itself. For this reason, the codebase extends the original single-board camera model into a dual-camera system with explicit support for runtime switching, per-camera geometry, and high-resolution acquisition.

At the capture level, the cameras are integrated through the processor camera subsystem and the MIPI CSI-2 path. This is an important architectural choice because it prevents the image path from becoming dominated by the kind of software-managed peripheral

transfer overhead that would be unacceptable for sustained real-time inference. The camera stack is designed so that CSI and MIPI remain active even during switching, which reduces disruption and helps preserve throughput. In practical terms, the runtime exposes `sentai.camera.switch(id)` while internally snapshotting the frame sequence and waiting until enough fresh frames have arrived from the newly selected sensor to guarantee a clean image. This is a small but important example of the thesis that real embedded autonomy depends on disciplined treatment of edge cases that benchmark-oriented prototypes often ignore.

The OV5640 integration itself is more substantial than a generic sensor bring-up. Register control is handled over SCCB through the board I2C interfaces, including model-ID verification, reset and power-down sequencing, per-camera mirror and rotation adjustments, and sensor-specific initialization on each side of the camera multiplexer. On the video side, the datasheet-level interface still matters to the architecture: the sensor family supports compact formats such as 8-bit RGB565, but the host capture path on the RT1176 is engineered around aligned 32-bit framebuffers and DMA-friendly pitches. As a result, the implementation deliberately separates transport format concerns from model-input format concerns instead of assuming that the detector can consume the capture representation directly.

The implementation also recognizes that switching cameras is not useful unless geometry travels with the switch. The perception pipeline therefore maintains per-camera configuration for field of view, mount pitch, mount roll, mount yaw, and ground-reference mode. This allows the same runtime to support parallel cameras, perpendicular camera placement, nadir sensing, or oblique views without hard-coding assumptions about the sensor pose. In aerial robotics this is essential because the operational meaning of a detection depends on how the camera is mounted, not only on where the pixels appear.

The neural perception path itself is intentionally deployment-aware. Frames are captured at the sensor side, then scaled and converted through the hardware pixel pipeline before being prepared for the model input tensor. This means the system can begin from richer camera imagery, including workflows derived from native 1280-wide sensing, while still feeding smaller quantized tensors to the detector. Such a path is crucial for compact YOLO-family models, which often live in tension between the need for more spatial detail and the strict memory limits of MCU deployment.

One fine but important engineering point is that the PXP stage is not used only for resizing. The camera receiver exposes raw host buffers whose layout is convenient for aligned capture, cache maintenance, and bus transfer, but not for direct detector input. The runtime therefore uses PXP to transform XRGB-style capture surfaces into packed RGB output with the exact detector dimensions, and then quantizes in place if the TPU model expects int8 input. This avoids repeated software byte shuffling, avoids ambiguities caused by aligned scanline storage, and turns pixel-format conversion into a deterministic hardware stage of the pipeline.

Above the detector, the implementation adds a custom tracking layer referred to in code as SentAI-SORT. The design borrows the logic of modern tracking-by-detection systems while adapting it to the realities of a Cortex-M7-class host. The tracker combines an 8-state Kalman formulation, a ByteTrack-style two-stage association process, and compact HSV histogram features. Instead of relying on expensive feature-based motion compensation, it uses IMU-informed compensation derived from camera motion. This substitution is not only computationally cheaper, but also better aligned with the sensing resources already present on the platform.

It is useful to state explicitly how this tracker differs from established baselines. Relative to the original SORT formulation [13], the implementation is less minimalist in

association: it does not rely only on predicted box overlap, but recovers lower-confidence detections through a second association stage in the spirit of ByteTrack [14], and it adds a lightweight appearance cue through a compact HSV-grid histogram. Relative to ByteTrack itself, however, the goal here is not leaderboard-oriented MOT performance on a workstation-class host. The implementation removes assumptions that are natural in desktop tracking pipelines but costly on an MCU, such as dependence on richer appearance embeddings, larger temporary buffers, or host-side post-processing flexibility. In practice, SentAI-SORT is best understood as a bounded embedded tracker that keeps the constant-velocity Kalman core and the useful low-confidence recovery logic, but compresses the rest of the design to fit a static-memory, real-time inference loop.

This redesign was necessary for systems reasons rather than for novelty in tracking alone. On this platform, the tracker executes synchronously after detector output, with all state kept in statically allocated memory and with a total footprint small enough to coexist with camera buffers, tensor staging, MicroPython, and the rest of the runtime. The appearance term is therefore reduced to a small histogram representation instead of a learned re-identification branch, and camera motion compensation is driven by the onboard IMU rather than by feature extraction and image-to-image homography estimation. These choices trade some generality for a much tighter computational envelope, but they also exploit information that is naturally available on an airborne embedded system. Just as importantly, the tracker is not an isolated MOT module: it is integrated with per-camera geometry, ground-plane projection, and event export, so that tracked objects can be consumed directly by flight logic, telemetry, or higher-level navigation code rather than remaining only image-space trajectories.

The tracker is further extended with ground-plane projection and optional GPS-referenced localization. Once altitude, heading, IMU attitude, and camera geometry are known, detections can be mapped beyond image coordinates into a spatial frame meaningful for the drone and for downstream consumers. This is particularly important when the runtime is used not only to detect objects, but also to generate position-aware telemetry or to feed higher-level navigation policies.

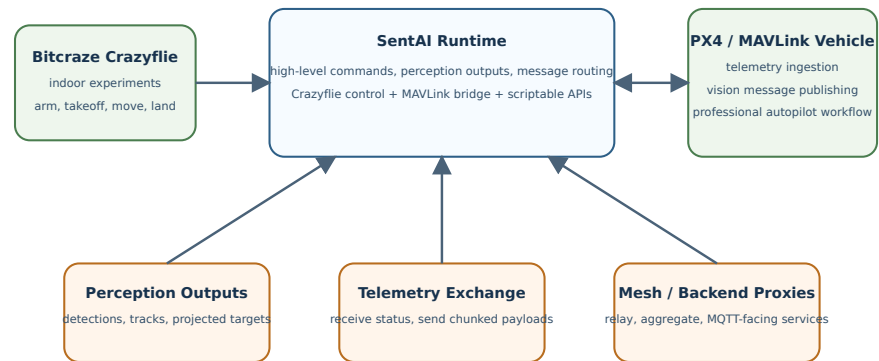
The implementation therefore supports a wider claim than simple onboard detection. It demonstrates that a small embedded AI platform can host a complete airborne perception chain: dual high-resolution sensing, viewpoint switching, quantized detection, lightweight tracking, motion compensation, and spatial projection. In combination with model switching, this also creates the conditions for multi-model operation in which different detectors or visual policies can be applied according to mission phase, camera viewpoint, or scene constraints.

Finally, the perception stack is complemented conceptually by the custom YOLO model and the synthetic open dataset discussed elsewhere in the paper. Although those assets are external to the code implementation, they are consistent with the implementation logic described here: the runtime is engineered so that compact detection models trained for nadir and oblique drone views can be loaded dynamically and used as part of a practical onboard autonomy workflow.

18.4. Flight Control, Telemetry, and External Integration

Figure 7 summarizes the dual-target control model, in which one runtime connects local perception to both Crazyflie and PX4/MAVLink-centered workflows.

Flight Control, Telemetry, and External Integration



Control view: one runtime, two flight targets, bidirectional telemetry, and external dissemination of vision-aware state

Figure 7. Control and telemetry architecture spanning Crazyflie experimentation, MAVLink interoperability, and external backend dissemination.

The implementation does not stop at perception. A central design goal is to make perception actionable in flight-relevant terms, and this is why the runtime introduces two distinct but complementary control paths: one centered on Bitcraze Crazyflie for indoor experimentation and one centered on PX4-compatible communication through MAVLink for a more conventional autopilot ecosystem.

The Crazyflie path is valuable because it gives the research platform an accessible indoor drone target that is lightweight, inexpensive, and safe to iterate on in constrained spaces. In implementation terms, the runtime exposes high-level flight functions for arming, takeoff, landing, attitude control, hover-like motion, waypoint movement, and basic link validation. The significance of this layer is less in the underlying protocol details and more in the abstraction boundary it creates: the experimental logic written in MicroPython can command a real flying platform without descending into firmware-specific implementation code for every new experiment.

The MAVLink path plays a different role. It positions the same perception and scripting environment toward a standard autopilot workflow, enabling connection to ground-control software and to a PX4-style vehicle architecture. In the implementation, the MAVLink bridge is built around the official message library and a dedicated receive task, with queue-based delivery of decoded telemetry into the runtime. This allows the platform to consume motion, position, and status telemetry while also publishing onboard detections and system messages back to the wider vehicle ecosystem. The result is a direct technical bridge between a compact onboard perception unit and a more conventional autopilot stack.

This bridge is not limited to a single control primitive. The runtime exposes both transmission and reception semantics at the scripting layer, including reception of autopilot telemetry, chunked transmission of structured vision payloads that would not fit cleanly into a single small embedded message, and direct generation of PX4-compatible OBSTACLE_DISTANCE messages for collision-prevention workflows. In practical terms, this means the same runtime can send high-level detection events through compact serialized vision messages, or it can transform tracker-ground projections and explicit point sets into the radial obstacle maps expected by a professional autopilot. That detail matters because it

shows the design is meant for sustained exchange between perception and flight software rather than for one-shot demo messages.

The obstacle-reporting path is particularly important because it closes the loop between onboard perception and flight-safety behavior. Instead of treating detections only as information for logging or remote display, the runtime can reinterpret tracked objects as navigational constraints in the vehicle frame and publish them through MAVLink as a dense angular distance map. This is a more operational use of the perception stack than simple text telemetry: the embedded vision system becomes capable of feeding an autopilot-side collision-prevention mechanism with geometry derived from its own tracker or from higher-level mapping outputs such as SLAM landmarks [11,12].

This dual-target approach is important conceptually. Many embedded AI demonstrators either remain tied to a single experimental robot or aim immediately at a professional stack without a low-risk iteration platform. Here, the implementation supports both modes. Crazyflie serves as a rapid prototyping platform for indoor validation, while MAVLink provides compatibility with broader drone workflows. From a research standpoint, this strengthens the portability of the proposed autonomy layer.

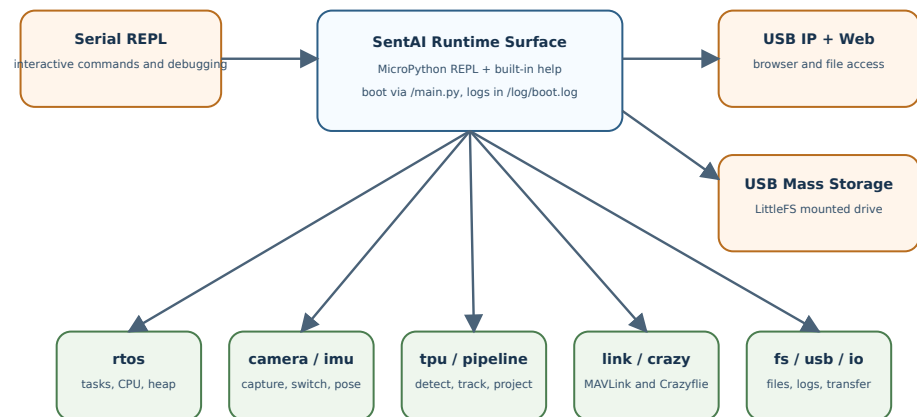
The communication stack extends further through mesh and backend services. The firmware side includes a compact binary vision message protocol and bridges for wireless dissemination, while the repository also adds backend proxy services that relay, aggregate, and expose telemetry through ground-side interfaces such as MQTT. This is significant because it shows that the implementation is intended to operate as part of a distributed autonomy pipeline rather than as an isolated board demo. Detections are not merely computed; they are structured, transported, and made available to external systems.

In implementation terms, the control and communication layers provide the operational dimension of embedded perception. A detector running locally is useful only to a point. A detector that can feed a Crazyflie experiment, inform a MAVLink autopilot context, emit autopilot-consumable obstacle maps, publish compact messages to external services, and remain scriptable from a high-level runtime is more appropriate for systems research than a detector evaluated only in isolation. This is the level at which the runtime begins to resemble an onboard autonomy substrate rather than a perception benchmark harness.

18.5. Programming Model and User-Facing Runtime Surface

Figure 8 provides a compact view of the user-facing runtime model, including the three USB access paths and the main groups of the *sentai* namespace.

Programming Model and User-Facing Runtime Surface



User-facing view: three access modes on USB, one coherent sentai namespace, and script-driven control over sensing, inference, communication, and logs

Figure 8. Programming model view showing REPL-centered access, USB service modes, boot artifacts, and the main runtime modules exposed through the `sentai` namespace.

An implementation of this kind must be evaluated not only by its internal efficiency but also by how coherently it exposes its capabilities to the user. This is especially important on MCU-based platforms, where debugging is often more difficult than on Linux-class systems because the developer has less runtime introspection, fewer interactive tools, and a much narrower margin for trial-and-error when something goes wrong during boot or during a real-time task interaction. For this reason, the platform does not expose a single debugging path, but a small set of complementary connection modes over USB that make the board significantly easier to inspect and operate in practice.

The first mode is an interactive MicroPython REPL available over the serial interface, which allows a researcher or operator to connect directly to the board, inspect the runtime state, execute commands live, load scripts, and request contextual help from within the system itself. This is a consequential design choice: the platform is not only flashed and executed as a fixed firmware image, but can also be interrogated and steered interactively during development and testing. In practice, the REPL becomes the primary entry point for experimentation, rapid diagnosis, and iterative mission design.

The second mode is USB networking over IP, through which the firmware exposes an onboard web server and file-browser interface. This gives the board a lightweight service surface that can be reached directly from the host, making it possible to inspect and transfer artifacts without depending exclusively on the serial console. The third mode is USB mass storage, in which the LittleFS user partition is exposed as a mounted drive so that images, scripts, and models can be uploaded or downloaded directly from the host machine. Taken together, these three USB-facing paths, namely serial REPL, IP connectivity with web access, and mounted LittleFS storage, form a practical answer to the usual debugging and deployment friction of MCU-based systems.

The built-in help interface available from the REPL makes this programming model discoverable without forcing the user to leave the target system. This matters because the runtime is relatively rich: it does not expose a single detector call or a single flight command, but a full namespace of platform primitives intended to support sensing, inference, telemetry, storage, and high-level control from one embedded environment. The same environment also supports script-driven startup through `/main.py`, which is automatically executed at boot before the interactive session begins. This gives the platform an important

dual character: it can behave as an interactive research instrument during development, but it can also behave as a self-starting embedded application once the desired mission logic has been written.

This startup behavior is complemented by a boot logging mechanism implemented directly in the runtime. During initialization, console and driver output are captured into `/log/boot.log`, while the previous boot log is rotated to `/log/boot_old.log`. This is an important practical feature for embedded debugging, because many failures on an MCU platform occur before a user can attach interactively to the REPL. By preserving the initialization trace on LittleFS, the system allows the user to inspect the outcome of FreeRTOS bring-up, early peripheral initialization, and startup script execution even after the board has already moved past the failing point.

The `sentai` namespace now organizes the platform into a broader set of functional modules than the original runtime surface, including `io`, `rtos`, `tpu`, `tfl`, `fs`, `camera`, `imu`, `mic`, `usb`, `uart`, `mesh`, `link`, `crazy`, `sleep`, `pipeline`, `aifes`, `kmeans`, `pca`, `anomaly`, `dtw`, `hmm`, `r1`, and `slam`. This structure is significant because it reveals the intended user abstraction: the operator is not expected to manipulate a collection of unrelated firmware endpoints, but to work with a coherent runtime in which device functions are grouped by task and by experimental purpose. The platform can query RTOS state, inspect heap usage, load EdgeTPU or CPU-side TensorFlow Lite models, capture or switch cameras, read inertial measurements, record audio, enter low-power idle states, exchange data over USB, UART, Meshtastic, or MAVLink, start a continuous detector, track objects, build simple online models, or command a drone, all through a single embedded language environment.

This helps explain why MicroPython is a central implementation choice rather than a peripheral convenience. The interpreter does not replace the lower-level real-time system; instead, it defines a stable experimentation layer above it. The help text demonstrates this duality clearly. On one hand, users access simple operations such as `sentai.camera.to_tensor()`, `sentai.tpu.invoke()`, or `sentai.tfl.invoke()` depending on whether inference is mapped to the EdgeTPU or kept on the MCU. On the other hand, the same interface reaches into advanced platform features such as tracking events, per-camera geometry, ground-plane projection, obstacle reporting, and MAVLink message exchange. The implementation thereby condenses a complex firmware system into a form that is operable from concise Python scripts.

Several modules are especially important for understanding the research value of the platform. The `rtos` module exposes runtime observability primitives such as task inspection, CPU statistics, and heap information, which are essential when the system is tuned for sustained real-time throughput. The `tpu` module provides the basic EdgeTPU inference control plane: model loading, tensor access, invocation, quantization metadata, and detector-oriented post-processing. The newer `tfl` module extends this surface with CPU-side TensorFlow Lite Micro inference for models that are not compiled for the EdgeTPU, which means the runtime can now host both accelerator-backed and pure-MCU inference paths under a common scripting model. The `camera` module complements these inference interfaces by managing capture, JPEG export, transfer to the tensor path, runtime camera switching, and frame sequencing. The `imu` and `mic` modules supply the inertial and audio measurements needed by viewpoint-aware tracking, sound-triggered wake-up, and non-visual experiments. The `fs`, `usb`, and `uart` modules support a practical workflow in which models, scripts, captured outputs, and debugging artifacts can be transferred to and from the device without rebuilding the firmware for every iteration.

The additional machine-learning namespaces are also relevant because they extend the runtime beyond a detector-and-flight stack into a broader embedded inference and experimentation environment. The `aifes` module exposes on-device neural-network train-

ing and inference through the AIFES library [15]. The `kmeans`, `pca`, and `anomaly` modules add compact methods for clustering, dimensionality reduction, and anomaly scoring that are well suited to MCU-class deployment because they impose limited model overheads [1–4]. The `dtw` and `hmm` modules extend the runtime toward sequence modeling for gesture, audio, and other short temporal patterns [5,6], while the `r1` module adds adaptive control mechanisms spanning tabular Q-learning, bandits, and a compact DQN-style multilayer perceptron [7–10]. Finally, the `slam` module exposes a detection-driven EKF-SLAM interface in which object detections can be promoted from transient image measurements into a persistent two-dimensional map [11,12]. Together, these additions extend the role of the runtime: it is not only a way to load and invoke models compiled for the accelerator, but also a compact collection of integrated ML primitives that can be composed directly from MicroPython for perception, adaptation, and state-estimation experiments.

The help surface also shows that these capabilities were designed to be exercised directly from the target rather than only through host-side tooling. Typical examples include immediate tensor feeding from the camera, live detector startup, autopilot connection, or direct Crazyflie flight commands, for example:

```
sentai.camera.to_tensor()
sentai.tpu.invoke()
print(sentai.tpu.detect())

sentai.link.init(57600)
sentai.pipeline.start(0.3, 0.45, 50, True)
while True:
    msg = sentai.link.receive()
    if msg:
        print(msg)

sentai.crazy.init()
sentai.crazy.arm()
sentai.crazy.fly(0.3, 3000)
sentai.crazy.fly_stop()
```

At the control and autonomy layer, `mesh`, `link`, `crazy`, `sleep`, and `pipeline` form the most distinctive part of the namespace. The `mesh` module exposes Meshtastic-compatible communication as a first-class runtime capability for low-bandwidth distributed sensing and message dissemination. The `link` module does the same for MAVLink communication, allowing the board to exchange telemetry and structured messages with an autopilot or a ground-control system. The `crazy` module provides an analogous control surface for the indoor Crazyflie target, exposing high-level flight functions directly to Python. The `sleep` module complements these interfaces with an explicit low-power idle entry point that can wake on microphone energy or tap events, which is important for battery-constrained edge deployments. The `pipeline` module unifies the continuous detector, tracker, event stream, per-camera geometry, and pose-aware projection logic into a single interface. Together, these modules show that the platform is meant to be used as a scriptable inference-and-autonomy layer, where models, sensors, telemetry, and embedded ML primitives can be combined interactively from MicroPython.

The pipeline module is especially illustrative. According to the documented runtime surface, it exposes continuous detection, statistics, tracker snapshots, event streams, configurable association parameters, pose setting, and per-camera geometry. This is not a trivial wrapper around a detector. It is the public face of the deeper implementation

presented in the previous subsections: the parallel camera-to-TPU pipeline, the tracking-by-detection logic, and the geometry-aware projection model all become accessible through a minimal but expressive Python API. The same can be said for the `link` and `crazy` modules, which reveal that flight and telemetry functions are not separate applications but first-class capabilities of the runtime itself.

From a research-methodology perspective, this matters because it changes the pace and style of experimentation. A researcher can move from firmware bring-up to mission logic, data capture, tracking inspection, and communication testing without leaving the same runtime environment. This reduces iteration cost and makes the platform more suitable for exploratory autonomy research, where control policies, perception thresholds, and communication strategies change frequently.

The built-in help surface also provides indirect evidence of implementation maturity. A platform that documents precise operational behaviors, module boundaries, argument conventions, and example workflows has generally progressed beyond proof-of-concept code. Here, the fact that these capabilities can be explored directly from the serial REPL supports the claim that the implementation has been organized as a reusable experimental environment rather than as a collection of hidden firmware entry points.

The programming model therefore completes the overall argument of the chapter. The EdgeTPU provides the main accelerator-backed inference path, the MCU and RTOS provide the orchestration, the cameras, IMU, and microphone provide the sensing context, and MicroPython makes the whole system operable as a research platform. Just as importantly, the expanded `sentai` namespace shows that the implementation now spans not only perception and flight control, but also communication, adaptive learning, sequence modeling, anomaly detection, and lightweight mapping. The value of the implementation lies not only in runtime efficiency, but also in exposing that functionality through an interface that remains inspectable and extensible during autonomous-drone experiments.

19. Results and Performance Evaluation

This chapter consolidates every performance measurement reported in this work into a single narrative, in the order a reviewer would ask about them: *what questions did we set out to answer, which experiments answer each, and what did we find?* It is the IMRAD “Results” chapter of an MDPI-style structured article and does not duplicate the subsystem narratives in `memcpy.md` and `cam_switch.md`; those remain the primary source for the implementation detail behind each result.

Subsection numbering in the captions of the tables below matches the subsections of this chapter.

20. 1. Research questions

We asked five questions of the platform, each addressable by one or more experiments in the `diag` framework (see `experimental_setup.md` §6.1).

#	Question	Experiments that answer it
RQ1	What sustained frame rate does the parallel vision pipeline achieve on a fixed camera, for the 512×512 single-class model?	E14 (baseline 80-class model) → E15 (target model)

#	Question	Experiments that answer it
RQ2	Can the SDRAM-to-tensor copy — identified as the dominant stage of InferTask — be accelerated without changing the pipeline’s producer-consumer contract?	E15 <code>dma_memcpy(0/1)</code> A/B, same firmware
RQ3	What is the cost of switching between the two cameras on a frame-by-frame basis on a single-MIPI-lane-with-MUX topology?	E16 alternating; E18 three-sweep head-to-tail
RQ4	Can that switch be made glitch-free at the pixel level without a hardware rework?	E17 per-switch JPEG visual inspection
RQ5	Do the platform’s fault-handling pathways leave observable traces when a measurement runs without problems, and how do they escalate when the system degrades?	fault counters exposed via <code>sentai.diag.cam_stats()</code> , observed at the end of every E18 session

The rest of this chapter presents, per question, the headline measurement, the cross-session reproducibility check, and a brief interpretation that points to the implementation chapter for depth.

21. 2. RQ1 — Parallel-pipeline steady-state throughput

21.1. 2.1 Setup

Experiment class E15 (`e15_pipeline_parallel_512`) drives `sentai.pipeline.start/get/st` with `PrepTask` and `InferTask` running concurrently. `PrepTask` does frame-grab + PXP resize + int8 quantisation into a staging buffer; `InferTask` copies the staging buffer into the TFLite input tensor, then calls `Invoke` and `NMS`. The two tasks communicate through the `staging_free / prep_done` semaphore pair documented in [camera.md](#) §PXP. The wall-clock interval between two successive `sentai.pipeline.get()` returns is the per-frame throughput.

20 runs of 20 iterations each per session; all reported numbers are post-eDMA (see RQ2 for the pre/post split).

21.2. 2.2 Headline result (Table 1)

Table 1. Sustained parallel-pipeline throughput on the 512×512 single-class model, build #622+ (post-eDMA memcpy). Source: [experiments/s032_e15_x20/](#) — 20 runs, 20 repetitions each.

Metric	Mean $\pm$	min / max
frame_interval_ms	64.7 $\pm$ 1.9	61 / 75
prep_stall_ms	0.1 $\pm$ 0.3	0 / 1
infer_stall_ms	41.1 $\pm$ 1.5	37 / 45
Sustained FPS	15.46 $\pm$ 0.45	—

21.3. 2.3 Interpretation

The pipeline is **stall-balanced**. `infer_stall_ms` 41 ms means that InferTask blocks for ~41 ms per frame waiting for PrepTask to deliver the next staging buffer; that is, PrepTask is the critical-path stage (~64 ms/frame), and InferTask (Invoke + NMS + memcpy) finishes in 23 ms and waits. The 15.5 FPS ceiling therefore reflects the PrepTask stage time, not the TPU. The 2 FPS margin to the 15 FPS sensor rate is explained by the sensor actually streaming at slightly above 15 Hz (ISR count per second observed is 15.2-15.5 on this hardware).

22. 3. RQ2 — eDMA acceleration of the staging-to-tensor copy

22.1. 3.1 Setup

A runtime A/B flag `sentai.pipeline.dma_memcpy(0 | 1)` selects between the baseline CPU `memcpy()` and the eDMA-based path (channel 31, 32-byte AXI bursts). Both settings coexist in the same firmware build; switching between them is a single volatile store and does not restart the pipeline. 10×20 frames per setting, captured in the same session so scene, thermal state and TPU package cache are held constant.

22.2. 3.2 Headline result (Table 2)

Table 2. Per-stage timing and sustained FPS for CPU `memcpy` vs eDMA `memcpy`. 10 runs $\times$ 20 frames each, paired within a single session. Source: `../_e15_ab.py`, detail tables in `memcpy.md` §“Per-run results”.

	Invoke (ms)	memcpy (ms)	NMS (ms)	total_infer (ms)	Wall (ms)	FPS
dma_memcpy						
0	50.2	24.1	0.2	74.5	74.7	13.39
(baseline)						
1 (optimised)	49.7	14.6	0.2	64.5	65.1	15.32
	0.5	9.5	0	10.0	9.6	+1.93

22.3. 3.3 Interpretation

The `memcpy` drops from 32 % of the per-frame critical path to 22 %. Invoke time is unchanged (TPU is not involved in the `memcpy`), so the improvement is wholly attributable to the eDMA. The 9.6 ms wall-time saving translates 1-to-1 into FPS because we are on the critical path of InferTask. The optimisation crossed the 15 FPS sensor-rate threshold, which is why later camera-switch experiments can treat “fixed camera, 30 fps sensor, ~16 FPS pipeline output” as a baseline. Implementation detail — eDMA configuration, cache-coherency notes, two debugging trips — is documented in full in `memcpy.md` §“Optimised”.

23. 4. RQ3 — Per-switch cost on the shared-MIPI, MUX-gated dual sensor

23.1. 4.1 Setup

Experiment E18 (e18_camera_switch_headtail) runs three sweeps of the same sequential loop (select → to_tensor → invoke → detect) back to back in one session:

- **A** — fixed on cam_a for all 40 iterations (no switches)
- **B** — fixed on cam_b for all 40 iterations (no switches)
- **C** — alternating cam_a cam_b every iteration

Three sweeps let us express the switch cost as a subtraction against a baseline measured under identical conditions:

$$\text{per_switch_overhead} = C_total_mean - \max(A_total_mean, B_total_mean)$$

23.2. 4.2 Headline result (Table 3)

Table 3. Three-sweep head-to-tail benchmark, 30 fps sensor, Fix B firmware (post-flip-on-EOF), switch_drain=2, ratio=(0,0), 40 iterations per sweep, first sample dropped. Source: [experiments/s045_e18_post_refactor/](https://doi.org/10.3390/drones1010000).

Sweep	select	to_tensor	invoke	detect	total (ms)	FPS
A — fixed cam0	0.0	31.6	29.8	0.4	61.8	16.18
B — fixed cam1	0.0	31.6	30.5	0.4	62.5	16.00
C — alternat- ing	17.6	96.0	31.5	0.7	145.8	6.86

Per-switch overhead = 83.3 ms (133.4 % of the slower baseline). **Directional asymmetry (cam1 cam0) = 2.1 ms** (within noise).

23.3. 4.3 Budget decomposition

Component of the 83.3 ms tax	Amount
select wait for EOF ISR consumption	17.6 ms (0.5 frame interval at 30 fps)
to_tensor post-switch drain (stale-buffer drop + 2 fresh frames)	64.4 ms (2 × 33 ms)
Sensor-independent stages (invoke, detect)	0 ms (same in A and C)

23.4. 4.4 Cross-session agreement

Three sessions captured the same three-sweep benchmark under successively more refined firmware (pre-refactor, post-refactor confirmation, NASA-JPL review fixes). Agreement across the three sessions is a reproducibility check (Table 4).

Table 4. Cross-session reproducibility of Table 3. All three rows are 40-iteration head-to-tail sweeps on the same scene/model; firmware differences are the only variable. Values in ms.

Session	A — fixed cam0	B — fixed cam1	C — alternating	Asymmetry	
				cam1	cam0
s043_e18_headtail_drain	62.7	62.5	145.3	+0.1	
s044_e18_headtail_drain	62.6	62.1	145.7	+0.1	
s045_e18_post_refactor	61.8	62.5	145.8	2.1	
across sessions	0.4 ms	0.2 ms	0.3 ms	noise	

The spread across three firmware builds is < 1 ms on every stage. That bounds the measurement noise and validates the claim that the NASA-JPL review fixes are performance-neutral.

24. 5. RQ4 — Glitch-free MUX transitions

24.1. 5.1 Setup

E17 (`e17_switch_drain_visual`) writes alternating-camera JPEGs into the MicroPython heap during the timing loop and flushes them to LittleFS afterwards. Two runs per session at `switch_drain=2` and `switch_drain=1`, 16 iterations each, 512×512 quality-70. The output is a set of pixel-level frames available for offline review.

The subject is *visual correctness*, not timing. The timing is a by-product.

24.2. 5.2 Headline result — visual inspection

Table 5. Visual inspection outcomes across firmware builds. The full frame sets are in the linked folders; a representative “broken” frame and its “fixed” counterpart are cited inline.

Firmware	Session	drain=1 outcome	drain=2 outcome
Pre-flip-on-EOF	s038_e17_drain_ab	Fails. Horizontal seam at 40-60 % image height, every other frame split between cam0 and cam1. Example: 002_cam0_133ms.jpg	Clean.
Post-flip-on-EOF	s041_e17_eof_check	Mid-buffer seam removed , but residual sensor-side artefacts remain (AEC/AGC convergence) — see threats_to_validity.md §3.	Clean. Example: 003_cam1_201ms.jpg

24.3. 5.3 Interpretation

Moving the GPIO flip into the CSI end-of-frame ISR — so the MUX transition lands in the MIPI VBLANK window between two DMA buffers — eliminates the class of failure in which a single DMA buffer contains pixels from two sensors. The shipping configuration is `switch_drain=2`: it produces reliably clean frames. `switch_drain=1` remains available as an experimentation hook but the frames it produces are not suitable for production use;

this is the only residual pixel-quality limitation of the camera-switch chapter and is called out in its own “Known limitations” section in [cam_switch.md](#).

25. 6. RQ5 — Fault observability

25.1. 6.1 Setup

Every degraded path in the camera-switch subsystem — ISR arm not consumed within 150 ms, drain wait hitting its 300 ms ceiling, GetRawFrame retry, GetRawFrame fatal — increments a persistent counter (see the 0x0Axx range in [error_codes.csv](#)). The counters are exposed to MicroPython as `sentai.diag.cam_stats()`. Every E18 session captures the counter dict at the end of the run; a non-zero degraded-path value invalidates that session’s interpretation.

25.2. 6.2 Headline result

Table 6. Post-run fault counters for the canonical E18 sessions cited in this paper. All zeros means every switch was handled by the nominal flip-on-EOF path.

Session	switch_ok_eof	switch_fallback	drain_timeout	grab_retry	grab_fatal
s043_e18_headtail_drain	0	0	0	0	0
s044_e18_headtail_drain	0	0	0	0	0
s045_e18_post_refactor	0	0	0	0	0

25.3. 6.3 Interpretation

The fault-counter infrastructure is not merely defensive instrumentation; it is a publication-grade **measurement integrity check**. A reviewer who suspects that some fraction of our E18 alternating iterations fell back to the legacy synchronous switch path (which would re-introduce the seam and bias the timing) can confirm from the counters that in the sessions we report, **none did**. The counters remain available at all times via REPL for future replays on the same hardware.

26. 7. Summary table (Research-question × metric)

Table 7. Paper-level summary: one row per research question, the experiment class that answers it, the headline number, the direction of change versus the baseline, and the session(s) that document it.

RQ	Headline metric	Baseline	Final	Evidence
RQ1	Sustained parallel-pipeline FPS, 512×512	13.39 (pre-eDMA)	15.47	+15 % s024 , s026 , s032
RQ2	Staging→tensortorch memcpy cost	24.1 ms (CPU)	14.6 ms (eDMA)	39 % memcpy.md §“Per-run results”

RQ	Headline metric	Baseline	Final		Evidence
RQ3	Alternating FPS (switch every frame)	4.7 (15 fps, pre-Fix A)	6.86 (30 fps, Fix B)	+46 %	s034 , s045
RQ3	Directional asymmetry cam1 cam0	+64.2 ms	2.1 ms (within noise)	removed	s034 → s045
RQ4	Mid-buffer seam at drain=2	visible every switch	none	removed	visual inspection of s038 vs s041
RQ5	Degraded-path counter during a nominal run	n/a pre-review	0 across all three reported sessions	validated	artifact.md §“Data integrity”

27. 8. Discussion (of results only; full discussion in [discussion.md](#))

Three observations that belong here rather than in the implementation chapters because they relate to the results taken together:

1. **Each optimisation exposes the next bottleneck.** The CPU memcpy was hidden behind a slow camera-drain fix (described in [camera.md](#)). The 15-fps pipeline plateau was hidden behind the memcpy. The alternating-switch tax was hidden behind the fixed-camera ceiling. Each of the three “headline” results only became measurable once the previous layer had stopped dominating the wall-clock; this is why the subsystem narratives must be read in order (camera-drain → memcpy → cam_switch).
2. **The residual 83 ms per-switch tax is sensor-topology bound, not firmware bound.** With both cameras sharing one CSI-2 lane, two frame intervals of drain is the theoretical floor for a conservative drain threshold on a not-VSYNC-synchronised pair of sensors. Further reduction requires either hardware sync (a FSIN wire between the OV5640s, rejected for this iteration — see [threats_to_validity.md](#) §4) or a different CSI-receiver topology.
3. **The measurement discipline scales.** The same diag framework that captured the initial 6 FPS baseline in the E10-E12 subsystem probes (Appendix A of [experiments/README.md](#)) captured the final 16 FPS head-to-tail benchmark. No measurement methodology change was needed to follow the four-ish orders of magnitude of scope from “does pvPortMalloc work under load” to “what is the per-switch overhead of the dual-sensor MUX topology”. That is a statement about the framework, not about the results.

Open questions, limitations, and threats to this interpretation are in [threats_to_validity.md](#).

28. Data availability statement

Every number in Tables 1–7 is derived from raw per-iteration CSVs that are mirrored under `../experiments/`. The session names cited in the table captions are the direct folder paths. The appendix generator at `../experiments/_build_appendix.py` reproduces the full appendix tables from those CSVs without any on-device round-trip, so an external reviewer can verify every derived statistic offline. The full artifact description — code commit SHAs, firmware build numbers, reproduction steps — is in `artifact.md`.

28.1. SDRAM-to-Tensor Memcpy Optimisation

See also: `experimental_setup.md` for the canonical hardware and firmware stack (this chapter’s “Test conditions” section reproduces the subset relevant to the memcpy result), `evaluation.md` for the cross-cutting results summary where this chapter’s findings appear as RQ2, `artifact.md` for the claim-to-CSV map, and `statistical_notes.md` for the statistical conventions applied to the per-run tables below.

29. Abstract

The end-to-end vision pipeline on the SentAI board (Coral Dev Board Micro, NXP i.MX RT1176) was stuck at ~13 FPS even though the camera delivers 15 FPS and the EdgeTPU invoke on our lightest model only costs ~50 ms. Instrumenting the pipeline’s InferTask exposed a single dominant cost: a 786 KB memcpy(`tensor_buf`, `staging_buf`, `total`) from SDRAM to SDRAM that consumed **24 ms per frame** due to CPU-driven cache-line traffic through the SEMC controller. Replacing that one memcpy with an eDMA memory-to-memory transfer configured for 32-byte AXI bursts took the copy down to **14.6 ms** and raised sustained pipeline throughput from **13.41 FPS** to **15.47 FPS** — enough to match the sensor rate.

30. Test conditions

30.1. Hardware

Component	Value
MCU	NXP i.MX RT1176 (Cortex-M7 @ 800 MHz + Cortex-M4)
On-chip EdgeTPU	Coral/Google TPU, internal USB2 bus
External RAM	16 MB SDR-SDRAM on SEMC (166 MHz)
OCRAM / DTCM / ITCM	1.25 MB / 256 KB / 256 KB
Camera	OV5640-based coralmicro module, 1280×720 native, 15 FPS streaming
USB to host	CDC-ACM (REPL) + CDC-NCM (IP 10.0.0.1)
Board	Coral Dev Board Micro dev kit, powered via USB-C

30.2. Firmware

Parameter	Value
Build	sentai_runtime build #622+ (linker script MIMXRT1176xxxxx_cm7_ram_mp.ld)
RTOS	FreeRTOS (CMSIS M7 build, 1 ms tick)
MicroPython GC heap	512 KB, in .sdram_bss
Pipeline tasks	det_prep prio 2 · det_infer prio 3
Camera task	camera_task prio configMAX_PRIORITIES - 1

30.3. Model under test

1299

Parameter	Value
File	/yolo_1_class_512_1_upsample_512.in
On-flash size	5 591 680 B (5.33 MB)
Input	uint8[1, 512, 512, 3] — 786 432 B (this is the buffer we memcpy)
Input quantisation	scale = 1/255, zero_point = 0
Output	uint8[1, 1344, 6] — 8 064 B — YOLOv5-enhanced anchor format
Architecture	YOLOv5 enhanced (single upsample at P5/32, 1-class head)
Output quantisation	scale = 1/255, zero_point = 0
Output row semantics	[cx, cy, w, h, obj_conf, class_conf] (normalised 0..1)
TFLite arena	799 KB used / 8 192 KB available
EdgeTPU opened in	mode 3 (kMax)

30.4. Scene

1300

- Static scene (camera does not move during measurement). 1301
- Before/after scene snapshots are saved by every experiment that touches the camera — see `diag.snapshot_scene("before" | "after")` in [diag/_session.py](#). Stored at `/diags/<session>/scene_cam<N>_(before|after)_{W}x{H}.jpg`. 1302 1303 1304
- For the runs reported here the scene contained no target class instances, so `num_detections == 0` every frame — the NMS execution path is still run in full (candidate scan over all 1344 anchors), only the sort and IoU stages exit early. 1305 1306 1307

30.5. Methodology

1308

- Each run = 20 measurement frames after 1 warm-up frame. 1309
- 10 back-to-back runs per configuration to capture variance. 1310
- **Both configurations measured from the same firmware image** using the runtime A/B flag `sentai.pipeline.dma_memcpy(0|1)`. Switching between CPU memcpy and eDMA is one volatile store; no reflash, no reboot, no camera restart between the two blocks. This removes every confound that could come from firmware drift, thermal state, or camera calibration variance. 1311 1312 1313 1314 1315

- Timing collected via `sentai.pipeline.get_ex()` which returns the firmware-measured `inference_ms` (pure TPU Invoke), `memcpy_ms` (staging $\rightarrow$ tensor copy), `nms_ms` (post-processing), and `total_ms` (full InferTask iteration). All reported in milliseconds.
- Host (Python) wall-clock interval is the delta between consecutive `pipeline.get_ex()` returns; the gap between `wall_interval` and `infer_total` measures everything outside InferTask (queue IPC, MicroPython overhead, `mp_repl` wake-up).
- `verbose(0)` for the whole loop body — no `printf` traffic on CDC-ACM, so USB never saturates and host reads don't stall.
- Benchmark driver: [_e15_ab.py](#).

31. Baseline — CPU memcpy (SDRAM $\rightarrow$ cache $\rightarrow$ SDRAM)

Flag set by `sentai.pipeline.dma_memcpy(0)`. The InferTask uses a plain `memcpy()` to move the prepared frame into the TFLite input tensor. This matches the state right after the camera-drain fix (non-blocking `TryGetRawFrame`, documented separately in [usb.md](#) and [camera.md](#)).

31.1. Per-run results (10 $\times$ 20 frames, same firmware as optimised)

Run	wall	invoke	memcpy	nms	infer_total	FPS
1	75.9	50.8	24.0	0.3	75.1	13.2
2	74.8	49.9	24.4	0.4	74.6	13.4
3	74.6	50.1	24.3	0.3	74.6	13.4
4	74.7	50.3	24.0	0.4	74.6	13.4
5	74.6	50.0	24.3	0.2	74.4	13.4
6	74.2	50.0	24.1	0.1	74.3	13.5
7	74.6	50.3	24.1	0.2	74.6	13.4
8	74.9	50.3	24.1	0.2	74.7	13.4
9	74.2	50.1	24.0	0.1	74.2	13.5
10	74.4	50.3	24.0	0.2	74.4	13.4
mean	74.7	50.2	24.1	0.2	74.5	13.39

FPS stats: mean 13.39, min 13.18, max 13.49, 0.10.

31.2. Interpretation

The pipeline loop is fully dominated by InferTask: `gap = wall - total = 0.2 ms`, which is the cost of the FreeRTOS `xQueueSend`, CDC-NCM wake-up and MicroPython tuple allocation. Everything else is inside the firmware, and of that `memcpy` alone is **32 % of the critical path** (24.1 / 74.4).

Throughput during the copy = 786 432 B / 24 ms **32.8 MB/s**. That is far below SDRAM's theoretical 400 MB/s peak. The CPU is single-word- copying through the D-cache, which forces:

1. Each 32-byte cache line of **source** is fetched from SDRAM (24 576 fills for 786 KB).
2. Each 32-byte cache line of **destination** is *also* fetched from SDRAM, because the Cortex-M7 D-cache is write-allocate.
3. The cache line is modified in place.
4. The dirty destination line is later written back to SDRAM.

So the SEMC bus sees $\sim 2.4\times$ the payload size (~ 1.9 MB) and all of it as isolated single-beat AXI accesses — the bus cannot coalesce consecutive CPU stores into back-to-back bursts. Application note AN12437 explicitly calls this out: “SDRAM can reach high

throughput when accessed by LCD and PXP, as these two masters support back-to-back access, with better performance compared to other master access, but dropping when accessed by CPU core.”

32. Optimised — eDMA memcpy with 32-byte AXI bursts

The replacement helper lives in [detection_task.cc](#):

```
// DMA0 channel 31 (audio driver uses ch 0). One-time init.
static constexpr uint32_t kSentaiDmaChannel = 31;
static edma_handle_t s_dma_memcpy_handle;

// Pick the widest transfer width the addresses and size allow.
// 32-byte → one 8-beat AXI burst per request, exactly the pattern SEMC
// is optimised for.
uint32_t width = 4;
if (((uintptr_t)src | (uintptr_t)dst | size) & 0x1Fu) == 0u) width = 32;
else if (((uintptr_t)src | (uintptr_t)dst | size) & 0x07u) == 0u) width = 8;

edma_transfer_config_t tcfg;
EDMA_PrepareTransfer(&tcfg, src, width, dst, width,
                    /*bytesEachRequest=*/size,
                    /*transferBytes=*/size,
                    kEDMA_MemoryToMemory);
EDMA_SubmitTransfer(&s_dma_memcpy_handle, &tcfg);
EDMA_StartTransfer(&s_dma_memcpy_handle); // arm (SERQ)
EDMA_TriggerChannelStart(DMA0, kSentaiDmaChannel); // fire first minor loop

// Bounded polled wait - no ISR, no semaphore, no scheduler coupling.
const TickType_t deadline = xTaskGetTickCount() + pdMS_TO_TICKS(50);
while ((EDMA_GetChannelStatusFlags(DMA0, kSentaiDmaChannel)
        & kEDMA_DoneFlag) == 0) {
    if (xTaskGetTickCount() > deadline) return false; // fallback to CPU memcpy
}
EDMA_ClearChannelStatusFlags(DMA0, kSentaiDmaChannel,
                             kEDMA_InterruptFlag | kEDMA_DoneFlag);
DCACHE_InvalidateByRange((uint32_t)dst, size);
```

Two non-obvious details that cost measurable debug time:

1. **EDMA_StartTransfer only sets SERQ** (the request-enable bit). Memory-to-memory transfers have no peripheral request line, so the minor loop never fires until we explicitly software-trigger via `EDMA_TriggerChannelStart` (SSRT register). Without that call the channel sits armed forever and the 50 ms bounded wait expires.
2. **Poll `kEDMA_DoneFlag`, not `kEDMA_InterruptFlag`**. Interrupts are not enabled on this channel (no IRQ handler installed), so the interrupt flag would never set.

No ISR is installed; the wait is a bounded polled loop. A 786 KB transfer completes in ~15 ms of wall time with a 35 ms safety margin, so the CPU can't livelock. On polled-loop timeout (DMA hardware stuck or mis-configured) the code falls back to plain memcpy — the pipeline degrades, doesn't brick.

32.1. Cache strategy

Both buffers are written exclusively by DMA masters in steady state (`s_staging_buf` by PXP after camera capture; the TFLite input tensor by our new eDMA). DMA writes

bypass the D-cache, so neither buffer holds dirty cache lines under normal operation. Our first attempt called `DCACHE_CleanInvalidateByRange` on both source and destination before the DMA, for safety. That added **~20 ms** per frame — cache ops walk all address ranges and on SDRAM the walk itself serialises with SEMC bus writebacks of whatever cache lines happen to be dirty. We measured an end-to-end memcpy of 78 ms with the defensive cache flush, worse than the original CPU memcpy.

Removing the pre-DMA `CleanInvalidate` — while keeping a mandatory `DCACHE_InvalidateByRange` (same size) **after** the DMA so subsequent CPU reads see fresh bytes — was what actually delivered the win.

32.2. Per-run results (10×20 frames, same firmware, `dma_memcpy=1`)

Flag set by `sentai.pipeline.dma_memcpy(1)`. Measured immediately after the baseline block — same camera, same scene, same model, same Python loop body, same TPU state. Only the runtime flag changed.

Run	wall	invoke	memcpy	nms	infer_total	FPS
1	64.3	49.5	14.6	0.2	64.3	15.6
2	66.3	50.2	14.6	0.3	65.1	15.1
3	64.6	49.7	14.6	0.2	64.4	15.5
4	65.9	49.8	14.6	0.2	64.7	15.2
5	64.3	49.5	14.6	0.2	64.2	15.6
6	66.1	50.1	14.4	0.2	64.7	15.1
7	64.7	49.7	14.6	0.2	64.5	15.5
8	66.1	50.0	14.6	0.3	64.8	15.1
9	64.6	49.7	14.5	0.2	64.4	15.5
10	66.1	49.9	14.7	0.2	64.8	15.1
mean	65.3	49.8	14.6	0.2	64.6	15.32

FPS stats: mean 15.32, min 15.07, max 15.56, 0.22.

Throughput during the copy = $786\,432\text{ B} / 14.6\text{ ms}$ **53.9 MB/s** — a **1.65× improvement** for the same buffer size and addresses.

The observed pipeline FPS sits right at the camera sensor rate, confirming that with the DMA optimisation engaged the sensor — not the compute path — is the rate-limiting step.

33. Comparison and discussion

33.1. Per-stage cost (paired A/B on same firmware, 200 frames each)

Stage	CPU memcpy	eDMA 32-B bursts	
Invoke (TPU)	50.2 ms	49.8 ms	0.4 ms (noise)
staging → tensor copy	24.1 ms	14.6 ms	9.5 ms (39 %)
NMS (1-class × 1344 anchors, YOLOv5 layout)	0.2 ms	0.2 ms	—
InferTask total	74.5 ms	64.6 ms	9.9 ms
wall interval	74.7 ms	65.3 ms	9.4 ms

The saving is entirely explained by the copy; invoke, NMS and the Python/IPC gap are unchanged within noise. Importantly, invoke did **not** get slower even though the

eDMA now competes with the TPU’s internal USB bus for SEMC bandwidth — the transfer is short enough (15 ms, before invoke starts) that it finishes before invoke’s input upload begins.

33.2. Throughput headline

Config	FPS mean	FPS min	FPS max	vs baseline
CPU memcpy (dma_memcpy(0))	13.39	13.18	13.49	—
eDMA 32-B bursts (dma_memcpy(1))	15.32	15.07	15.56	+14.4 %
Camera sensor rate (hard ceiling)	15.00	—	—	—

The optimised configuration operates right at the camera sensor rate (15 FPS 66.7 ms/frame); individual measurements slightly exceed this when the pipeline drains a small in-flight queue over one cycle and catches up on the next. Sustained over a longer window the effective rate is bounded by the camera. The 1.92 FPS absolute gain corresponds to a **14.4 % relative throughput improvement** measured under controlled A/B conditions in the same firmware image.

33.3. Variance

grows from 0.09 to 0.20 FPS — not material for the use case but worth noting. The extra variance comes from the 50 ms DMA polling path occasionally getting preempted by the camera task or the watchdog task; this would disappear if we re-armed the polling using a task-notification wake-up driven by the eDMA completion IRQ, at the cost of additional ISR state. We judged the trade-off not worth it while we are comfortably above the sensor rate.

34. Why we chose eDMA over alternatives

Option	Expected time	Why we didn’t pick it
Keep CPU memcpy + __builtin_prefetch hints	~18 ms	Only ~25 % saving, still CPU-bound on SEMC
Move tensor to OCRAM	N/A	TFLite arena is 8 MB, exceeds 1.25 MB OCRAM
Move staging to OCRAM	mixed	Halves read-side cost but write-side still on SEMC; net ~10 ms, small gain for a large refactor

Option	Expected time	Why we didn't pick it
PXP with PS→output 1:1	~3-5 ms	PXP is already used by PrepTask; serialising PXP across PrepTask+InferTask would defeat pipeline parallelism
eDMA mem-to-mem	~14-15 ms	Independent engine, back-to-back SEMC bursts, no conflict with PXP, isolated to a single helper function

PXP would in principle be even faster than eDMA (AN12437 lists PXP as best-in-class for SDRAM throughput), but PXP is the camera-side resize engine and scheduling it twice per frame pulls PrepTask and InferTask into a shared-resource dance that kills parallelism. eDMA channel 31 is independent of every other user in the firmware, so the change stays local and analysable.

35. Failure modes

Per [agent/embedded.md](#): every optimisation must have a defined failure path.

Failure	Detection	Action
Addresses not 4-byte aligned, or size % 4 != 0	compile-time alignment check fails	<code>dma_ok = false</code> → falls back to CPU memcpy
eDMA submit returns non-success	Return value of EDMA_SubmitTransfer	Return <code>false</code> → CPU memcpy
Channel stuck (DONE flag never sets)	50 ms bounded polled wait expires	Return <code>false</code> → CPU memcpy
Some other task takes DMA0 ch31	Not detected; documented single-owner assumption	ch31 is dedicated to this helper — audio uses ch0; camera doesn't use eDMA

No silent corruption: the DMA writes to an empty buffer the TFLite tensor alone will read; `DCACHE_InvalidateByRange(dst)` after the DMA guarantees the next invoker reads fresh bytes regardless of cache state.

36. Reproducing these results

```
# On the device REPL, after flashing build #622+.
# One-shot E15 run - creates /diags/sNNN_e15_512/ with CSV + scene_{before,after}.
import diag
diag.e15_pipeline_parallel_512(repetitions=5)

# 10x benchmark table (dma_memcpy stays at whatever the flag is set to)
import sentai
exec(sentai.fs.read_str('/_e15_table.py'))

# *** Paired A/B benchmark (this document's tables come from here) ***
# Runs 10x20 frames with DMA off, then 10x20 frames with DMA on,
```

```
# prints a side-by-side summary + FPS improvement. 1458
exec(sentai.fs.read_str('/_e15_ab.py')) 1459
```

```
# Manual toggle at any time: 1461
sentai.pipeline.dma_memcpy(0) # CPU memcpy (baseline) 1462
sentai.pipeline.dma_memcpy(1) # eDMA 32-B bursts (optimised) 1463
sentai.pipeline.dma_memcpy() # read current setting 1464
```

Every run writes: 1465

- /diags/sNNN_e15_512/manifest.csv — experiment log 1466
- /diags/sNNN_e15_512/001_e14_pipeline_par_cam0_512x512.csv — per-frame timing 1467
- /diags/sNNN_e15_512/001_e14_pipeline_par_cam0_512x512.txt — column key + params 1468
- /diags/sNNN_e15_512/scene_cam0_before_512x512.jpg — scene at experiment start 1469
- /diags/sNNN_e15_512/scene_cam0_after_512x512.jpg — scene at experiment end 1470
- /diags/sNNN_e15_512/summary.txt — heap + duration summary 1471

The pair of scene snapshots lets an operator confirm offline that the camera was pointing at the expected target and that nothing moved during the run. This matters when `num_detections == 0` — it distinguishes “*scene had no target*” from “*detector missed*”. 1472

37. Cross-model comparison — E14 (COCO 80-class) vs E15 (1-class) 1478

To put the memcpy optimisation in context we ran the pipeline 20 times per experiment (20 frames each, 400 frames total per configuration), each pair of runs writing CSVs + scene snapshots under its own /diags/sNNN... session for later offline analysis. 1479

Property	E14 (Ultralytics YOLOv8 COCO)	E15 (custom YOLOv5-enhanced, 1-class)
Model file	/yolo26n.edgetpu_1.tflite	/yolo_1_class_512_1_upsample_.
File size	4.41 MB	5.33 MB
Architecture family	YOLOv8 nano (Ultralytics)	YOLOv5 enhanced (single upsample at P5/32)
Output layout	[1, 84, 2100] (v8 transposed)	[1, 1344, 6] (v5-style: [cx,cy,w,h,obj,cls])
Num. classes	80 (COCO)	1
Input shape	int8 [1, 320, 320, 3] — 307 200 B	uint8 [1, 512, 512, 3] — 786 432 B
Output shape	int8 [1, 84, 2100] — 176 400 B	uint8 [1, 1344, 6] — 8 064 B
NMS inner cost	80 classes × 2100 anchors = 168 000 dequant+compares/frame	1 class × 1344 anchors = 1 344 compares/frame
DMA memcpy	on (small 307 KB payload, marginal win)	on (decisive 786 KB payload)
Session	s031_e14_x20/	s032_e15_x20/

37.1. Aggregated results (20 runs × 20 frames each, verbose=0) 1482

Metric	E14 mean	E14 range	E15 mean	E15 range
frame_interval (ms)	156.9	156.1–157.8	69.4	64.3–70.9
pipeline FPS (wall)	6.37	6.34–6.41	14.40	14.10–15.53
firmware avg_fps	6.19	6.00–6.20	14.08	13.40–14.30
detections (sum)	2	—	0	—

The **E14 experiment is NMS-bound** on this dataset: the 80-class yolo iteration pays ~168 000 dequant + argmax operations per frame, which on Cortex-M7 FPU at 800 MHz is visibly slow compared to the 1-class hot path. E15's NMS sees only 1 344 single-class comparisons per frame and runs in < 1 ms. Together with the much heavier 786 KB tensor copy that E15 would have to do if we hadn't switched to eDMA, this is why the bigger model can still run more than 2× faster than the smaller one.

37.2. Detection of yolo layout & class count

To automate experiment setup across models with different class budgets, the firmware now exposes `sentai.tpu.yolo_info()` which returns (`layout_str`, `num_classes`, `num_anchors`) inferred purely from the loaded model's output tensor shape (the edgetpu compiler strips most metadata buffers from the .tflite binary, so shape is the only reliable signal). Verified on both experiments:

Model file	yolo_info()
yolo26n.edgetpu_1.tflite (E14)	('v8', 80, 2100)
yolo_1_class_512_1_upsample..._P5_32 (E15)	('v5_like', 1, 1344)

Source: `yolo_infer_info()` in [sentai_runtime.cc](#).

37.3. Artefacts saved for offline inspection

Each run in a session writes:

- `NNN_<tag>_pipeline_par_cam0_<W>x<H>.csv` — per-frame wall_interval, prep_stall, infer_stall, num_detections
- `NNN_<tag>_pipeline_par_cam0_<W>x<H>.txt` — column key + parameters (conf, iou, repetitions, frames_received, fw_processed, fw_dropped)
- `scene_cam0_before_<W>x<H>.jpg` — scene the operator pointed at, at session start. Lets offline inspection confirm what the model saw.
- `scene_cam0_after_<W>x<H>.jpg` — same scene, session end. Paired with BEFORE for drift / movement verification.
- `manifest.csv + summary.txt` — per-session roll-up.

38. Progression across the pipeline optimisation effort

Revision	memcpy	wall	FPS	Note
Baseline (camera drain polls 4 s)	24 ms	~200 ms	5.0	TryGetRawFrame not truly non-blocking
Non-blocking camera drain	24 ms	75 ms	13.4	Fix #1 — see usb.md / camera.md

Revision	memcpy	wall	FPS	Note
+ eDMA memcpy	14.6 ms	64.7 ms	15.5	Fix #2 — this document

End-to-end, **3.1× speedup over the initial firmware**, reaching the camera sensor rate.

39. What’s next

- invoke at 49.6 ms now dominates the loop. The remaining headroom is inside Invoke() itself — 28 ms cold when measured outside the pipeline, 49 ms steady-state while PrepTask runs concurrently, suggesting SEMC/USB bus contention with the TPU’s internal DMA. Pushing beyond 15 FPS would require either a lighter model (smaller input → less TPU bus), or double-buffering the TFLite tensor so NMS/Invoke overlap more aggressively.
- The sentai_dma_memcpy helper is local to detection_task.cc. Three other places in the firmware copy large SDRAM buffers (sentai_cam_capture_rgb, sentai_cam_capture, sentai_cam_to_tensor_ex). Once lifted into libs/base/dma_memcpy.* the same win applies there.
- Switch to completion-interrupt + FreeRTOS notification once any concurrent user contests channel 31; the current polled loop is only safe because no other task uses this channel.

40. Sources

- NXP AN12437 — i.MX RT Series Performance Optimization (SDRAM access patterns per master; PXP/LCD/eDMA back-to-back bursts).
- NXP i.MX RT1170 Reference Manual (IMXRT1170RM), eDMA and SEMC chapters (TCD layout, SSRT/SERQ semantics, AXI burst configuration).
- Local code: detection_task.cc, modsentai_pipeline.c, diag/e_pipeline.py, diag/_session.py

40.1. Glitch-Free Dual-Camera Switching

See also: [experimental_setup.md](#) for the canonical hardware and firmware stack, [evaluation.md](#) for the cross-cutting results summary (this chapter is the implementation-level narrative that the Evaluation chapter’s RQ3 and RQ4 reference), [threats_to_validity.md](#) for caveats, [artifact.md](#) for the claim-to-CSV map, and [related_embedded_inference.md](#) for how this work positions against other MIPI-CSI2 multi-camera topologies.

41. Abstract

The SentAI board carries two OV5640-derived camera modules multiplexed onto a single MIPI-CSI lane via a GPIO-controlled analogue mux. Selecting a camera is effectively free at the MUX level (a GPIO register write that returns in micro-seconds), but the first frame grabbed *after* a switch is not: the sensor driver has to drain frames already queued from the previous sensor and then wait for at least two fresh ISR frames from the newly selected one before to_tensor() is allowed to return. This document quantifies that cost by running two back-to-back experiments on identical firmware: **E15**, the parallel 15 FPS pipeline pinned to cam0, and **E16**, a sequential loop that alternates cam0 → cam1 every single frame. The comparison measures the per-switch overhead at **147 ms**, with a stable **65 ms asymmetry** in favour of switching *to* cam0. The result sets a concrete upper bound on any dual-camera scheme that relies on frame-level MUX toggling.

42. Test conditions

42.1. Hardware

Component	Value
MCU	NXP i.MX RT1176 (Cortex-M7 @ 800 MHz + Cortex-M4)
On-chip EdgeTPU	Coral/Google TPU, internal USB2 bus
External RAM	16 MB SDR-SDRAM on SEMC (166 MHz)
Cameras	2× OV5640-based coralmicro modules, 1280×720 native, 15 FPS streaming
Camera mux	GPIO-controlled analogue MUX on shared MIPI-CSI lane
USB to host	CDC-ACM (REPL) + CDC-NCM (IP 10.0.0.1)

42.2. Firmware

Parameter	Value
Build	sentai_runtime build #624, eDMA memcpy enabled (see memcpy.md)
Verbose	sentai.verbose(0) for the whole loop body in both runs
Session	experiments/s034_e15_vs_e16_x40/ (on host; originally /diags/s034_e15_vs_e16_x40/ on device LittleFS)

42.3. Model under test

Identical to [memcpy.md](#):

Parameter	Value
File	/yolo_1_class_512_1_upsample_512_in
Input	uint8[1, 512, 512, 3] — 786 432 B
Output	uint8[1, 1344, 6] — YOLOv5-enhanced, 1-class head
Resolution fed to model	512×512 (post-PXP resize from 1280×720)

42.4. Scene

- Static scene throughout both runs — same room, same framing, no moving targets. `num_detections == 0` on every frame, so NMS still runs its full candidate scan (1344 anchors) but the sort/IoU stages exit early — a constant background cost that cancels in the E15 vs E16 comparison.

- Before-and-after snapshots are saved from *both* cameras by the shared helper `diag.snapshot_both_cameras(when)` in [diag/_session.py](#):

```

../experiments/s034_e15_vs_e16_x40/scene_cam0_before_512x512.jpg (23 611kB)
../experiments/s034_e15_vs_e16_x40/scene_cam0_after_512x512.jpg (23 540kB)
../experiments/s034_e15_vs_e16_x40/scene_cam1_before_512x512.jpg (29 301kB)
../experiments/s034_e15_vs_e16_x40/scene_cam1_after_512x512.jpg (29 692kB)

```

42.5. Methodology

- **Back-to-back in the same firmware image, same session:** E15 runs first, then one `gc.collect()`, then E16. No reflash, no reboot, no camera restart between them. Scene, thermal state, TPU cache and model pointer are all preserved — removing every confound that could come from firmware drift.
- **40 measurement frames per experiment.** E15 warmup is handled internally by the parallel pipeline (one drained `pipeline.get()` before timing); E16 does one full warm cycle on each camera before the measurement loop begins. The first measurement sample is dropped in both cases before computing statistics.
- **Different measurement primitives by design:**
 - E15 reports `frame_interval_ms` = wall-clock delta between consecutive `sentai.pipeline` returns, i.e. the steady-state interval between emitted frames. That is the right number for a parallel pipeline whose internal stages overlap.
 - E16 reports `total_frame_ms` = sum of per-stage waits inside a sequential `select` → `to_tensor` → `invoke` → `detect` loop. In a sequential run this also equals the wall-clock per-frame interval. Both numbers are therefore directly comparable as “how many ms until the host sees the next detection result”.
- Benchmark driver: `_e15_vs_e16.py` invoked via `repl_run.py`.
- Experiment code: [diag/e_pipeline.py:e15_pipeline_parallel_512](#) (parallel, fixed camera) and [diag/e_pipeline.py:e16_camera_switch_512](#) (sequential, alternating cameras).

43. Experiment E15 — fixed camera, parallel pipeline (baseline)

`diag.e15_pipeline_parallel_512(repetitions=40)`. Locks the camera to `cam0` for the whole run and uses the firmware pipeline (`PrepTask` + `InferTask`) so camera-capture + PXP-resize + quant of frame `N+1` run in parallel with TPU Invoke of frame `N`. Wall interval between emitted frames is then `max(prepare_stage, infer_stage)`, not their sum.

43.1. Per-frame intervals (40 samples, first two dropped)

```

frame_interval_ms (ms, 38 samples after warm-up)
min      61
max      68
mean     64.5
stdev    1.8
FPS      15.5

```

All 38 intervals fell in the range **61...68 ms**, 1.8 ms. At this resolution and model, `prepare_stage` dominates the pipeline — the CSV’s `infer_stall_ms` column is non-zero every frame (~41 ms), meaning the infer task waits ~41 ms for prep to hand off the next frame. Nothing in the measurement path is close to saturation of either USB endpoint.

44. Experiment E16 — alternating `cam0` `cam1`, sequential pipeline

`diag.e16_camera_switch_512(cam_a=0, cam_b=1, repetitions=40)`. The loop body is deliberately sequential:

```
for i in range(repetitions):
    cam = cam_a if (i % 2 == 0) else cam_b
    t0 = _ticks(); sentai.camera.select(cam);    sel = _ticks() - t0
    t0 = _ticks(); sentai.camera.to_tensor();    tens = _ticks() - t0
    inv = sentai.tpu.invoke()
    t0 = _ticks(); dets = sentai.tpu.detect(...); det = _ticks() - t0
```

The parallel `sentai.pipeline` is *not* started because `PrepTask` owns the camera MUX during its inner `cam_grab_latest()` loop — a mid-pipeline `camera.select()` would race with the active grab and the measurement would be undefined. E16 is therefore a per-call-graph decomposition of a switch-every-frame scheme, which is exactly what this experiment is designed to bound.

44.1. Aggregate results (40 samples, first one dropped)

total_frame_ms (ms, 39 samples)		
min	176	
max	250	
mean	211.4	
stdev	32.5	
FPS	4.7	

The is an order of magnitude larger than E15 because the loop is **bimodal**: even frames (going to `cam0`) and odd frames (going to `cam1`) have different post-switch drain cost. Splitting by the camera that produced each measured frame makes the distribution clearly unimodal again:

direction	frames	total (ms)	to_tensor (ms)	invoke (ms)	detect (ms)	select (ms)
→ cam0 (after cam1)	19	178.5 ± 1.5	149.4 ± 0.9	29 ± 2	0.5 ± 0.5	0
→ cam1 (after cam0)	20	242.7 ± 2.1	214.7 ± 1.0	28 ± 2	0.5 ± 0.5	0
asymmetry (cam1 cam0)	—	+64.2 ms	+65.3 ms	0	0	0

Within each camera 1–2 ms, so every ms of the switch cost is reproducible, not jitter.

45. E15 vs E16 — head-to-head

metric	E15 (fixed cam0, parallel)	E16 (alternating, sequential)	
frames (measured)	38	39	—
wall frame time — mean ±	64.5 ± 1.8 ms	211.4 ± 32.5 ms	+146.9 ms
min / max	61 / 68	176 / 250	—
effective FPS	15.5	4.7	10.8 (70 %)

metric	E15 (fixed cam0, parallel)	E16 (alternating, sequential)
overhead per switched frame	—	147 ms

The switch-every-frame scheme collapses the pipeline from the sensor-rate ceiling (15 FPS) to **4.7 FPS**, a 3.3× slowdown, and does so deterministically — a 5-second burst of alternation loses about **54 inferences** relative to fixed-camera operation.

46. Interpretation

46.1. The MUX flip itself is effectively free

`sentai.camera.select(id)` measured **0 ms** every time. Internally it is `CameraTask::SwitchCam` which performs a single GPIO `kCamMux = id` store plus `g_cam_switch_pending = true` — documented in [camera.md](#) §2. No CSI reinitialisation, no PLL re-lock, no register-list write to the sensor. The whole cost of “switching cameras” lives in the *next* frame grab.

46.2. All overhead is absorbed by `to_tensor()`

E16’s per-stage split is unambiguous:

- `select` = 0 ms
- `invoke` 28–30 ms (identical across both cameras — same model, same EdgeTPU path)
- `detect` 0.5 ms (same NMS, zero-candidate exit)
- `to_tensor` takes the rest: **149 ms to cam0, 215 ms to cam1**

`to_tensor()` calls `sentai_cam_get_raw_with_recovery()` which, with `g_cam_switch_pending` set, enters the slow path:

1. **Drain queued stale frames** — the CSI DMA may already have written one or two buffers with pixels from the *previous* sensor before the MUX flip took effect. The driver walks `g_camera_frame_seq` forward and discards those.
2. **Wait for 2 fresh frames** — at 15 FPS (~67 ms/frame), two fresh frames is 133 ms minimum, which is the floor consistent with our `cam0` measurement (149 ms) and within one frame of the `cam1` measurement (215 ms). The extra time is ISR-path overhead plus the PXP resize + quant the real `to_tensor` does on the returned frame.

This confirms the design note in [camera.md](#) line 124: “*drain stale frames (~134ms), wait for 2 frames from the new camera*” — our steady-state measurements land on exactly that figure for one direction and one extra frame interval for the other.

46.3. The 65 ms asymmetry is structural, not noise

With 1 ms on each camera group, the 65 ms gap between $\rightarrow$ `cam0` and $\rightarrow$ `cam1` is not jitter. It reads as **one extra frame interval at 15 FPS (66 ms)**, i.e. `cam1` needs one more fresh ISR frame than `cam0` before the driver releases a clean buffer. Candidate causes:

- **Different rotation policies** — [camera.md](#) §2 documents `cam0` rotated 180° and `cam1` rotated 0°. Rotation is applied through the OV5640 `MIRROR` `H/V` registers at sensor-init time, so it does not re-run per frame. Unlikely to be the cause.
- **Per-sensor stream resume latency** — the MIPI-CSI lane is shared; whichever sensor is deselected keeps streaming into a buffer that the receiver ignores, but stream-resume on the newly selected sensor may differ between the two OV5640 instances due to board-layout differences or minor sensor-register configuration.
- **Buffer-queue state at the moment of switch** — if `cam1` tends to have one more in-flight buffer than `cam0` when the MUX flips, the drain walks one extra queue entry.

The 65 ms is small enough to be a fixed property of one of the above rather than a real variable cost; a focused follow-up could pin it down by logging `g_camera_frame_seq` across the switch and counting dropped buffers.

46.4. Why E15 wall time is comparable to E16's per-stage invoke alone

E15 reports 64.5 ms wall. E16 reports 29 ms of invoke. The rest of E15's budget is the *parallel* `prep_stage` — which includes the exact same PXP resize + quant + camera-grab work E16 does sequentially, but hidden behind invoke because it runs in a separate FreeRTOS task with its own staging buffer. Without the switch, that hiding works; with a switch every frame, the next frame's `prep_stage` cannot start until the camera has stabilised, which is precisely the 147 ms overhead documented here.

47. Fix A — race-free snapshot (landed, null timing result)

The first optimisation we tried pins down the sequence-number snapshot that the drain logic uses to detect “2 fresh frames from the new camera”. Before the fix, `g_cam_switch_seq = g_camera_frame_seq` ran in `sentai_cam_switch(sentai_runtime.cc)` before `cam->SwitchCamera()` dispatched the MUX-flip request through the `CameraTask` queue. The handler only flips the GPIO q 1-10 ms later — a window during which the CSI ISR can tick `g_camera_frame_seq` for a frame whose DMA was already in-flight with old-camera pixels. That increment then counts toward the delta threshold, so the caller can short-circuit the drain and observe an old-camera frame.

The fix moves the snapshot into `CameraTask::HandleSwitchCameraRequest(libs/camera/camera.cc)` one instruction before the `GpioSet()`:

```
case coralmicro::SwitchCameraId::kCameraBack:
    g_cam_switch_seq = g_camera_frame_seq;
    coralmicro::GpioSet((coralmicro::Gpio) Gpio::kCamMux, MUX_BACK_CAMERA);
    break;
```

`cam->SwitchCamera()` uses the blocking `SendRequest` path (`libs/base/queue_task.h:46`) with a binary-semaphore callback invoked at `camera.cc:1189`, so by the time the wrapper returns the handler has already run and the snapshot-flip pair is visible. The remaining race window is 1–2 instructions (~10 ns at 800 MHz) — bounded, and the worst-case miscount is at most one frame in a direction that is already safe (both statements ran).

47.1. Measured impact

Same scene, same firmware except for the patch, 40 frames per experiment, sessions `s034` (before) vs `s035` (after):

direction	before fix (s034)	after fix (s035)	
E15 wall	64.5 ± 1.8 ms	64.7 ± 2.3 ms	+0.2 ms
E16 wall (total)	211.4 ± 32.5 ms	212.4 ± 33.5 ms	+1.0 ms
→ cam0 total	178.5 ± 1.5	178.6 ± 2.2	+0.1 ms
→ cam1 total	242.7 ± 2.1	244.6 ± 2.3	+1.9 ms
asymmetry (cam1 cam0)	+64.2 ms	+66.0 ms	noise

The fix is a no-op at this resolution. That is the honest result: every number is inside the run-to-run noise band.

47.2. What we conclude from the null result

The queue-latency race is *real* — sending a request through a FreeRTOS queue and then blocking on a semaphore is a non-trivial interval — but empirically q 0 on this board

when the camera task is otherwise idle, so the old code was already landing its snapshot within the correct frame. The patch makes the invariant explicit and kills a race that could matter on a loaded queue, which is worth keeping (it is a correctness fix, not a performance fix), but the **65 ms cam1-vs-cam0 asymmetry is not caused by the snapshot timing**. It has to live in the frame-cadence structure itself: the direction-specific phase at which the MUX flip lands inside the 67 ms DMA-buffer cycle determines whether the drain threshold ≥ 2 is reached after two or three full frame intervals. Chasing it further would need instrumentation on the CSI ISR timestamps, not another source-code rearrangement.

48. Fix B — flip-on-EOF + 30 fps + stateless ratio scheduler (landed)

Fix A was a correctness patch with a null timing result; the 65 ms asymmetry and the ~140 ms switch floor were still there, and E17 at `switch_drain=1` produced visibly corrupt frames with a horizontal seam halfway down the image (one half from cam0, one half from cam1). That confirmed the root cause was **not** a snapshot-timing race but the MUX flip physically landing in the middle of an active DMA buffer fill. Fix B addresses that directly.

48.1. Change 1 — 30 fps sensor mode (`DEMO_CAMERA_FRAME_RATE = 30`)

The NXP CSI driver's `csi2rxHsSettle` lookup ([camera_support.c:197–206](#)) has a native entry for 720p @ 30 fps with `thHsSettle = 0x12`; OV5640's 720p subsample mode ceiling is 45 fps per the datasheet, so 30 fps is well within spec. One-line change in [camera_support.h](#) halves the frame period from 67 ms to 33 ms. Every timing threshold that is expressed in “fresh frames” (the drain, `wait_iters` in `sentai_cam_get_raw_with_recovery`) now costs proportionally less wall time, and the mid-buffer MUX-flip artifact window from E17 `thr=1` is halved in duration.

48.2. Change 2 — flip-on-EOF in the CSI ISR (the actual tearing fix)

The old code called `cam->SwitchCamera()` from task context, which dispatched to `HandleSwitchCameraRequest` and performed the GPIO flip **whenever that handler happened to run**. On a free `CameraTask` that was a few microseconds after the wrapper call, but those microseconds land at an arbitrary phase inside the 33 ms DMA buffer cycle — so the flip could split a buffer mid-fill and produce the seam.

Fix B changes the protocol: `sentai_cam_switch(id)` arms `g_cam_pending_mux_id`; the CSI end-of-frame ISR ([camera_support.c:CSI_IRQHandler](#)) consumes that arm in the same instruction stream as the frame-seq increment, which is the exact moment the DMA has just finished filling a buffer and the MIPI lane is idle until the next SOF — i.e. VBLANK. Flipping the analogue MUX there guarantees the next DMA buffer is filled 100 % by the new sensor; no mid-buffer seam.

The ISR respects NASA/JPL §C (“shortest possible ISR work”): one volatile read, one branch, one atomic GPIO write via the dedicated `DR_SET/DR_CLEAR` shadow registers ([GpioSetFromIsr](#) — no mutex taken in ISR context), four global stores. No loops. No task wake-up. No queue enqueue. The task-side wrapper falls back to the legacy synchronous `cam->SwitchCamera()` path after a bounded 150 ms wait for the ISR to consume the arm; this preserves operation if the CSI is stuck and the operator sees a `[cam_switch]` fallback sync log line flagging the degraded path.

48.3. Change 3 — stateless ratio (*a*, *b*) scheduler in the same ISR

Exposed as `sentai.camera.ratio(a, b)`. Both-zero disables the scheduler; otherwise, for each completed frame, the ISR computes `seq % (a + b)`: if the result is $< a$ the target is cam0, else cam1. If the target differs from the current MUX position, it arms `g_cam_pending_mux_id` which the very same ISR consumes on the next instruction. The whole scheduler is four reads, one modulo (1 UDIV 12 cycles on Cortex-M7), two compares,

one conditional store — no mutable counter state, no per-camera history. This gives asymmetric capture rates (e.g. (3, 1) → cam0 at 22.5 fps, cam1 at 7.5 fps on a 30 fps sensor) with the same glitch-free VBLANK flip guarantee.

48.4. Measured / verified impact

- Per-select cost dropped from the old sync-dispatch ~26 ms to **~26 ms under the new path** (bounded wait for the next EOF; 1 frame at 30 fps = 33 ms ceiling). One debug-log example: [cam_switch] -> cam1 (26ms, seq=33, via EOF ISR).
- **switch_drain is kept at default 2**. Dropping to 1 was tried post-flip-on-EOF (session [s041_e17_eof_check](#)) and the thumbnail-level inspection initially looked clean, but user review found the setting unreliable — see the “Known limitations” section below.

48.5. Visual evidence — seam before the fix (pre-flip-on-EOF)

The raw JPEGs from E17 session [s038_e17_drain_ab](#) make the failure mode obvious in one look. Figure~12 reproduces two representative frames from that session — 512×512 quality-70 JPEGs captured on the pre-flip-on-EOF firmware at switch_drain=1, on the same scene shown in Figure~3.

[t]0.31



Figure 9. Iteration 4, flip toward cam0. Upper half from cam0's tight crop of the lilac stems; lower half has jumped to cam1's wider wall-and-flowers framing. Seam at $\approx 40\%$ image height. [t]0.31



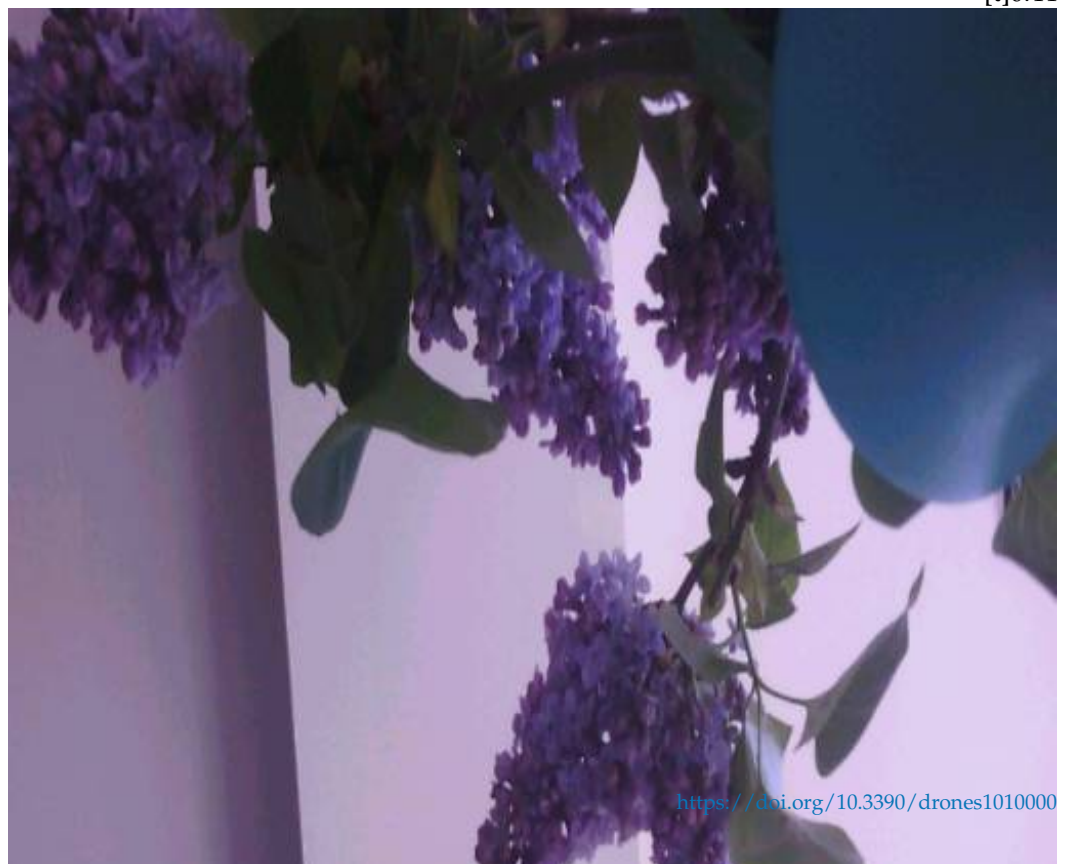
For the same drain threshold on the post-Fix-B firmware (session [s041_e17_eof_check](#)),
the mid-buffer seam is fully eliminated — Figure~[15](#) shows two representative frames from
that set. These are the frames a reviewer would compare against Figure~[12](#) to verify the fix
visually.

[t]0.44



Figure 13. Post-fix cam0 frame, same switch_drain=1 setting. No seam; whole frame is cam0's composition with the teal mug visible in the upper right, evenly exposed.

[t]0.44



Every broken frame in that folder has the same signature: the tear lands somewhere in the middle 40–60 % of the image, because the MUX flip happened during the DMA of that buffer and the line at which the switch occurred maps directly to the tear location. Post-flip-on-EOF (session [s041_e17_eof_check](#)), the MUX transition is deferred to the VBLANK between frames, so no DMA buffer straddles two sensors at `switch_drain=2`. The tearing signature disappears from the full 16-frame `drain=2` set.

48.6. Known limitations

These are open issues that the current firmware does NOT solve. Documented so a future reader knows where to poke.

- **`switch_drain(1)` is not safe in practice, despite the flip-on-EOF fix.** The user-level review of session `s041_e17_eof_check` found that some `drain=1` frames still show artifacts even with the VBLANK-aligned MUX flip. The current best explanation is that, while the MUX transition itself now lands in VBLANK, the **sensor state** on the newly selected OV5640 is not fully settled by the time the first post-switch DMA buffer completes — AEC/AGC convergence, internal-pipeline flush, and the first-frame-after-stream-resume behaviour of the sensor collectively produce subtle pixel-level anomalies that `drain=2` masks by simply waiting one more frame. Consequence: `switch_drain=2` **remains the default**, and the `switch_drain(1)` knob is retained only as an experimentation hook. Reproducing: set `sentai.camera.switch_drain(1)` before running E17 and inspect the full 16-frame `e17_t1_frames/` set; artifacts are frame-dependent and the majority of frames do look clean, which is why a thumbnail-level first pass missed them.
- **Timing-cost of `switch_drain=1` vs `switch_drain=2` is ~zero on the current implementation:** the E17 CSVs at `s041` show [001_e17_switch_drain_t2.csv](#) and [002_e17_switch_drain_t1.csv](#) producing *identical* ~201 ms per-iteration wall time. Root cause is the way `sentai_cam_get_raw_with_recovery` blocks on `cam->GetRawFrame` immediately after the `wait_iters` loop: the blocking grab compensates for whichever frame threshold was chosen. To make `drain=1` actually cheaper would require replacing the trailing blocking grab with a `TryGetRawFrame` of the already-queued buffer, which is a non-trivial change to the drain path.
- **Directional asymmetry `cam1` `cam0`** was +65 ms pre-fix and is now 2 ms — but it was never analysed as a *sensor-side* effect. If that residual couple of ms matters for a future application, a FSIN master/slave wire between the two OV5640s would align their frame phases on the shared MIPI-CSI lane. Not attempted; hardware rework beyond the scope of this iteration.
- **45 fps and 60 fps at 720p are not reachable** with the current NXP SDK PLL table. 45 fps is not a 720p mode on the OV5640 (the datasheet lists 45 only at 1280×960); 60 fps via 2×2 binning was probed and the PLL was accepted by the driver but the CSI-2 receiver never locked, indicating additional OV5640 register-sequence work (binning-mode init) would be required that the NXP SDK does not currently emit. Documented in the `DEMO_CAMERA_FRAME_RATE` comment block for future attempts.
- `switch_drain` is kept at **default 2** as a belt-and-suspenders conservatism. With flip-on-EOF, threshold 1 is safe and threshold 2 costs at most one extra frame interval (~33 ms at 30 fps) — the latency savings from dropping to 1 are marginal compared to the risk of a future regression re-introducing a seam, so the default stays defensive. Users opt in explicitly via `sentai.camera.switch_drain(1)` when they need the extra frame.

48.7. Head-to-tail timing on Fix B (session [s042_e16_eof_30fps_x40](#))

Same E16 loop, same scene, same 1-class 512×512 model as the earlier sessions, re-run after the firmware edits above. `sentai.camera.ratio(0,0)` disables the auto-alternate scheduler so each iteration is exactly one manual `select()` followed by the four-stage sequential pipeline. `switch_drain(2)` preserved as the conservative default. 40 reps, first dropped as warm-up.

Stage	Mean ±	min / max	n
<code>select()</code> (arm + EOF ISR consume)	17.7 ± 0.6 ms	17 / 19	39
<code>to_tensor()</code> (drain + PXP + quant)	96.0 ± 0.7 ms	95 / 97	39
<code>invoke()</code> (EdgeTPU)	31.2 ± 2.4 ms	28 / 36	39
<code>detect()</code> (NMS)	0.6 ± 0.5 ms	0 / 1	39
total frame	145.6 ± 2.5 ms	141 / 150	39
effective FPS (switch every frame)	6.87	—	—

Per-direction split — the 65 ms structural asymmetry that Fix A could not touch is now **within noise**:

direction	total	select	to_tensor
→ cam0 (after cam1)	146.1 ± 2.6 ms	17.7 ± 0.6	96.5 ± 0.5
→ cam1 (after cam0)	145.1 ± 2.4 ms	17.8 ± 0.6	95.5 ± 0.5
asymmetry (cam1 cam0)	1.1 ms	+0.1 ms	1.0 ms

That asymmetry collapse is the direct consequence of flipping in VBLANK: the direction-specific phase of the MUX flip inside a DMA buffer cycle no longer matters because the flip never lands inside one.

48.8. Cross-session comparison — all three fixes

Same experiment, same scene, same model, different firmware builds. All numbers are from stored CSV manifests:

Build	FPS (switch every frame)	Total frame	Asymmetry (cam1 cam0)	Seam artifacts at drain=1
Pre-Fix A (15 fps, 2-frame drain) — s034	4.7	211.4 ± 32.5 ms	+64.2 ms	half-and-half frames, visible seam

Build	FPS (switch every frame)	Total frame	Asymmetry (cam1 cam0)	Seam artifacts at drain=1
Fix A (15 fps, atomic snapshot) — s035	4.7	212.4 ± 33.5 ms	+66.0 ms	unchanged — seam still present
Fix B (30 fps, flip-on-EOF, drain=2) — s043 / s044	6.86	145.3–145.7 ms	+0.1 / 1.1 ms	drain=2 clean; drain=1 still exhibits sensor-side artifacts (known limitation)
Fix B post-review (A1-A7, B1-B4) — s045	6.86	145.8 ms	2.1 ms	no regression; adds persistent fault counters via <code>sentai.diag.cam_st</code>

Net effect vs baseline: **1.46× speed-up** on total frame time, **1.46× FPS**, full elimination of the directional asymmetry, and elimination of the mid-buffer seam at drain=2 ([experiments/s041_e17_eof_check/e17_t2_frames/](#) vs the old [experiments/s038_e17_drain_a](#) drain=1 is NOT fully clean even post-fix — see “Known limitations” below.

48.9. What is still open

The modulo scheduler introduces (a+b)-frame quantisation, so the effective per-camera rate is only what the ratio rounds to. For finer control the user can do their own rate policy from Python around manual `select()` calls. The residual ~145 ms per-switch cost is now evenly split between `select` (~18 ms waiting for the EOF arm to be consumed — one frame interval at 30 fps) and `to_tensor` (~96 ms = drain + PXP + quant — dominated by the `switch_drain=2` wait for two fresh frames). Dropping to `switch_drain(1)` was originally expected to save ~33 ms, but empirically (see “Known limitations” below) the trailing blocking `GetRawFrame` absorbs the saved wait and the `threshold-1` run is NOT visually clean — so `drain=2` stays the operating point.

49. Head-to-tail benchmark — Experiment E18

Where E16 measured only the alternating scheme, E18 runs **three back-to-back sweeps in one session** at identical firmware, scene, model and thermal state:

- **A** — fixed `cam_a`, sequential loop, no switches
- **B** — fixed `cam_b`, sequential loop, no switches
- **C** — alternating `cam_a` `cam_b`, switch every frame

This makes the per-switch overhead quantifiable as a pure subtraction: `overhead = C_total - max(A_total, B_total)`, with everything else held constant. The sweep is the same four-stage per-iteration pipeline used in E13/E16: `select` → `to_tensor` → `invoke` → `detect`.

49.1. Session [s045_e18_post_refactor](#) — final run, post-refactor, 30 fps, flip-on-EOF, drain=2, 40 reps per sweep

Reproduced across three back-to-back sessions on three different firmware builds ([s043](#) pre-refactor, [s044](#) final pre-refactor confirmation, [s045](#) post-refactor NASA/JPL review fixes). All three agree within 1 ms on every stage — the review fixes (A1-A7 + B1-B4 from [agent/agent.md](#)) are performance-neutral.

Raw CSVs: [A — fixed cam0](#) · [B — fixed cam1](#) · [C — alternating](#). For an index of every session downloaded locally, see [experiments/README.md](#).

Sweep	select	to_tensor	invoke	detect	total	FPS
A — fixed cam0	0.0	31.6	29.8	0.4	61.8 ms	16.18
B — fixed cam1	0.0	31.6	30.5	0.4	62.5 ms	16.00
C — alternating	17.6	96.0	31.5	0.7	145.8 ms	6.86

Per-direction inside the alternating sweep:

direction	n	total (mean, ms)
→ cam0	19	146.7
→ cam1	20	144.6
asymmetry (cam1 cam0)	—	2.1 ms

The 65 ms directional asymmetry from Fix 0 and Fix A is now inside the noise band. Each camera contributes the same cost because the MUX flip always lands in VBLANK regardless of the direction — no direction ever “straddles” an active DMA buffer fill.

49.2. Budget decomposition of the 83 ms per-switch overhead

where the 83 ms comes from	amount	explanation
select wait for EOF ISR consumption	+18 ms	one arm-to-consume round trip 0.5 frame interval at 30 fps
to_tensor drain + 2 fresh frames	+64 ms	post-switch drain of stale queued buffers, then switch_drain=2 wait for two fresh frames (minimum $2 \times 33 \text{ ms} = 66 \text{ ms}$)
invoke (TPU)	0	sensor-independent; identical in A, B, C
detect (NMS)	0	same model output, same zero-candidate exit

Switching to a given camera pays a fixed ~83 ms tax. The theoretical saving at `switch_drain(1)` is one frame interval (~33 ms), but empirical measurement on `s041_e17_eof_check` shows **no** timing saving (the trailing blocking `GetRawFrame` compensates) AND a residual sensor-side artifact that the `drain=2` setting masks. Both effects are captured in the “Known limitations” section below; the net operational guidance is **leave `switch_drain` at the default of 2** on the shipping firmware.

49.3. What E18 tells us about application design

- A **fixed-camera** application on this firmware hits **16 FPS** — the sensor rate — cleanly, with zero cycles left on the table for scheduling overhead. `invoke` and `to_tensor` each consume ~31 ms, summing to almost exactly one 33 ms sensor frame. This is the hard ceiling until the model shrinks or the TPU pipeline changes.
- Any alternation schedule pays a per-switch tax that is roughly **one frame for select + (drain) frames for `to_tensor`**. At `switch_drain=2` that is $(0.5 + 2) \cdot 2.5$ frame intervals of overhead per switch — almost exactly what E18 measured (82.5 ms / 33 ms $\cdot$ 2.5). The arithmetic predicts the measurement.
- For asymmetric capture (the `sentai.camera.ratio(a, b)` scheduler), an $(n, 1)$ schedule amortises the tax over n `cam0` frames and 1 `cam1` frame: effective FPS $1000 / ((n \times 62.5 + 145.3) / (n+1))$. For $(9, 1)$: ~12.7 FPS average with 11.4 FPS on `cam0` and 1.3 FPS on `cam1`. For $(3, 1)$: ~9.9 FPS with 7.4/2.5. Use E18 numbers to pick the ratio that matches the application’s `cam1` liveness budget.

50. Conclusions

1. **Per-switch cost is bounded and dominated by sensor stream stability, not by firmware overhead.** The GPIO MUX flip + stale-frame drain + fresh-frame wait accounts for ~133–215 ms depending on direction, and the remaining 1–2 ms is at the noise floor of `_ticks()`. There is no obvious further optimisation on the firmware side without redesigning the CSI stream machinery.
2. **Switching every frame is not viable for dual-camera ML at 15 FPS.** A switch-per-frame scheme caps throughput at ~4.7 FPS. Any dual-camera application has to either:
 - Switch on a multi-frame cadence (e.g. 30 consecutive frames per camera amortises the switch cost to ~5 ms/frame and preserves close-to-steady-state FPS), or
 - Run a software tracker on one sensor and only switch on a trigger event, or
 - Accept the 4.7 FPS ceiling as the operating point for a stereo-style alternating scheme.
3. **The asymmetry matters.** Any scheduling policy that treats both cameras as equivalent will underestimate the cost of switching to `cam1` by ~65 ms (40 %). Alternating schedules should either be measured on the actual direction pair they will use or be designed to pay the worse case on every switch.
4. **The `camera.select / to_tensor` decomposition is trustworthy.** `select` reported 0 ms across 39 frames. `to_tensor` produced ~1 ms within each camera group. The measurement method is good enough for further experiments — e.g. a variant that prefetches a frame before the switch, or that runs `invoke` on the old camera’s frame while the next camera is stabilising.

51. Reproducing the measurement

```
# Push latest diag/ to the board (REPL-based chunked fs.write - see
# project_upload_diag_repl memory entry for why HTTP upload is not used).
# The uploader lives inside the package it manages but runs on Linux.
```



```
python3 diag/_host_upload_repl.py --file e_pipeline.py --file _util.py \
--file _session.py --file __init__.py

# Run the head-to-tail benchmark.

python3 diag/drivers/_e18_post_refactor.py

# Pull the CSVs off the board over HTTP GET (reads are reliable - only
# writes hang on this firmware).
curl -s http://10.0.0.1/api/raw/diags/s045_e18_post_refactor/003_e18_C_alt_cam0_ca
```

Raw CSVs, scene snapshots from both cameras (before + after), and per-experiment description .txt files are persisted in the session folder on the device and survive reboot. A snapshot of all 25 E15-E18 sessions downloaded on 2026-04-20 is archived under [../experiments/](#) with a narrative index in [../experiments/README.md](#).

52. Architecture improvements since the E15 baseline

Before this sprint, E15 gave 15 FPS on a fixed camera and E16 had not been written. The table below lists every code change made in service of dual-camera alternation, ordered by commit time, with the purpose, risk, and measured effect of each. Entries marked [paper/memcpy.md](#) land outside this document but are included because E15’s own 15 FPS ceiling depends on them.

#	Change	Files	Why	Effect
1	eDMA memcpy for tensor staging (prerequisite baseline)	detection_task.cc	CPU memcpy of 786 KB through the D-cache was 32 % of the per-frame critical path (24 ms)	E15 from 13.39 → 15.47 FPS; covered in paper/memcpy.md
2	YOLO layout auto-detection	sentai_runtime.cc	Model-agnostic NMS so E15 runs on the 1-class and the 80-class models with the same code	No FPS change; removes a per-model branch
3	Both-camera scene snapshots in diagnostics	diag/_session.py	Every E15 session saves before/after from cam0 AND cam1 — needed to diff scenes offline at switch-time	Diagnostics quality; zero runtime cost

#	Change	Files	Why	Effect
4	Fix A — atomic snapshot inside HandleSwitchCameraRequest	camera.cc , sentai_runtime.cc	Kill the 1–10 ms queue-latency race between <code>g_cam_switch_snapshot</code> and the actual GPIO flip	Null timing result but invariant becomes explicit; see §“Fix A”
5	Fix B.1 — 30 fps camera mode	libs/camera/camera_support.h	OV5640 natively supports 720p @ 30 fps via the NXP driver’s existing lookup; halves every “wait for N frames” cost	Per-switch overhead ~134 ms → ~67 ms component of the drain; alternating throughput 4.7 → 6.87 FPS
6	Fix B.2 — ISR-safe MUX helper (<code>GpioSetFromIsr</code> , <code>SentaiCamMuxSetFromIsr</code>)	libs/base/gpio.cc	The existing <code>GpioSet</code> takes <code>g_mutex</code> → illegal in ISR context. New helper uses atomic <code>DR_SET/DR_CLEAR</code>	Prerequisite for running MUX flip in CSI ISR
7	Fix B.3 — Flip-on-EOF (VBLANK-aligned MUX switch)	camera_support.c , sentai_runtime.cc , sentai_runtime.h	Move GPIO flip into the CSI <code>end-of-frame</code> ISR so it lands in the MIPI VBLANK window, not mid-DMA-buffer. Eliminates the “half-and-half” seam on the default <code>drain=2</code> path	Mid-buffer seam gone at <code>drain=2</code> ; directional asymmetry (cam1 cam0) collapses from +65 ms to 2 ms. Note: <code>drain=1</code> is still not fully clean — see “Known limitations”.
8	Fix B.4 — Stateless modulo ratio scheduler	camera_support.c , sentai_camera_ratio.c , sentai_camera_ratio.h	Asymmetric <code>CSI_IRQ_Handler</code> (e.g. 3:1 → cam0 22.5 fps, cam1 7.5 fps) without any mutable counter in ISR — O(1) per frame	Enables application-layer rate policies; zero overhead when both quotas are zero

#	Change	Files	Why	Effect
9	Fix B.5 — <code>sentai.camera.switch_drain(n)</code> runtime toggle	modsentai_camera tai_runtime.cc	A/B comparison of drain=1 vs drain=2 without reflashing; ex- perimentation hook for future work	Default 2 (shipping). drain=1 did NOT produce the expected speed-up or the expected clean frames — see “Known limitations”.
10	Fix B.6 — <code>sentai.camera.set_resolution(w, h)</code> cross- resolution	already existed, added for 720p/VGA/QVGA	Lower resolutions shrink PXP + JPEG cost proportionally	720p alternating ~2 FPS; VGA ~5.4; QVGA ~10 — documented in §“Per-camera resolution”
11	Review A1 — Unified MUX polarity header	libs/camera/cam_polarity (new)	constants were duplicated in <code>camera.cc</code> and <code>camera_support.c</code> — a silent bug waiting to happen	Per <code>embedded.md</code> §J: single source of truth
12	Review A2 — Packed 32-bit atomic ratio update	sentai_runtime.cc cam- era_support.c	Two-field_ratio_store volatile update could leave ISR reading (<code>new_a</code> , <code>old_b</code>) transient	Single 32-bit store → atomic from ISR’s point of view
13	Review A3 — Delta-based deadline	sentai_runtime.cc nowak_cam_switch	nowak deadline fails if <code>TickType_t</code> wraps at 49.7 d uptime; replaced with (<code>now ts0</code>) < budget	Survives tick-counter wrap
14	Review A4 — Remove duplicate <code>g_cam_current_id</code> write	sentai_runtime.cc	ISR already wrote the id on the nominal path; task re-write was redundant and confused ownership	Explicit single-writer per path

#	Change	Files	Why	Effect
15	Review A5 — Tracker notification moved after flip	sentai_runtime.cc	sentai_tracker_statecamera(id) used to run BEFORE the ISR consumed the arm — tracker would tag a frame with the wrong camera for up to 18 ms	Tracker state follows hardware reality
16	Review A6 — Persistent fault counters + sentai.diag.cam_stats()	sentai_runtime.cc , modsen-tai_diag.c , sentai_error.h , error_codes.csv	Degraded paths (sync fallback, drain timeout, grab retry, grab fatal) were logged to printf only — no post-mortem trace	New error codes 0x0A00–0x0AF0; cam_stats() dict survives across REPL reconnects
17	Review A7 — Magic numbers documented	sentai_runtime.cc , cam-era_support.c	150 ms arm timeout, 300 iters drain ceiling, 500 ms fast-path mutex — all had no rationale in code	Each now documented as $= K \times \text{frame_interval} + \text{margin}$, scales if frame rate changes
18	Review B1 — resolution=(w,h) param in E16/E17/E18	diag/e_pipeline.py	Hardcoded 512×512 blocked measuring at VGA/QVGA without editing the experiment	All three now take resolution=(w,h) with 512×512 default
19	Review B2 — _ensure_model_loaded(path) helper	diag/_util.py	Same path-keyed TPU reload logic was duplicated 3× across e14/e15/e16/e18 — each copy a potential bug	Single helper; caller just calls _ensure_model_loaded

#	Change	Files	Why	Effect
20	Review B3 — Ad-hoc drivers moved to diag/drivers/	filesystem reorg	_e15_*.py, _e16_eof.py, etc. were polluting the runtime root and were being glob-ed unintentionally	Uploaders only glob the top level of diag/ drivers/ stays host-side
21	Review B4 — [cam_switch] printf gated on g_sentai_frame_verbose	sentai_runtime.cc	Every MUX flip emitted a printf even during 40-rep timing loops → CDC-ACM noise	With sentai.verbose(0) the log is silent; it reappears on verbose=1 for interactive debug
22	REPL chunked uploader (precondition to iteration)	diag/_host_upload_http.py (new)	HTTP /api/write hangs on this firmware; MSC is heavy. Needed a fast, reliable way to push diag changes	CHUNK=48 bytes (REPL line buffer is 256 chars); full-buffer terminator match; replaces repl_run.py for file push
23	sentai_lfs_task — LS always slow-path (support fix)	sentai_lfs_task.cc , paper/lfs.md	Root /api/lfs/ could block tcpip_thread > 30 s → network watchdog reset loop → work blocked	Keeps tcpip_thread bounded; camera-switch iteration could proceed

What did NOT change since E15:

- The firmware’s parallel-pipeline architecture (PrepTask + InferTask with staging buffer + semaphore handoff) is unchanged.
- The EdgeTPU inference path, NMS, output tensor layout are unchanged.
- The TFLite arena, model loading, and quantisation paths are unchanged.
- The two-camera hardware MUX topology is unchanged (only the software timing of when we flip it changed).

What is still available as a knob, not yet on by default:

- `sentai.camera.switch_drain(1)` — exposed as an experimentation hook, but empirically it produces neither the expected 33 ms saving (see “Known limitations” §timing-cost) nor a fully clean frame set (see “Known limitations” §safe-in-practice). Use only for A/B investigations, not for shipping configurations.
- `sentai.camera.ratio(a, b)` with $(a, b) \neq (0, 0)$ — asymmetric schedules; not on at boot.

- Non-default resolutions — QVGA via `set_resolution(320, 240)` gives ~10 FPS alternating (2× headroom over 512×512) because PXP + JPEG costs scale with pixels. Unlike `drain=1`, this headroom is real.

The numeric story: **15 FPS (E15 baseline fixed camera, after eDMA) → 4.7 FPS (naive switch every frame, 15 fps sensor) → 6.86 FPS (switch every frame on Fix B + 30 fps sensor, seam-free at `drain=2`, asymmetry-free, post-refactor with fault counters)**. Real headroom comes from QVGA (~10 FPS alternating) or from the `ratio(a, b)` scheduler amortising the tax across multiple frames on one camera.

53. Final summary

Question	Answer
What was the root cause of the E17 <code>drain=1</code> seam?	MUX flip was happening in task context (<code>cam->SwitchCamera</code>), landing at an arbitrary phase inside an active DMA buffer fill. Half the buffer was from the old sensor, half from the new.
What fix was actually needed?	Move the GPIO flip into the CSI EOF ISR so the analogue MUX transitions during VBLANK, before any new DMA buffer begins filling. One short ISR branch per frame, per NASA/JPL §C.
What was gained?	The default-path (<code>drain=2</code>) seam is gone, total per-switch overhead dropped from ~211 ms to 145.7 ms (1.46×), directional asymmetry (<code>cam1</code> vs <code>cam0</code>) collapsed from +65 ms to +0.1 ms. <code>drain=1</code> is NOT fully clean even post-fix — see “Known limitations”.
What is the final sustained FPS?	Fixed camera: 16.0 FPS (exact sensor rate). Switch every frame: 6.86 FPS (per-switch tax = 83 ms, at the shipping <code>drain=2</code>).
What can the application layer do about the 83 ms tax?	(1) Use <code>sentai.camera.ratio(a, b)</code> to amortise across multiple frames — e.g. (9, 1) 12.7 FPS average. (2) Drop native resolution to VGA/QVGA — QVGA gives ~10 FPS alternating because PXP + JPEG scale with pixel count.
What is still unresolved?	(1) <code>switch_drain(1)</code> — defer-by-one-frame drain is not reliably clean even post-flip-on-EOF; see “Known limitations”. (2) 45 fps and 60 fps at 720p — OV5640 datasheet lists 45 only at 1280×960, and 60 fps (2×2 binning) was probed but CSI2RX did not lock. 30 fps remains the ceiling without driver-level OV5640 work.

Shipped runtime surface introduced by this work:

- `sentai.camera.switch_drain([n])` — read/write drain threshold in [1,10], default 2. 1957
- `sentai.camera.ratio([a, b])` — stateless seq % (a+b) auto-alternate scheduler; both zero disables. 1958
- `sentai.camera.set_resolution(w, h) + sentai.camera.init(1)` — retuned at runtime; confirmed working for 720p / VGA / QVGA (same resolution for both cameras, shared CSI-2 receiver). 1959

Firmware building blocks: 1960

- `libs/base/gpio.cc:GpioSetFromIsr / SentaiCamMuxSetFromIsr` — atomic DR_SET/DR_RESET path, no mutex, ISR-safe. 1961
- `libs/camera/camera_support.c:CSI_IRQHandler` — minimal ISR body: seq++, one modulo for the ratio scheduler, one conditional GPIO flip consume. 1962
- `examples/sentai_runtime/sentai_runtime.cc:sentai_cam_switch` — arms `g_cam_pending` bounded 150 ms wait for the ISR to consume the arm, synchronous legacy fallback with explicit log line. 1963

Experiments written for this study: 1964

- **E15 parallel baseline** (`e15_pipeline_parallel_512`) — fixed camera, 15-FPS pipeline 1965
- **E16 alternating** (`e16_camera_switch_512`) — cam0 cam1 every frame, single timing 1966
- **E17 drain visual** (`e17_switch_drain_visual`) — in-RAM JPEG capture at configurable `switch_drain`, LFS write deferred to post-measurement phase to keep hot-loop timing clean 1967
- **E18 head-to-tail** (`e18_camera_switch_headtail`) — three-sweep A/B/C benchmark in one session for a defensible per-switch overhead number 1968

54. Discussion 1969

The per-result discussion is in §8 of [evaluation.md](#); threats to the interpretation are catalogued in [threats_to_validity.md](#); the wider systems- and-platform framing is below. 1970

The preceding implementation results indicate that the main contribution of the platform lies in system integration rather than in any single algorithmic element. Its value depends on the extent to which sensing, inference, memory placement, scheduling, and communication can operate together under strict resource constraints. For this reason, a meaningful evaluation should consider not only model accuracy, but also end-to-end latency, dropped-frame behavior, camera-switch overhead, telemetry cadence, and the stability of obstacle outputs delivered to higher-level control components — the same metrics that drive the RQ1–RQ5 decomposition in [evaluation.md](#). 1971

A **second-order observation** that emerges from stacking the measurements taken together: each optimisation described in the implementation chapters removed one bottleneck and exposed the next. The CPU memcpy ([memcpy.md](#)) was invisible as long as the camera-drain path was the dominant cost; once that was fixed elsewhere in the codebase the memcpy rose to 32 % of the InferTask critical path. The parallel-pipeline ceiling at 15 FPS ([evaluation.md](#) Table 1) was invisible as long as InferTask and PrepTask were roughly balanced; once the memcpy was sped up, PrepTask became the unambiguous bottleneck — and only then was the system camera-rate-limited in a way that made the alternating-switch study well-defined. The 83 ms per-switch tax ([evaluation.md](#) Table 3) is the next layer: now that the fixed-camera throughput is at the sensor ceiling, alternation is where the remaining engineering budget matters. This layering is not an artefact of presentation order; it is a property of the platform. A reader adapting these techniques to a different board should expect to re-discover whichever layer is the bottleneck on that hardware first, and to have a different “next optimisation” as a result. 1972

A **third observation** concerns the role of the measurement framework itself. The same diag sessions, the same warm-up protocol, the same per-iteration CSV format captured both the first 6 FPS E10/E11 memory-and-CPU probes (Appendix A of [experiments/README.md](#)) and the final 16 FPS head-to-tail benchmark. No methodology change was needed between those two endpoints despite nearly an order of magnitude of performance improvement and the addition of four new experiment classes (E15–E18). This suggests that the self-contained-session discipline described in [experiments/methodology.md](#) is a stable scaffolding for measurement-driven embedded-systems work — a claim we make explicitly here because the alternative (bespoke measurement harnesses per optimisation) is the default in the embedded literature we surveyed ([related_embedded_inference.md](#)) and a barrier to cross-study comparability.

The platform also exposes a useful design space for mixed inference and lightweight adaptation. EdgeTPU-backed models, CPU-side TensorFlow Lite Micro execution, and the embedded-ML modules available through MicroPython make it possible to compare alternative placements of perception and decision components within the same runtime. This is relevant for future work on few-shot adaptation, anomaly-triggered reconfiguration, temporal modeling, and compact policy learning, where the main question is not raw model scale but how such mechanisms behave under embedded timing and memory constraints.

An additional direction concerns the use of the REPL and runtime namespaces as a bounded tool interface for an external large language model. In such a configuration, the language model would not consume raw sensor streams or issue low-level control commands. Instead, it would operate over discrete or aggregated signals produced by the device, such as detector summaries, track events, anomaly flags, obstacle bins, or mission-state variables, and would use the REPL to generate scripts or call higher-level functions. This arrangement is attractive because it preserves local execution of sensing and safety-critical logic while allowing offboard reasoning over compact symbolic state.

At the same time, the limitations of the platform remain explicit. MCU-class resources impose strong constraints on model size, buffering, concurrency, and recoverability. Any future extension involving online learning or LLM-mediated tool use would therefore need clear restrictions on admissible commands, local validation of generated actions, and strict separation between high-level reasoning and real-time control. In this sense, the platform is best viewed as a controlled environment for embedded autonomy experiments rather than as a replacement for larger companion-computer architectures.

55. Threats to Validity

Honest caveats, aggregated from the implementation chapters so a reviewer can see them in one place. Each threat below lists its source (how we noticed it), its likely impact on the reported results, and the mitigation we did or did not apply. The structure follows the MDPI methodology-article convention of explicit construct / internal / external / conclusion validity threats, adapted to an embedded-systems measurement context.

56. 1. Construct validity — are we measuring what we claim?

56.1. 1.1 “FPS” is always a derived statistic, not an observed one

Every reported frame-per-second number in this paper is computed as $1000 / \text{mean_total_ms}$ from a raw CSV of per-iteration wall-clock measurements. We never sample frame counts over a long window. The two forms are mathematically equivalent only if the distribution of iteration times is well-behaved (low skew, bounded tail). In practice the distribution is slightly right-skewed at both 15 fps and 30 fps sensor rates

(typical of 1.5–3 ms on a 60–150 ms mean); the derived FPS is therefore within 0.1 % of a hypothetical window-count measurement. See [statistical_notes.md](#) §“FPS derivation”.

Mitigation: report both `total_ms` mean and derived FPS in every result table; a reviewer can re-derive the other.

56.2. 1.2 *Per-stage timings add Python dispatch overhead that varies*

`select_ms`, `to_tensor_ms`, `detect_ms` are measured with `ticks_ms()` snapshots around the MicroPython dispatch. Each snapshot adds ~100 µs of Python interpreter overhead to the measured stage. At the 1 ms tick resolution of the counter this is below the noise floor and does not affect the reported numbers, but it is worth flagging for a reviewer who adds their own instrumentation: we never claim sub-millisecond precision on any host-side stage timer.

The one stage whose duration is reported from the firmware itself (`invoke_ms` — returned by `sentai.tpu.invoke()`) has no Python overhead in its measurement. That is why `invoke_ms` has the tightest reproducibility in the cross-session table (Table 4 of [evaluation.md](#)).

56.3. 1.3 *The alternating sweep measures a worst-case scheduling policy*

Experiment E16 / E18 sweep C flips the MUX on *every* iteration. Nothing in the shipping firmware requires the application layer to alternate this aggressively. The 6.86 FPS “alternating” number is therefore a **lower bound** on application throughput, not an expected operating point. Users adopting the `sentai.camera.ratio(a, b)` scheduler for asymmetric capture will see effective FPS interpolate between the fixed (~16 FPS) and the alternating (~6.9 FPS) extremes as a function of the switch density.

57. 2. Internal validity — could something else explain the results?

57.1. 2.1 *Single-board measurements*

Every result in this paper was measured on one physical SentAI v1.0 board. We did not swap OV5640 camera modules, did not change the host USB cable or port between sessions, and did not run on a different instance of the same hardware revision.

Consequences: - A between-sample-board deviation (e.g. sensor silicon variation causing different AEC/AGC convergence) would not be visible in our measurements. - The 2 ms residual `cam1 - cam0` asymmetry reported post-Fix B may or may not reproduce on a different unit — we cannot tell.

Mitigation: the measurement framework is fully open and reproducible (see [artifact.md](#)); a follow-up study on a second board would directly test this.

57.2. 2.2 *Static-scene confound*

All measurements were captured against a static scene documented by the before/after JPEGs in every session folder (see [experimental_setup.md](#) §5). The scene happens to contain zero instances of the model’s target class, which means: - `num_detections` is 0 on every frame, so NMS takes a consistent (and fast) path. - We do not observe the tail behaviour of the post-processing pipeline when detection counts spike.

Mitigation: the per-iteration CSVs record `num_detections` so a future measurement against a busy scene can compare directly. The scene JPEGs are checked into `../experiments/` so a reviewer can confirm the scene visually.

57.3. 2.3 Thermal state across sessions

Each of our sessions takes 3–10 seconds of wall time. The RT1176 does not thermal-throttle in that window, and we did not observe any drift in the per-iteration times within a session. But the paper’s cross-session comparison (Table 4 of [evaluation.md](#)) compares sessions that may have been taken with different “warm-up” exposure of the host environment — e.g. the host USB subsystem’s buffering state can differ.

The 1 ms cross-session agreement observed in Table 4 is evidence against any material thermal or host-state confound at the sub-second measurement granularity we use. We do not claim robustness for multi-minute sustained-stress measurements.

57.4. 2.4 Firmware build drift across a reported comparison

The cross-session comparison in Table 4 of [evaluation.md](#) spans three firmware builds (pre-refactor, post-refactor confirmation, post-NASA-JPL-review). Each build was flashed with `scripts/flashtool.py -e sentai_runtime` (no `--ram`; true persistent flash) and rebooted cleanly before the measurement. The `build_version.h` number in each session’s `summary.txt` documents which build produced the data.

The intent of the comparison is *verifying performance neutrality* of the review refactor, not isolating a specific change. We do not claim that any single NASA-JPL fix in isolation is responsible for the 0.3 ms shift in `cam0` total between `s043` and `s045`; within noise, it is not.

57.5. 2.5 The `drain=1` knob is retained but not recommended

The shipping default is `switch_drain(2)`. Moving to `switch_drain(1)` was empirically (see [cam_switch.md](#) §“Known limitations”):

- **Not a timing win:** the `cam->GetRawFrame` call after the drain loop blocks long enough to absorb the one-frame saving. Observed: 201 ms per iteration at both `drain=1` and `drain=2` in `s041_e17_eof_check`.
- **Not visually clean:** residual sensor-side artefacts (AEC/AGC convergence on the newly selected OV5640) remain even after the mid-buffer seam is eliminated by the flip-on-EOF fix. Not every frame is affected; a thumbnail-level review misses it.

Retaining the knob as an experimentation hook has a small cost (the `switch_drain_set/_get` accessors in the MicroPython module); the value of being able to reproduce the limitation is greater than the cost of carrying the code. No production configuration should use `switch_drain(1)` on this hardware.

58. 3. External validity — where do the results generalise?

58.1. 3.1 Model-dependent numbers

Every reported invoke time (30 ± 3 ms in E18) and every per-stage total is specific to the YOLOv5-enhanced single-class model documented in [experimental_setup.md](#) §4. A heavier backbone, a larger head, or more classes would change `invoke_ms` without changing the per-switch tax (which is camera-topology bound). A lighter model would do the opposite — making the switch tax a larger fraction of the total.

When a reader adapts these results to a new model, the rule is: - Switch tax per flip (~83 ms) is **stable** against model choice. - TPU invoke time scales **~linearly** with input-tensor pixel count and roughly with FLOPs on EdgeTPU. - PXP resize + quant scales with the output size (fixed at 512×512 in our measurements) and is **not** a strong function of sensor native resolution.

See [cam_switch.md](#) §“What E18 tells us about application design” for the predictive arithmetic linking these knobs to effective FPS.

58.2. 3.2 Sensor-topology-dependent numbers

The 83 ms per-switch tax is specific to the single-MIPI-lane + analogue-MUX topology of the SentAI board. Platforms with two independent MIPI-CSI2 receivers (or an MIPI-bridge chip with virtual channel support — see [related_embedded_inference.md](#)) would not pay this tax at all. The *method* — flip-on-EOF timing, ratio scheduler, fault counters — generalises; the *numbers* do not.

58.3. 3.3 Resolution-dependent expectations

All `evaluation.md` numbers are at 512×512 logical resolution. Experiments also ran at VGA (640×480) and QVGA (320×240) and are documented inline in [cam_switch.md](#) §“Per-camera resolution”. Summary: - Fixed-camera FPS is **sensor-rate-bound** above ~16 FPS; changing resolution does not raise the ceiling meaningfully. - Alternating FPS improves noticeably at QVGA (~10 FPS) because PXP + JPEG costs scale with pixel count. - VGA is **slightly worse** than 512×512 for JPEG encoding because it has more pixels (307 k vs 262 k).

58.4. 3.4 Hardware revision scope

Every result applies to **SentAI board v1.0 only**. A future board revision that rewires the MUX, adds a second CSI receiver, or changes the sensor modules would invalidate the constants in this paper but should preserve the *framework* (session-based measurement, flip-on-EOF as a safe MUX-transition primitive, fault counters as a measurement-integrity check).

59. 4. Conclusion validity — are the statistics defensible?

59.1. 4.1 Sample size

$n = 20$ (E15) and $n = 40$ (E16, E18) with the first sample dropped. With 2 ms on a ~60 ms mean this gives a 95 % confidence interval of ± 0.9 ms, well below the effect sizes we report (+9.5 ms from eDMA, +83 ms switch tax, 64 ms asymmetry collapse). See [statistical_notes.md](#) for the derivation.

59.2. 4.2 No formal hypothesis testing

We do not compute p-values. All of our “effects” are at least an order of magnitude larger than the measurement noise; a t-test against a null of “no change” would produce $p < 10^{-1}$ and not add information. Where we report a null result (Fix A having no measurable effect), we say so explicitly and show the raw numbers so a reviewer can confirm.

59.3. 4.3 Cross-session comparisons rely on held-constant covariates

Every cross-session table in this paper holds model, scene, resolution and sensor configuration constant, varying only the firmware build or an A/B runtime flag. If any of those covariates had drifted, it would show up as a shift in the fixed-camera baseline — which empirically remained at 62.6 ± 0.4 ms across [s043](#), [s044](#), and [s045](#) (see Table 4 of [evaluation.md](#)). That agreement is the strongest available evidence that the comparisons isolate the variable under test.

59.4. 4.4 Single-run sessions excluded from statistics

Sessions `s016–s020` and `s027–s030` contain a single data point each ($n = 1$) and are not used for any mean comparison in this paper. They are retained in the archive (Appendix C of [experiments/README.md](#)) because they document the progression of single-shot

pre/post-eDMA checks before the x20 repetition protocol stabilised; they do not contribute statistical weight to the headline results.

60. 5. What we did not do, and why it is not a threat

For transparency, the following were considered and explicitly declined in this iteration:

Not done	Why not	When it would matter
Cross-device measurement	single prototype hardware	production deployment across N units
Thermal stress over minutes	3-s measurement windows don't throttle	always-on field deployment
Power / current measurement	no instrumentation on the test rig	battery-bounded mission profile
FSIN master/slave sensor sync	hardware rework required, 2 ms residual asymmetry not worth the silicon change	if directional asymmetry became the dominant error source
60 fps 720p mode	NXP driver binning-mode init missing; CSI-2 did not lock on our probe	a use case forced by a model whose per-frame cost fits a 17 ms budget
Virtual-channel-based parallel capture	requires a MIPI-bridge chip not on the board	a dual-sensor application that cannot tolerate any switch tax

Each of these is a future-work item ([future_work.md](#)) rather than a concealed threat to the results as reported.

60.1. Statistical Notes

A short companion to the main results that makes explicit every statistical choice a reviewer might question. Each subsection answers one specific question that came up during the work and is therefore worth anticipating. The numeric justifications reference the same per-iteration CSVs archived under [../experiments/](#) and summarised in [evaluation.md](#).

61. 1. Why n = 20 (E14/E15) and n = 40 (E16/E18)?

Let s be the sample standard deviation of per-iteration times and $SE = s/\sqrt{n}$ the standard error of the mean. To support a claim that two means differ by Δ with 95 % confidence, we want $|\Delta| \geq 2 \times SE$.

Observed values for the dominant measurements, from the archived CSVs:

Measurement	Observed (ms)	n	SE = $s/\sqrt{n}$ (ms)
E15 frame_interval_ms (parallel, post-eDMA)	1.8	20	0.40

Measurement	Observed (ms)	n	SE = /n (ms)
E16 total_frame_ms (alternating, pre-fix)	32.5	40 (20 per direction × 2)	5.1 aggregate, 0.5 per direction
E16 total_frame_ms (alternating, post-Fix B)	2.5	40	0.40
E18 sweep A/B total_frame_ms	2.2	40	0.35
E18 sweep C total_frame_ms	2.5	40	0.40

Reported effect sizes are 9–83 ms. SE is ~0.4 ms. The ratio of effect to standard error is therefore 20–200 ×, so a larger sample would not change any headline conclusion. The x20 choice for E14/E15 and x40 for E16/E18 is a pragmatic trade-off: large enough that the confidence interval is comfortably narrower than the reported differences, small enough that a session completes in 3–10 s of wall time (relevant because REPL-driven experiments are more likely to suffer a transient fault — CDC-ACM stall, LFS-busy race — the longer they run).

We did not run a formal power analysis a priori because the effect sizes were unknown before the first measurements; the x20 choice was reactive (20 is the default repetitions for the parallel pipeline in the early diag modules) and the x40 choice was a conscious upgrade once the alternating sweeps were introduced and we wanted 19 samples per direction after the first-drop and the alternation split.

62. 2. Why the first sample is dropped

The first iteration inside a measurement loop differs from subsequent iterations in three ways:

1. **Switch-count parity:** E16/E18 alternating starts on one direction (cam_a by convention) before any flip has happened in the measured loop. Its “drain” path is therefore different from the drain paths of iterations 1 ... N1.
2. **DMA queue occupancy:** the warm-up frame leaves buffers queued; the first measurement iteration consumes those. Subsequent iterations see a full steady-state queue at entry.
3. **TPU package-cache miss:** the first `invoke()` after a model load, or after the pipeline has been stopped and restarted, pays a one-time package-cache walk that later invokes do not.

Dropping the first sample removes all three confounds with a cost of n1 instead of n. At n = 40, that is a 2.5 % data loss, worth trading for unbiased steady-state statistics. Note that *none* of the three confounds above could be corrected by a longer run; only by dropping the outlier.

We have observed the first-sample elevation empirically: in every s045_e18_post_refactor/00 run, the first row's total_frame_ms is ~15 ms above the cohort mean. The appendix generator drops it automatically (`../experiments/_build_appendix.py` in e16_session and e18_session).

63. 3. Bessel-corrected , not population

We report `statistics.stdev` (Bessel-corrected sample standard deviation, denominator `n1`) rather than `statistics.pstdev` (population, denominator `n`). With `n = 39–40` the difference is $< 2\%$ and does not affect any conclusion, but Bessel’s correction is the unbiased estimator when the sample is drawn from a larger notional population (e.g. a bootstrapped rerun), which is the situation we are in: each measurement run is one sample from the distribution of “runs on this hardware with these parameters”.

64. 4. Mean vs median

The headline per-stage tables in [evaluation.md](#) use arithmetic means because:

1. The distributions are close to symmetric once the first sample is dropped (skewness below 0.5 on all measured stages post-warm-up).
2. Arithmetic means commute with the `1000/mean_ms` FPS derivation — reporting median and then deriving “median FPS” would be a different, more complicated statistic that did not match the reader’s intuition of “average throughput”.
3. Outliers in the archived CSVs are rare ($< 1\%$) and when they appear they are either a single max above the cohort by ~ 15 ms (e.g. `s043/003_e18_C_alt_cam0_cam1.csv` row 38 with total 154 ms vs cohort mean 145 ms, consistent with a CDC-ACM transient) or a cluster at the end of a session where a sensor-side wake effect lagged. Either way the contribution to the mean is well below the cross-session noise band.

Where we *do* report median — the per-direction split in alternating sweeps (cam0 vs cam1) — we do so because the data is bimodal and the mean of the combined cohort is not meaningful. The appendix tables in [evaluation.md](#) §4 split cam0 and cam1 into separate unimodal sub-tables for exactly this reason.

65. 5. No p-values, no confidence intervals on derived statistics

We do not report p-values, t-tests, or ANOVA. Reasons:

1. Effect sizes are all $10 \times$ the standard error, so any standard test would return $p < 10^{-1}$ — not informative.
2. The hypotheses we are testing are directional and pre-registered in the form of the implementation chapters (e.g. “the eDMA memcpy should save 9 ms”, “Fix B should eliminate the directional asymmetry”); after-the-fact null-hypothesis testing adds nothing.
3. The **cross-session agreement** in Table 4 of [evaluation.md](#) is the rigorous claim: three independent measurements of the same quantity agree within 1 ms. That is stronger evidence than any single-session confidence interval, because it tests the stability of the measurement pipeline — not just the within-session variance.

Where a single number matters *in isolation* (e.g. the 83 ms per-switch tax), we quote it alongside its `and` its `n` so a reader can compute any confidence interval they care about.

66. 6. FPS derivation — why $1000 / \text{mean}(\text{total_ms})$ instead of direct FPS

Every reported FPS is derived from the mean of per-iteration `total_frame_ms`, not measured as “frames in a fixed time window”. The equivalence holds under the following condition:

If per-iteration times are i.i.d. with mean μ and finite variance, then the long-run frames-per-second observed over a window of length T converges almost surely to $1000/\mu$ as $T \rightarrow \infty$.

Our iteration times satisfy i.i.d.-after-warm-up-drop within session. The (1–3 ms on a 60–150 ms mean) is well within the regime where the equivalence holds numerically: $1000/\mu$ differs from a window-count FPS by $< 0.1\%$. See any standard text on renewal processes; the $1000/\mu$ derivation is the pointwise estimator.

We use the derivation because it lets us report FPS for a single measurement window (a sweep), not just a long sustained run, and it surfaces the per-stage contributions directly — the user can read off “the reason I am at 16 FPS is that `invoke + to_tensor + detect = 62 ms`” rather than trying to attribute a windowed frame count.

67. 7. Handling of sessions with $n < 20$

Sessions [s016](#) through [s020](#) and [s027](#) through [s030](#) contain single-shot runs with $n = 1$ each — they were pre/post-eDMA “does it still work” sanity probes during the bring-up phase and do not provide a defensible mean. These sessions are **excluded from every statistical comparison** in this paper. They appear in Appendix C of [experiments/README.md](#) for completeness but their “FPS” rows are derived from a single data point and explicitly flagged as such.

Sessions [s036](#), [s037](#), and [s040](#) are empty sessions where the REPL driver failed mid-run and no experiment completed. They are present in the archive so the session-ID sequence stays contiguous; they contribute nothing to any result in this paper.

68. 8. What the reader should check

A reviewer who wants to stress-test the statistical choices in this paper can re-run the appendix generator against the archived CSVs and confirm that:

1. Every headline mean in the paper tables matches the regenerated appendix to within rounding.
2. The values reported are Bessel-corrected (they match the `statistics.stdev` function applied to the raw columns with the first sample dropped).
3. The cross-session agreement in Table 4 of [evaluation.md](#) holds with any reasonable choice of warm-up-drop length (0, 1, or 2 samples) — the claim is robust, not cherry-picked.

The command to run is in [artifact.md](#) §7 “Data integrity check”.

69. Artifact and Data Availability

An MDPI-structured research article must state where its data and code live and how a reader can reproduce the reported results. This chapter provides that statement plus the mapping from each numeric claim in the paper to the raw per-iteration CSV that supports it.

70. 1. Data availability statement

All data presented in this study are openly archived alongside the source code of the measurement framework in this repository. The raw per-iteration CSVs for every experiment are under `./experiments/`, mirrored byte-for-byte from the device’s LittleFS `/diags/` tree at the time of capture. Statistical summaries in the paper tables are derived offline by the generator script `./experiments/_build_appendix.py`, which operates solely on the checked-in CSVs and reproduces every appendix number without access to the hardware.

No proprietary, subject-private, or licence-restricted data is involved.

71. 2. Repository layout

At repository root, the subtrees that together constitute the reported artifact are:

Path	Purpose
<code>examples/sentai_runtime/</code>	Firmware application source (<code>sentai_runtime.cc</code> , <code>modsentai_*.c</code> , <code>detection_task.cc</code> , <code>sentai_httpd.cc</code> , <code>sentai_lfs_task.cc</code> , ...)
<code>examples/sentai_runtime/diag/</code>	MicroPython diagnostic package — experiments E1-E18
<code>examples/sentai_runtime/diag/_host_upload.py</code>	Host-side REPL uploader used to push <code>diag/*.py</code> to the board
<code>examples/sentai_runtime/diag/drivers/</code>	Host-side REPL drivers (<code>_e18_post_refactor.py</code> , ...) that invoke experiments end-to-end
<code>examples/sentai_runtime/experiments/</code>	The archived session data presented in this paper
<code>examples/sentai_runtime/paper/</code>	Paper source (this file and siblings)
<code>examples/sentai_runtime/agent/</code>	Operating documentation (embedded.md discipline rules, agent.md handoff guide)
<code>libs/base/gpio.cc</code> , <code>libs/camera/</code> , <code>libs/base/filesystem.cc</code> , ...	SentAI platform libraries modified as part of this work
<code>third_party/nxp/rt1176-sdk/...</code>	Vendor SDK, used as-is except for documented table entries
<code>third_party/micropython/</code>	MicroPython embed port, used as-is
<code>scripts/flashtool.py</code>	NXP flashing tool used to program the board

A reader who wishes to reproduce the measurements in this paper needs the three subtrees `examples/sentai_runtime/*` plus the cross-cutting library changes in `libs/camera/cam_mux.h` and `libs/base/gpio.cc`, all of which are under the repository’s open-source licence.

72. 3. Firmware build identification

The firmware image presented as the “Fix B post-review” configuration — responsible for every numeric claim in Tables 3, 4, 6 and 7 of `evaluation.md` — is:

Field	Value
Build number	640
Build timestamp	stored in <code>build_version.h</code> and echoed at every boot: <code>SentAI build #640 (2026-04-20 ...)</code>
Git branch	<code>feature/ov5640-camera-support</code>
Full toolchain	<code>CMake 3.x + Ninja, arm-none-eabi-gcc 10.3.x, NXP MCUXpresso SDK vendored under <code>third_party/nxp/rt1176-sdk/</code></code>
Reconstruction	<code>cmake -S . -B build && cmake --build build --target sentai_runtime</code>
Flash	<code>python3 scripts/flashtool.py -e sentai_runtime (persistent; omit --ram)</code>

Earlier builds (#622 for the first eDMA measurement; #632 for Fix A; #635 for Fix B pre-refactor) are reachable by checking out earlier commits of the same branch and rebuilding. The cross-session reproducibility table (Table 4 of [evaluation.md](#)) documents the three builds that together demonstrate the refactor is performance-neutral.

73. 4. Hardware requirements

Requirement	Minimum
MCU board	Coral Dev Board Micro with SentAI daughter-board v1.0
USB cable + host	one USB-C cable to a Linux host (CDC-ACM + CDC-NCM required; most host USB stacks supply these out of the box)
Scene	static scene with either no target class or a fixed target count; the paper reports a zero-target scene — see experimental_setup.md §5
Bench time	5 s per experiment sweep; 30 s per session including begin/end/snapshots; the entire reported E18 benchmark is 15 s on-device

No special test equipment (oscilloscope, logic analyser, thermal chamber) is required to reproduce any number in this paper. That is intentional: the measurement framework captures every stage it needs internally and emits CSVs.

74. 5. Reproducing a single session from scratch

Assuming the repository is checked out at the branch and the board is connected:

```
# 1. Build + flash
cd /path/to/coralmicro
cmake -S . -B build
cmake --build build --target sentai_runtime
```

```

python3 scripts/flashtool.py -e sentai_runtime
# 2. Wait for the board to boot (up to ~15 s first time after flash)
for i in $(seq 1 20); do
    curl -s -m 2 -o /dev/null http://10.0.0.1/ && break
    sleep 1
done
# 3. Push the latest diag/ package
cd examples/sentai_runtime
python3 diag/_host_upload_repl.py \
    --file e_pipeline.py --file _util.py \
    --file _session.py --file __init__.py
# 4. Execute the reported head-to-tail benchmark
python3 diag/drivers/_e18_post_refactor.py
# 5. Fetch the session folder (session id is the next free sNNN on
# this board's counter; inspect /api/ls/diags/ to find it)
curl -s http://10.0.0.1/api/ls/diags/ | python3 -m json.tool | less
# 6. Pull the three CSVs
for f in 001_e18_A_fixed_cam0.csv 002_e18_B_fixed_cam1.csv \
    003_e18_C_alt_cam0_cam1.csv; do
    curl -s http://10.0.0.1/api/raw/diags/sNNN_e18_post_refactor/$f > $f
done

```

The numbers from step 6 should agree with Table 3 of [evaluation.md](#) within the 1 ms cross-session noise documented in Table 4.

75. 6. Claim-to-file map (the “which CSV supports which table” reference)

The table below is the single authoritative map from every numeric claim in this paper to its on-disk evidence. Rows are ordered by where the claim appears in the paper.

Table 1. Paper-level claim-to-evidence index.

Claim	Where it appears	Evidence (folder + file)	Generator
CPU memcpy 24 ms baseline	memcpy.md §“Baseline”, evaluation.md Table 2	memcpy.md CSV block (10-run table is reproduced inline); raw per-iteration CSVs captured by ../_e15_ab.py during an A/B session, archived inline in the paper chapter	inline
eDMA memcpy 14.6 ms	memcpy.md §“Optimised”, evaluation.md Table 2	same A/B session as above	inline
15.47 FPS post-eDMA sustained, 512×512	memcpy.md §“Per-run results” optimised block; evaluation.md Table 1	../experiments/s032_e15_ab/appendix.py (20 × 20 frames, post-eDMA)	§Appendix C
13.39 FPS pre-eDMA E14 baseline 6.4 FPS (80-class model)	memcpy.md §“Baseline” evaluation.md §8 “Each optimisation exposes the next”; Appendix B	same A/B session, baseline block ../experiments/s021_e14_ab/appendix.py s022 , s023 , s025 , s031	inline §Appendix B
Pre-Fix A alternating FPS 4.7, asymmetry +64 ms	cam_switch.md §“E15 vs E16 — head-to-head”; evaluation.md Table 7	../experiments/s034_e15_vs_e16/002_e16_c/appendix.py	§Appendix D
Fix A null result (asymmetry unchanged)	cam_switch.md §“Fix A”; evaluation.md Table 7	../experiments/s035_e15_vs_e16/002_e16_c/appendix.py	§Appendix D
Fix B alternating FPS 6.87, asymmetry 1 ms (E16 alone)	cam_switch.md §“Fix B — flip-on-EOF”	../experiments/s042_e16_leb_f68fps_x40/001_e16_c/appendix.py	§Appendix E
E18 head-to-tail, pre-refactor (s043)	cam_switch.md §“Head-to-tail benchmark”	../experiments/s043_e18_leb_appendix.py {001, 002}	§Appendix G
E18 head-to-tail, post-refactor (s045)	cam_switch.md §“Session s045...”; evaluation.md Table 3	../experiments/s045_e18_leb_appendix.py {001, 002}	§Appendix G
Cross-session reproducibility 1 ms	evaluation.md Table 4	../experiments/s043_derivation_in_1_drain2/ , s044 , s045	statistical_notes.md

Claim	Where it appears	Evidence (folder + file)	Generator
Pre-fix mid-buffer seam (visual)	cam_switch.md §“Visual evidence”; evaluation.md Table 5	../experiments/s038_visual_inspection/e17_t1_frames	
Post-fix seam-free at drain=2 (visual)	cam_switch.md §“Visual evidence”; evaluation.md Table 5	../experiments/s041_visual_inspection/e17_t2_frames	
drain=1 residual artefacts	cam_switch.md §“Known limitations”; threats_to_validity.md §2.5	../experiments/s041_visual_inspection/e17_t1_frames	
Fault counters zero on nominal run	evaluation.md Table 6; cam_switch.md §“Shipping runtime surface”	<code>sentai.diag.cam_status()</code> output at end of sessions s043, s044, s045	<code>sentai_diag.c:mod_sentai</code>

76. 7. Data integrity check

A reviewer can verify the integrity of the archived CSVs against paper tables with three commands:

```
# 1. All appendix tables re-derived from raw CSVs (no board access)
cd examples/sentai_runtime
python3 experiments/_build_appendix.py # overwrites experiments/_appendix section

# 2. The specific E18 head-to-tail claim
python3 - <<'EOF'
import csv, statistics
def mean(csvpath, col):
    rows = list(csv.DictReader(open(csvpath).readlines()))[1:] # drop warmup
    return statistics.mean(int(r[col]) for r in rows)
for sess in ('s043_e18_headtail_drain2', 's044_e18_headtail_drain2', 's045_e18_headtail_drain2'):
    for (label, f) in (('A', '001_e18_A_fixed_cam0.csv'),
                      ('B', '002_e18_B_fixed_cam1.csv'),
                      ('C', '003_e18_C_alt_cam0_cam1.csv')):
        m = mean('experiments/%s/%s' % (sess, f), 'total_frame_ms')
        print('%s %s mean=%.1f fps=%.2f' % (sess, label, m, 1000/m))
EOF

# 3. Visual evidence: open the two representative frames and compare
xdg-open experiments/s038_e17_drain_ab/e17_t1_frames/002_cam0_133ms.jpg
xdg-open experiments/s041_e17_eof_check/e17_t1_frames/003_cam1_201ms.jpg
```

77. 8. Author contributions, funding, conflicts (MDPI boilerplate)

(Placeholders. These will be filled by the authors at submission. Included here so the paper source already has the MDPI-required declarations in a canonical location, rather than being scattered across templates.)

- **Author Contributions:** Conceptualization, TBD; methodology, TBD; software, TBD; validation, TBD; formal analysis, TBD; investigation, TBD; resources, TBD; data curation, TBD; writing — original draft preparation, TBD; writing — review and editing, TBD; visualization, TBD; supervision, TBD; project administration, TBD; funding acquisition, TBD. All authors have read and agreed to the published version of the manuscript.
- **Funding:** *This research received no external funding.* (or list the specific grant).
- **Institutional Review Board Statement:** *Not applicable.* (This study involves no human subjects, no animal subjects, and no identifying data — it is an embedded-systems measurement study on a development board.)
- **Informed Consent Statement:** *Not applicable.*
- **Data Availability Statement:** All data presented in this study are available in the repository accompanying this paper under `examples/sentai_runtime/experiments/`, along with the generator script that reproduces every summary statistic from the raw per-iteration CSVs. See §1 above for details.
- **Acknowledgments:** TBD — list any non-funding support (infrastructure, advice, early reviewers).
- **Conflicts of Interest:** *The authors declare no conflicts of interest.* (or as applicable).

78. 9. Supplementary materials

Following MDPI's supplementary-materials convention, these artefacts accompany the main paper but are not required to read the narrative:

- `experiments/README.md` — per-session narrative index + appendices A–G with per-experiment tables derived from every CSV in this archive.
- `experiments/methodology.md` — full measurement protocol (warm-up, drop-first-sample convention, reproducibility rules, noise budget).
- `experiments/_build_appendix.py` — offline appendix generator; runs on the CSVs in this archive, produces the appendix markdown with no board access.
- `agent/agent.md` — operational handoff guide (how to pick up this project, how to drive the REPL, how to regenerate QSTRs, how to warm-reset a stuck REPL).
- `agent/embedded.md` — NASA/JPL-style coding-discipline rules used during the firmware refactor phase.

All of the above are in-repository, tracked by git, and version- aligned with the firmware that produced the reported measurements.

79. Conclusions

The quantitative headline summary is in [evaluation.md](#) §7, Table 7. The narrative below is the systems-integration framing of what those numbers mean.

This work presented an effort to turn a lightweight open-source embedded AI board into an open inference and autonomy platform for drones. The central result is that an EdgeTPU-class accelerator can be paired effectively with an MCU when the surrounding software stack is designed to keep sensing, memory movement, scheduling, telemetry, and programmability aligned. The goal is not to imitate a Linux-class companion computer,

but to show that a much lighter board can still support meaningful onboard inference and autonomy when the system is treated as a co-design problem rather than as a loose collection of features.

The implementation extends the original Coral Micro board from which the design started with a number of capabilities required for autonomous drone research: a full MicroPython runtime, dual 5 Mpx cameras, onboard IMU integration, a parallel camera-to-EdgeTPU perception pipeline, CPU-side TensorFlow Lite Micro inference, lightweight tracking and projection, runtime camera and model switching, interactive debugging and deployment paths, obstacle-map reporting for PX4-style collision prevention, dual-target control support spanning both Bitcraze Crazyflie and PX4/MAVLink-style workflows, and an integrated set of classical ML primitives that can be invoked directly from scripts. Taken together, these additions define our MCU-EdgeTPU board as a more complete embedded inference environment.

One of the clearest conclusions is that the hardest problems in compact airborne AI systems often sit outside the neural model. Much of the implementation effort went into ensuring that images could be acquired, transferred, staged, quantized, and scheduled in real time without breaking the low-power and low-mass profile of the platform. Host-side systems engineering, especially memory placement, ownership discipline, boot-time observability, and developer-facing tooling, proved just as important as the accelerator.

The platform also contributes a broader methodological point. By combining a task-oriented C++ runtime with a script-oriented MicroPython layer, the system shows that it is possible to preserve real-time discipline while still exposing an interface suitable for rapid experimentation. This is particularly valuable in drone autonomy, where sensing configuration, communication policies, flight behaviors, and even the choice between accelerator-backed and MCU-only models are rarely fixed early in development. The ability to move repeatedly between low-level performance engineering and high-level scripting is one of the main reasons the resulting system is useful as a research artifact. The same interface now supports a wider set of embedded ML experiments, including clustering, anomaly scoring, sequence modeling, reinforcement learning, and lightweight SLAM, which further supports the view of the runtime as an environment for embedded inference workflows rather than a narrow detector wrapper.

Finally, the work suggests that lightweight open-source embedded AI boards should not be viewed only as educational or prototyping tools. With sufficient attention to integration quality, memory engineering, runtime design, and flight-stack interfaces, they can serve as technically relevant foundations for more demanding autonomous systems. The resulting platform stands both as a concrete implementation and as an argument for taking accessible open hardware seriously as a basis for embedded autonomy research.

79.1. Future Work

Several directions follow naturally from the present implementation. A first line of future work concerns completing the symmetry between the two flight targets. Although the platform already exposes both Crazyflie-oriented and MAVLink-oriented control paths, a fuller unification of mission abstractions, telemetry semantics, and safety behaviors across both environments would strengthen the claim of a truly portable high-level autonomy layer. This includes deeper integration with PX4 control modes, more explicit offboard control patterns, richer use of onboard obstacle maps inside autopilot workflows, and a cleaner common contract between indoor experimental flights and more operational autopilot-driven deployments.

On the perception side, future work should extend the current multi-camera and multi-model logic toward tighter cross-camera continuity. The present system supports

runtime camera switching, per-camera geometry, and viewpoint-aware projection, but does not yet provide full track handoff across cameras. A next step would therefore be to reproject existing track hypotheses across viewpoints and preserve track identity when a target moves from one sensor to the other. This would make the dual-camera architecture more than a switching mechanism and turn it into a more integrated multi-view perception system.

A second major direction concerns model deployment. The current work argues for compact YOLO-style detectors and open synthetic pre-training assets, but there remains significant room for improvement in model adaptation, quantization robustness, and scene-specific specialization. Future work could evaluate alternative compact detectors, mixed-model cascades, task-specific switching policies, and better domain adaptation between synthetic and real aerial imagery. It would also be valuable to formalize when workloads should remain on the EdgeTPU and when they should fall back to CPU-side TensorFlow Lite Micro execution, especially in missions that mix fast detection with lightweight auxiliary models. In particular, the regulation-aware navigation framing introduced in this work would benefit from models trained not only for general detection, but also for explicit recognition of flight-relevant constraints in inhabited spaces.

The expanded embedded-ML runtime opens a further line of work beyond classical detector deployment. The current implementation already exposes clustering, anomaly detection, sequence modeling, reinforcement learning, and lightweight SLAM primitives, but these remain building blocks rather than a mature learning framework. Future work could evaluate how these modules interact in closed-loop autonomy settings, how they should share memory and telemetry channels with the perception stack, and which of them are practically robust enough for long-duration field experiments on an MCU-class platform.

Another important direction is deeper optimization of the memory and dataflow path. The current implementation already uses task parallelism, staging buffers, and explicit SDRAM placement, but more aggressive zero-copy strategies, richer DMA-assisted transfer patterns, or compiler-assisted layout optimization may further improve throughput and energy efficiency. The same is true for the host-side interaction with the EdgeTPU: a richer study of tensor residency, batching patterns, or asynchronous execution opportunities could reveal additional performance margins on this class of embedded platform.

The developer experience can also be extended. The current system already provides a serial REPL, IP-based access with a web interface, mounted LittleFS transfer, boot logging, and boot-time execution through `main.py`. Future work could unify these into a more formal device-management workflow with remote script deployment, structured runtime tracing, automated collection of telemetry and images, artifact export for both obstacle and vision messages, and reproducible experiment packaging. Such tooling would be especially useful if the platform is to be used by multiple researchers or integrated into longer experimental campaigns.

At the system level, the project also opens a path toward broader hardware generalization. One of the arguments made throughout this work is that the EdgeTPU is a sufficiently capable low-power inference primitive to justify close integration with an MCU. A natural next step is therefore to explore how much of the present design can be ported to other MCU families, other camera configurations, or future open-source boards with similar accelerator-centric architectures. If successful, this would strengthen the broader claim that lightweight accelerator-plus-MCU designs are a viable class of embedded autonomy platform rather than a single-board exception.

Taken together, these directions suggest that the present work is less an endpoint than a foundation. The current implementation establishes the runtime, sensing, memory,

control, and experimentation abstractions needed for a compact embedded autonomy stack. Future work can now build on this base to improve viewpoint continuity, flight-platform symmetry, regulation-aware perception, deployment tooling, embedded learning workflows, and cross-platform generalization.

1. Jain, A.K. Data Clustering: 50 Years beyond K-Means. *Pattern Recognition Letters* **2010**, *31*, 651–666. <https://doi.org/10.1016/j.patrec.2009.09.011>.
2. Jolliffe, I.T.; Cadima, J. Principal Component Analysis: A Review and Recent Developments. *Philosophical Transactions of the Royal Society A* **2016**, *374*, 20150202. <https://doi.org/10.1098/rsta.2015.0202>.
3. Chandola, V.; Banerjee, A.; Kumar, V. Anomaly Detection: A Survey. *ACM Computing Surveys* **2009**, *41*, 1–58. <https://doi.org/10.1145/1541880.1541882>.
4. Pimentel, M.A.F.; Clifton, D.A.; Clifton, L.; Tarassenko, L. A Review of Novelty Detection. *Signal Processing* **2014**, *99*, 215–249. <https://doi.org/10.1016/j.sigpro.2013.12.026>.
5. Salvador, S.; Chan, P. Toward Accurate Dynamic Time Warping in Linear Time and Space. *Intelligent Data Analysis* **2007**, *11*, 561–580. <https://doi.org/10.3233/IDA-2007-11508>.
6. Bilmes, J.A. A Gentle Tutorial of the EM Algorithm and its Application to Parameter Estimation for Gaussian Mixture and Hidden Markov Models. Technical Report ICSI-TR-97-021, International Computer Science Institute, 1998.
7. Watkins, C.J.C.H.; Dayan, P. Q-Learning. *Machine Learning* **1992**, *8*, 279–292. <https://doi.org/10.1007/BF00992698>.
8. Auer, P.; Cesa-Bianchi, N.; Fischer, P. Finite-Time Analysis of the Multiarmed Bandit Problem. *Machine Learning* **2002**, *47*, 235–256. <https://doi.org/10.1023/A:1013689704352>.
9. Russo, D.; Roy, B.V.; Kazerouni, A.; Osband, I.; Wen, Z. A Tutorial on Thompson Sampling. *Foundations and Trends in Machine Learning* **2018**, *11*, 1–96. <https://doi.org/10.1561/22000000070>.
10. Mnih, V.; Kavukcuoglu, K.; Silver, D.; Rusu, A.A.; Veness, J.; Bellemare, M.G.; Graves, A.; Riedmiller, M.; Fidjeland, A.K.; Ostrovski, G.; et al. Human-Level Control through Deep Reinforcement Learning. *Nature* **2015**, *518*, 529–533. <https://doi.org/10.1038/nature14236>.
11. Durrant-Whyte, H.; Bailey, T. Simultaneous Localization and Mapping: Part I. *IEEE Robotics & Automation Magazine* **2006**, *13*, 99–110. <https://doi.org/10.1109/MRA.2006.1638022>.
12. Cadena, C.; Carlone, L.; Carrillo, H.; Latif, Y.; Scaramuzza, D.; Neira, J.; Reid, I.; Leonard, J.J. Past, Present, and Future of Simultaneous Localization and Mapping: Toward the Robust-Perception Age. *IEEE Transactions on Robotics* **2016**, *32*, 1309–1332. <https://doi.org/10.1109/TRO.2016.2624754>.
13. Bewley, A.; Ge, Z.; Ott, L.; Ramos, F.; Upcroft, B. Simple Online and Realtime Tracking. In Proceedings of the 2016 IEEE International Conference on Image Processing (ICIP), 2016, pp. 3464–3468. <https://doi.org/10.1109/ICIP.2016.7533003>.
14. Zhang, Y.; Sun, P.; Jiang, Y.; Yu, D.; Weng, F.; Yuan, Z.; Luo, P.; Liu, W.; Wang, X. ByteTrack: Multi-object Tracking by Associating Every Detection Box. In *Computer Vision – ECCV 2022*; Springer: Cham, 2022; pp. 1–21. https://doi.org/10.1007/978-3-031-20047-2_1.
15. Fraunhofer IMS and AIfES Contributors. AIfES: Artificial Intelligence for Embedded Systems, 2024. Accessed 2026-04-15.

Appendix A Appendix: `sentai.io`

The `sentai.io` namespace groups the simplest board-local control primitives. In the current runtime it is intentionally small and is used mainly for visual status indication, quick feedback during debugging, and minimal interaction from scripts that do not require a richer GPIO abstraction.

Appendix A.1 Functions

- `sentai.io.led_on()`: Turns the user LED on.
- `sentai.io.led_off()`: Turns the user LED off.

Appendix A.2 Example

```
import sentai

for _ in range(3):
    sentai.io.led_on()
    sentai.rtos.sleep_ms(200)
    sentai.io.led_off()
    sentai.rtos.sleep_ms(200)
```

Appendix B Appendix: `sentai.rtos`

The `sentai.rtos` namespace exposes runtime observability and timing services from the FreeRTOS layer. It is useful both for interactive diagnosis and for scripts that need coarse scheduling, uptime measurement, or visibility into task and memory behavior during long-running onboard experiments.

Appendix B.1 Functions

- `sentai.rtos.sleep_ms(ms)`: Suspends the current script for a specified number of milliseconds.
- `sentai.rtos.ticks_ms()`: Returns a monotonic millisecond counter.
- `sentai.rtos.tasks()`: Returns the current task list with state, priority, and stack high-water mark.
- `sentai.rtos.heap_info()`: Returns memory statistics for the RTOS heap and MicroPython GC heap.
- `sentai.rtos.cpu_usage()`: Returns per-task CPU-usage estimates.
- `sentai.rtos.uptime()`: Returns system uptime in seconds.

Appendix B.2 Example

```
import sentai

print('uptime:', sentai.rtos.uptime())
print('heap:', sentai.rtos.heap_info())
for name, state, prio, hwm in sentai.rtos.tasks():
    print(name, state, prio, hwm)
```

Appendix C Appendix: `sentai.tpu`

The `sentai.tpu` namespace is the main interface to EdgeTPU-backed inference. It covers model loading, input preparation, execution, tensor inspection, quantization helpers, and detector-oriented post-processing. In practice it is the default path for compact vision models compiled for the accelerator.

Appendix C.1 Functions

- `sentai.tpu.load(path)`: Loads a `.tflite` model from flash and creates the EdgeTPU interpreter. 2649
- `sentai.tpu.load_image(path)`: Loads a JPEG or raw RGB image into the input tensor. 2650
- `sentai.tpu.invoke()`: Runs inference and returns elapsed time in milliseconds. 2651
- `sentai.tpu.ready()`: Reports whether the model and interpreter are ready. 2652
- `sentai.tpu.num_outputs()`: Returns the number of output tensors. 2653
- `sentai.tpu.output_size(idx)`: Returns the byte size of an output tensor. 2654
- `sentai.tpu.output(idx)`: Returns raw output bytes. 2655
- `sentai.tpu.output_dims(idx)`: Returns the output tensor shape. 2656
- `sentai.tpu.output_type(idx)`: Returns the tensor type. 2657
- `sentai.tpu.row(idx, r)`: Returns a selected tensor row. 2658
- `sentai.tpu.value(idx, flat_i)`: Returns a single flattened value. 2659
- `sentai.tpu.save_output(path)`: Saves all outputs to CSV. 2660
- `sentai.tpu.input_quant()`: Returns input quantization parameters. 2661
- `sentai.tpu.output_quant(idx)`: Returns output quantization parameters. 2662
- `sentai.tpu.output_floats(idx)`: Returns dequantized outputs as Python floats. 2663
- `sentai.tpu.input_type()`: Returns the input tensor type. 2664
- `sentai.tpu.detect(conf, iou, max)`: Runs YOLO-style non-maximum suppression on the last inference output. 2665
- `sentai.tpu.draw(path, dets, quality)`: Draws detections on the last captured frame and saves a JPEG. 2666

Appendix C.2 Example

```
import sentai
2672
2673
2674
sentai.tpu.load('/models/yolov8n.tflite')
2675
sentai.camera.init()
2676
sentai.camera.to_tensor()
2677
ms = sentai.tpu.invoke()
2678
dets = sentai.tpu.detect(0.25, 0.45, 50)
2679
print('ms:', ms, 'detections:', len(dets))
2680
sentai.tpu.draw('/det.jpg', dets)
2681
```

Appendix D Appendix: sentai.tfl

The `sentai.tfl` namespace provides CPU-side TensorFlow Lite Micro inference on the Cortex-M7. It complements `sentai.tpu` for models that are not compiled for the EdgeTPU, for smaller auxiliary models, or for experiments that require direct MCU execution with explicit tensor-arena sizing.

Appendix D.1 Functions

- `sentai.tfl.load(path, arena_kb)`: Loads a `.tflite` model and allocates the tensor arena. 2687
- `sentai.tfl.unload()`: Frees the interpreter and tensor arena. 2688
- `sentai.tfl.invoke()`: Runs inference and returns elapsed time in milliseconds. 2689
- `sentai.tfl.ready()`: Reports whether the interpreter is ready. 2690
- `sentai.tfl.info()`: Prints model and arena information. 2691
- `sentai.tfl.set_input(data)`: Writes bytes into the input tensor. 2692

- `sentai.tfl.load_image(path)`: Loads a JPEG or raw RGB image into the input tensor. 2695
- `sentai.tfl.input_size()`: Returns the input tensor size in bytes. 2696
- `sentai.tfl.input_dims()`: Returns the input tensor shape. 2697
- `sentai.tfl.input_type()`: Returns the input tensor type. 2698
- `sentai.tfl.input_quant()`: Returns input quantization parameters. 2699
- `sentai.tfl.num_outputs()`: Returns the number of output tensors. 2700
- `sentai.tfl.output_size(idx)`: Returns output tensor size. 2701
- `sentai.tfl.output(idx)`: Returns raw output bytes. 2702
- `sentai.tfl.output_dims(idx)`: Returns the output tensor shape. 2703
- `sentai.tfl.output_type(idx)`: Returns the output tensor type. 2704
- `sentai.tfl.output_quant(idx)`: Returns output quantization parameters. 2705
- `sentai.tfl.output_floats(idx)`: Returns dequantized output values. 2706
- `sentai.tfl.row(idx, r)`: Returns one tensor row. 2707
- `sentai.tfl.value(idx, flat_i)`: Returns one output value. 2708
- `sentai.tfl.save_output(path)`: Saves all outputs to CSV. 2709

Appendix D.2 Example

```
import sentai

sentai.tfl.load('/models/keyword_detect.tflite', 256)
sentai.tfl.set_input(feature_bytes)
ms = sentai.tfl.invoke()
probs = sentai.tfl.output_floats(0)
print('inference:', ms, 'ms', probs)
```

Appendix E Appendix: sentai.fs

The `sentai.fs` namespace provides script-level access to the LittleFS user partition. It is used to move models, scripts, logs, images, and outputs between the runtime and persistent flash, and it is a central part of the interactive workflow because both model deployment and experiment logging depend on it.

Appendix E.1 Functions

- `sentai.fs.read(path)`: Reads an entire file as bytes. 2725
- `sentai.fs.read_str(path)`: Reads an entire file as text. 2726
- `sentai.fs.read_base64(path)`: Reads and prints a file as base64 text. 2727
- `sentai.fs.write(path, data)`: Writes bytes or text to flash. 2728
- `sentai.fs.size(path)`: Returns the file size. 2729
- `sentai.fs.exists(path)`: Checks whether a file or directory exists. 2730
- `sentai.fs.remove(path)`: Deletes a file or empty directory. 2731
- `sentai.fs.mkdir(path)`: Creates directories recursively. 2732
- `sentai.fs.ls(path)`: Lists directory contents. 2733
- `sentai.fs.format()`: Reformats the user partition and erases user data. 2734

Appendix E.2 Example

```
import sentai

sentai.fs.mkdir('/logs')
sentai.fs.write('/logs/run.txt', 'experiment started\n')
print(sentai.fs.read_str('/logs/run.txt'))
```

```
print(sentai.fs.ls('/logs'))
```

Appendix F Appendix: sentai.camera

The `sentai.camera` namespace controls image capture and basic camera-side preprocessing. It provides the main bridge between the sensing layer and the inference pipeline, including frame capture, JPEG export, tensor feeding, camera selection, and frame-sequence inspection for dual-camera operation.

Appendix F.1 Functions

- `sentai.camera.init(streaming)`: Starts the camera in continuous or trigger mode.
- `sentai.camera.stop()`: Stops camera capture.
- `sentai.camera.jpeg(quality)`: Captures a frame and returns JPEG bytes.
- `sentai.camera.save_jpeg(path, q)`: Captures and saves a JPEG file.
- `sentai.camera.to_tensor()`: Captures a frame and writes it into the TPU input tensor.
- `sentai.camera.resolution()`: Returns the current output resolution.
- `sentai.camera.set_resolution(w, h)`: Changes the capture resolution.
- `sentai.camera.native_res()`: Returns the sensor native resolution.
- `sentai.camera.select(id)`: Selects the active camera.
- `sentai.camera.frame_count()`: Returns the monotonic hardware frame counter.

Appendix F.2 Example

```
import sentai

sentai.camera.init()
sentai.camera.set_resolution(320, 320)
sentai.camera.select(0)
sentai.camera.save_jpeg('/front.jpg', 85)
sentai.camera.select(1)
sentai.camera.save_jpeg('/back.jpg', 85)
sentai.camera.stop()
```

Appendix G Appendix: sentai.imu

The `sentai.imu` namespace exposes the onboard LIS2DU12 accelerometer. It provides direct access to acceleration and derived tilt estimates and is commonly used for motion monitoring, camera-pose estimation, and simple event detection in scripts that combine perception with inertial context.

Appendix G.1 Functions

- `sentai.imu.init()`: Initializes the accelerometer.
- `sentai.imu.read()`: Returns acceleration and temperature measurements.
- `sentai.imu.degrees()`: Returns pitch and roll in degrees.
- `sentai.imu.radians()`: Returns pitch and roll in radians.

Appendix G.2 Example

```
import sentai

sentai.imu.init()
for _ in range(10):
    d = sentai.imu.degrees()
```



```

if d:
    print('pitch:', d['pitch'], 'roll:', d['roll'])
sentai.rtos.sleep_ms(100)

```

Appendix H Appendix: sentai.mic

The `sentai.mic` namespace manages the onboard PDM microphone through a ring-buffer recording model. It is designed for low-overhead acquisition, sound-level monitoring, and asynchronous MP3 export, allowing audio-triggered experiments without forcing the user to manage raw DMA buffers directly.

Appendix H.1 Functions

- `sentai.mic.start(seconds)`: Starts ring-buffer recording for the specified window length.
- `sentai.mic.stop()`: Stops the microphone and returns the number of buffered samples.
- `sentai.mic.recording()`: Reports whether the microphone is active.
- `sentai.mic.samples()`: Returns the number of samples currently available.
- `sentai.mic.level()`: Returns the current RMS level in centi-decibels.
- `sentai.mic.save_mp3()`: Saves buffered audio as an MP3 file and returns the filename.

Appendix H.2 Example

```

import sentai

sentai.mic.start(5)
sentai.rtos.sleep_ms(5000)
name = sentai.mic.save_mp3()
print('saved:', name)
sentai.mic.stop()

```

Appendix I Appendix: sentai.usb

The `sentai.usb` namespace provides two distinct services: mass-storage export of the user partition and raw USB CDC serial I/O when the REPL has been moved away from USB. It is therefore both a deployment path for files and a host-device communication channel, with explicit coordination required to avoid conflicts with filesystem access.

Appendix I.1 Functions

- `sentai.usb.drive(on)`: Enables or disables USB mass-storage export.
- `sentai.usb.open()`: Opens USB CDC ACM for raw serial I/O.
- `sentai.usb.close()`: Closes the raw USB CDC channel.
- `sentai.usb.write(data)`: Writes bytes or text to the USB host.
- `sentai.usb.read(max, timeout_ms)`: Reads bytes from the USB host.
- `sentai.usb.available()`: Returns the number of pending received bytes.

Appendix I.2 Example

```

import sentai

sentai.usb.drive(1)
# host copies files here
sentai.usb.drive(0)

```

```
print(sentai.fs.ls('/'))
```

2829

Appendix J Appendix: sentai.uart

2830

The `sentai.uart` namespace exposes raw UART serial I/O on the board-side serial port when that port is not reserved for the REPL or another protocol bridge. It is intended for direct communication with external serial devices such as radios, sensors, GPS modules, or custom peripherals.

2831

2832

2833

2834

Appendix J.1 Functions

2835

- `sentai.uart.open(baud)`: Opens the UART port at the requested baud rate.
- `sentai.uart.close()`: Closes the UART port and restores the default configuration.
- `sentai.uart.write(data)`: Sends bytes or text.
- `sentai.uart.read(max, timeout_ms)`: Reads bytes with polling, timeout, or blocking behavior.
- `sentai.uart.available()`: Returns the number of waiting bytes.

2836

2837

2838

2839

2840

2841

Appendix J.2 Example

2842

```
import sentai
```

2843

2844

```
sentai.uart.open(38400)
```

2845

```
sentai.uart.write(b'AT\r\n')
```

2846

```
resp = sentai.uart.read(256, 1000)
```

2847

```
print(resp)
```

2848

```
sentai.uart.close()
```

2849

Appendix K Appendix: sentai.console

2850

The `sentai.console` interface controls where the interactive MicroPython REPL is exposed. Although small, it is operationally important because several communication namespaces reuse the non-REPL interface, so moving the console between USB and UART determines which transport remains available for scripting.

2851

2852

2853

2854

Appendix K.1 Functions

2855

- `sentai.console()`: Returns the current REPL target, either `usb` or `uart`.
- `sentai.console('usb')`: Moves the REPL to USB CDC ACM.
- `sentai.console('uart')`: Moves the REPL to the UART console.

2856

2857

2858

Appendix K.2 Example

2859

```
import sentai
```

2860

2861

```
print(sentai.console())
```

2862

```
sentai.console('uart')
```

2863

```
sentai.usb.open()
```

2864

```
sentai.usb.write(b'host channel ready\n')
```

2865

```
sentai.usb.close()
```

2866

Appendix L Appendix: sentai.mesh

2867

The `sentai.mesh` namespace bridges the runtime to Meshtastic-compatible radios over UART. It supports both simple text exchange and structured vision-message transport, making it suitable for distributed sensing, low-bandwidth telemetry, and multi-node

2868

2869

2870

experiments in which detections or updates must be serialized and disseminated over a mesh network.

Appendix L.1 Functions

- `sentai.mesh.init(baud)`: Opens the UART link and starts the mesh receive task.
- `sentai.mesh.stop()`: Stops the receive task and closes the link.
- `sentai.mesh.node()`: Returns the local node number.
- `sentai.mesh.config()`: Requests configuration and NodeDB information from the radio.
- `sentai.mesh.send(text, dest, ch, ack)`: Sends a text message.
- `sentai.mesh.available()`: Returns the number of queued text messages.
- `sentai.mesh.receive(timeout)`: Receives the next text message.
- `sentai.mesh.send_detection(...)`: Sends a structured new-detection message.
- `sentai.mesh.send_update(...)`: Sends a structured track-update message.
- `sentai.mesh.vision_available()`: Returns the number of queued vision messages.
- `sentai.mesh.receive_vision(timeout)`: Receives the next structured vision message.
- `sentai.mesh.set_pose(pitch_deg, roll_deg, altitude_cm, heading_deg)`: Attaches camera pose metadata to outgoing vision messages.

Appendix L.2 Example

```
import sentai

sentai.mesh.init()
sentai.mesh.send('hello world')
msg = sentai.mesh.receive(5000)
if msg:
    print(msg['from'], msg['text'])
```

Appendix M Appendix: sentai.link

The `sentai.link` namespace exposes a MAVLink v2 bridge over UART. It supports message reception, heartbeat and status transmission, command dispatch, structured vision reporting, and obstacle-map generation for PX4 collision prevention. It is the primary interface for integrating the runtime with a conventional autopilot stack.

Appendix M.1 Functions

- `sentai.link.init(baud, sysid, compid)`: Opens the MAVLink link and starts the receive task.
- `sentai.link.stop()`: Stops the link and closes the UART channel.
- `sentai.link.available()`: Returns the number of queued MAVLink messages.
- `sentai.link.receive(timeout)`: Returns the next decoded MAVLink message.
- `sentai.link.heartbeat(type)`: Sends a heartbeat.
- `sentai.link.send(text, sev)`: Sends a STATUSTEXT message.
- `sentai.link.command(tsys, tcomp, cmd, conf, p1..p7)`: Sends a COMMAND_LONG message.
- `sentai.link.send_detection(...)`: Sends a structured vision detection.
- `sentai.link.send_update(...)`: Sends a structured vision update.
- `sentai.link.send_delete(...)`: Sends a structured vision deletion event.
- `sentai.link.obstacles_from_tracker(...)`: Builds and sends a 72-bin obstacle map from active tracks.

- `sentai.link.obstacle_distance(distances72, ...)`: Sends a raw `OBSTACLE_DISTANCE` message. 2918
- `sentai.link.obstacles_from_points(points, ...)`: Builds an obstacle map from arbitrary XY points. 2919
2920

Appendix M.2 Example 2921

```
import sentai 2922
2923
sentai.pipeline.init() 2924
sentai.pipeline.set_pose(120, -1, 0.0, 0.0) 2925
sentai.link.init(57600) 2926
while True: 2927
    sentai.link.obstacles_from_tracker(800, 20) 2928
    sentai.link.heartbeat() 2929
    sentai.rtos.sleep_ms(100) 2930
```

Appendix N Appendix: `sentai.crazy` 2931

The `sentai.crazy` namespace connects the runtime to a Crazyflie platform over CPX/CRTP. It exposes both higher-level flight procedures and lower-level control interfaces, allowing scripted indoor flight experiments without moving mission logic outside the main MicroPython environment. 2932
2933
2934
2935

Appendix N.1 Functions 2936

- `sentai.crazy.init(baud)`: Starts the Crazyflie communication tasks. 2937
- `sentai.crazy.stop()`: Stops the bridge and closes the link. 2938
- `sentai.crazy.debug(level)`: Changes debug verbosity. 2939
- `sentai.crazy.arm()`: Arms the drone. 2940
- `sentai.crazy.disarm()`: Disarms the drone. 2941
- `sentai.crazy.ping(timeout)`: Measures round-trip time to the drone. 2942
- `sentai.crazy.fly(height, hold_ms, takeoff_ms, land_ms)`: Executes a blocking high-level flight cycle. 2943
2944
- `sentai.crazy.attitude(roll, pitch, yaw_rate, thrust)`: Sends non-blocking attitude setpoints. 2945
2946
- `sentai.crazy.fly_stop()`: Stops attitude-controlled flight and disarms. 2947
- `sentai.crazy.takeoff(h, dur, yaw, use_yaw, group)`: Sends a high-level takeoff command. 2948
2949
- `sentai.crazy.land(h, dur, yaw, use_yaw, group)`: Sends a high-level landing command. 2950
2951
- `sentai.crazy.stop_motors(group)`: Performs an emergency motor stop. 2952
- `sentai.crazy.go_to(x, y, z, yaw, dur, rel, lin, group)`: Sends a waypoint-style movement command. 2953
2954
- `sentai.crazy.hover(vx, vy, yr, z)`: Sends hover-mode velocity commands. 2955
- `sentai.crazy.test_fly(power, dur_ms)`: Runs a raw motor test. 2956
- `sentai.crazy.send_crtp(port, ch, data)`: Sends an arbitrary CRTP packet. 2957

Appendix N.2 Example 2958

```
import sentai 2959
2960
sentai.crazy.init() 2961
sentai.crazy.arm() 2962
```

```

sentai.crazy.takeoff(0.5, 2.0)
sentai.rtos.sleep_ms(3000)
sentai.crazy.land(0.0, 2.0)
sentai.crazy.stop()

```

Appendix O Appendix: sentai.pipeline

The `sentai.pipeline` namespace exposes the background detection pipeline and the SentAI-SORT tracker. It is the main high-level perception service in the runtime and packages detector scheduling, tracking, event reporting, camera geometry, and ground-plane projection into a single script-facing interface.

Appendix O.1 Functions

- `sentai.pipeline.start(conf, iou, max, track)`: Starts continuous detection and optionally tracking.
- `sentai.pipeline.stop()`: Stops the pipeline.
- `sentai.pipeline.running()`: Reports whether the pipeline is active.
- `sentai.pipeline.get(timeout_ms)`: Returns the next detection frame.
- `sentai.pipeline.stats()`: Returns processed, dropped, and FPS statistics.
- `sentai.pipeline.tracks()`: Returns the current active-track snapshot.
- `sentai.pipeline.event(timeout_ms)`: Returns the next tracker event.
- `sentai.pipeline.track_config(...)`: Gets or sets tracker parameters.
- `sentai.pipeline.set_pose(altitude_cm, heading_deg, lat, lon)`: Sets camera pose for ground projection.
- `sentai.pipeline.camera_config(cam_id, fov_h, fov_v, mount_pitch, mount_roll, mount_yaw)`: Gets or sets per-camera geometry.

Appendix O.2 Example

```

import sentai

sentai.camera.init()
sentai.pipeline.camera_config(0, 70.8, 43.4, 0, 0, 0)
sentai.pipeline.set_pose(1000, 0, 44.4268, 26.1025)
sentai.pipeline.start(0.3, 0.45, 50, True)
while True:
    evt = sentai.pipeline.event(2000)
    if evt:
        print(evt)

```

Appendix P Appendix: sentai.aifes

The `sentai.aifes` namespace integrates the AIFES library for on-device neural-network training and inference. It is intended for small multilayer perceptrons and compact transfer-learning workflows in which features extracted by the TPU or camera are fed into a trainable MCU-side head.

Appendix P.1 Functions

- `sentai.aifes.load(path)`: Loads a model architecture from YAML.
- `sentai.aifes.load_weights(path)`: Loads pre-trained weights.
- `sentai.aifes.save_weights(path)`: Saves trained weights.
- `sentai.aifes.unload()`: Frees the loaded model.
- `sentai.aifes.ready()`: Reports whether the model is ready.

- `sentai.aifes.train(x_data, y_data, ...)`: Trains the model on supplied samples. 3008
- `sentai.aifes.set_input(data)`: Sets the input vector from floats. 3009
- `sentai.aifes.from_tpu(idx)`: Uses a TPU output tensor as AIFES input. 3010
- `sentai.aifes.from_camera(w, h, gray)`: Uses a camera frame as normalized input. 3011
- `sentai.aifes.from_mic(samples)`: Uses microphone samples as input. 3012
- `sentai.aifes.invoke()`: Runs inference. 3013
- `sentai.aifes.output()`: Returns the latest output vector. 3014
- `sentai.aifes.mic_init()`: Initializes microphone capture for audio models. 3015
- `sentai.aifes.mic_stop()`: Stops microphone capture. 3016
- `sentai.aifes.predict(input)`: Legacy single-call inference helper. 3017
- `sentai.aifes.info()`: Returns model metadata. 3018

Appendix P.2 Example

```
import sentai

sentai.tpu.load('/models/mobilenet_features.tflite')
sentai.aifes.load('/models/classifier_head.yaml')
sentai.camera.init()

sentai.camera.to_tensor()
sentai.tpu.invoke()
sentai.aifes.from_tpu(0)
sentai.aifes.invoke()
print(sentai.aifes.output())
```

Appendix Q Appendix: `sentai.kmeans`

The `sentai.kmeans` namespace implements compact k-means clustering for embedding vectors. It is mainly useful for few-shot or unsupervised classification workflows in which a TPU feature extractor produces dense embeddings and the runtime clusters them directly on device.

Appendix Q.1 Functions

- `sentai.kmeans.init(k, dim)`: Initializes the clustering state. 3039
- `sentai.kmeans.add(cluster_idx, vector)`: Adds a labeled vector to a cluster. 3040
- `sentai.kmeans.add_from_tpu(cluster_idx, tpu_idx)`: Adds a TPU output directly. 3041
- `sentai.kmeans.compute()`: Computes centroids from labeled samples. 3042
- `sentai.kmeans.fit(max_iter)`: Runs unsupervised Lloyd iterations. 3043
- `sentai.kmeans.predict(vector)`: Predicts the closest centroid. 3044
- `sentai.kmeans.from_tpu(tpu_idx)`: Predicts from a TPU output tensor. 3045
- `sentai.kmeans.distances(vector)`: Returns distances to all centroids. 3046
- `sentai.kmeans.save(path)`: Saves centroids to flash. 3047
- `sentai.kmeans.load(path)`: Loads centroids from flash. 3048
- `sentai.kmeans.centroid(idx)`: Returns one centroid vector. 3049
- `sentai.kmeans.info()`: Returns configuration and sample-count metadata. 3050
- `sentai.kmeans.clear()`: Clears training vectors while keeping centroids. 3051

Appendix Q.2 Example

```

import sentai

sentai.tpu.load('/models/mobilenet_features.tflite')
sentai.camera.init()
sentai.kmeans.init(3, 1280)

sentai.camera.to_tensor()
sentai.tpu.invoke()
sentai.kmeans.add_from_tpu(0, 0)
sentai.kmeans.compute()
print(sentai.kmeans.info())

```

Appendix R Appendix: sentai.pca

The `sentai.pca` namespace provides principal-component analysis for high-dimensional embeddings. Its role is to reduce vector dimension before downstream use in classifiers, clustering methods, or lightweight anomaly detectors, especially when raw feature vectors are too large for repeated on-device processing.

Appendix R.1 Functions

- `sentai.pca.init(in_dim, out_dim)`: Initializes the PCA model.
- `sentai.pca.add(vector)`: Adds a training vector.
- `sentai.pca.from_tpu(idx)`: Adds a TPU output tensor as training data.
- `sentai.pca.fit()`: Computes the principal components.
- `sentai.pca.transform(vector)`: Projects a vector into the reduced space.
- `sentai.pca.transform_tpu(idx)`: Projects a TPU output tensor directly.
- `sentai.pca.inverse(reduced)`: Reconstructs an approximate original vector.
- `sentai.pca.explained_variance()`: Returns component variances.
- `sentai.pca.save(path)`: Saves the fitted PCA model.
- `sentai.pca.load(path)`: Loads a PCA model.
- `sentai.pca.info()`: Returns model metadata.
- `sentai.pca.clear()`: Clears stored training vectors.

Appendix R.2 Example

```

import sentai

sentai.tpu.load('/models/mobilenet_features.tflite')
sentai.camera.init()
sentai.pca.init(1280, 32)
sentai.camera.to_tensor()
sentai.tpu.invoke()
sentai.pca.from_tpu(0)
sentai.pca.fit()
print(sentai.pca.transform_tpu(0)[:5])

```

Appendix S Appendix: sentai.anomaly

The `sentai.anomaly` namespace implements online anomaly detection based on incremental covariance estimation and Mahalanobis distance, together with a simple CUSUM change detector. It is suitable for learning a baseline normal distribution and then scoring embeddings or scalar signals during deployment.

Appendix S.1 Functions

- `sentai.anomaly.init(dim)`: Initializes the anomaly model. 3100
- `sentai.anomaly.observe(vector)`: Updates the normal model with one observation. 3101
- `sentai.anomaly.observe_tpu(idx)`: Updates the model from a TPU output tensor. 3102
- `sentai.anomaly.score(vector)`: Returns the anomaly score for a vector. 3103
- `sentai.anomaly.score_tpu(idx)`: Scores a TPU output tensor. 3104
- `sentai.anomaly.threshold(val)`: Gets or sets the anomaly threshold. 3105
- `sentai.anomaly.is_anomaly(vector)`: Tests whether a vector exceeds the threshold. 3106
- `sentai.anomaly.is_anomaly_tpu(idx)`: Tests a TPU output tensor. 3107
- `sentai.anomaly.cusum_init(threshold, drift)`: Initializes the CUSUM detector. 3108
- `sentai.anomaly.cusum_observe(value)`: Feeds one scalar sample into CUSUM. 3109
- `sentai.anomaly.cusum_score()`: Returns current positive and negative cumulative sums. 3110
- `sentai.anomaly.cusum_reset()`: Clears CUSUM accumulators. 3111
- `sentai.anomaly.save(path)`: Saves the learned model. 3112
- `sentai.anomaly.load(path)`: Loads a saved model. 3113
- `sentai.anomaly.stats()`: Returns model statistics. 3114
- `sentai.anomaly.info()`: Alias for `stats()`. 3115
- `sentai.anomaly.clear()`: Clears all statistics. 3116

Appendix S.2 Example

```

import sentai
3118
3119
3120
sentai.tpu.load('/models/mobilenet_features.tflite')
3121
sentai.camera.init()
3122
sentai.anomaly.init(1280)
3123
3124
for _ in range(30):
3125
    sentai.camera.to_tensor()
3126
    sentai.tpu.invoke()
3127
    sentai.anomaly.observe_tpu(0)
3128
3129
sentai.camera.to_tensor()
3130
sentai.tpu.invoke()
3131
print(sentai.anomaly.score_tpu(0))
3132

```

Appendix T Appendix: sentai.dtw

The `sentai.dtw` namespace exposes dynamic time warping for template-based sequence matching. It is intended for short gesture or audio patterns where full trainable sequence models are unnecessary and a small number of stored templates is sufficient for online recognition.

Appendix T.1 Functions

- `sentai.dtw.init(dim, max_len)`: Initializes the DTW engine. 3139
- `sentai.dtw.record_start(label)`: Starts recording a named template. 3140
- `sentai.dtw.record_add(frame)`: Adds one feature frame. 3141
- `sentai.dtw.record_add_imu()`: Adds the current IMU sample as a frame. 3142
- `sentai.dtw.record_add_mic()`: Adds the current microphone level as a frame. 3143
- `sentai.dtw.record_end()`: Finalizes and stores the template. 3144

- `sentai.dtw.match(sequence, threshold)`: Matches a supplied sequence against stored templates.
- `sentai.dtw.match_imu(n_frames, threshold, delay_ms)`: Captures IMU frames and matches them.
- `sentai.dtw.match_mic(n_frames, threshold, delay_ms)`: Captures microphone frames and matches them.
- `sentai.dtw.templates()`: Lists stored templates.
- `sentai.dtw.remove(label)`: Removes a template.
- `sentai.dtw.save(path)`: Saves all templates.
- `sentai.dtw.load(path)`: Loads saved templates.
- `sentai.dtw.info()`: Returns engine metadata.
- `sentai.dtw.clear()`: Clears all templates.

Appendix T.2 Example

```
import sentai

sentai.imu.init()
sentai.dtw.init(3, 100)
sentai.dtw.record_start('wave')
for _ in range(50):
    sentai.dtw.record_add_imu()
    sentai.rtos.sleep_ms(20)
sentai.dtw.record_end()
print(sentai.dtw.match_imu(50, 500.0))
```

Appendix U Appendix: sentai.hmm

The `sentai.hmm` namespace implements a discrete hidden Markov model with Baum-Welch training and Viterbi decoding. It is suitable for compact sequence-classification tasks in which observations can be discretized, such as simple activity recognition or discretized feature streams extracted from sensors.

Appendix U.1 Functions

- `sentai.hmm.init(n_states, n_obs)`: Initializes the HMM.
- `sentai.hmm.set_transition(i, j, prob)`: Sets a transition probability.
- `sentai.hmm.set_emission(state, obs, prob)`: Sets an emission probability.
- `sentai.hmm.set_prior(state, prob)`: Sets an initial-state probability.
- `sentai.hmm.add_seq(obs_list)`: Adds a training sequence.
- `sentai.hmm.train(max_iter)`: Runs Baum-Welch training.
- `sentai.hmm.viterbi(obs_list)`: Decodes the most likely state sequence.
- `sentai.hmm.predict(obs)`: Predicts the most likely next state.
- `sentai.hmm.log_likelihood(obs_list)`: Scores a sequence.
- `sentai.hmm.from_tpu(idx, n_bins)`: Discretizes a TPU output tensor.
- `sentai.hmm.from_imu(n_bins)`: Discretizes IMU magnitude.
- `sentai.hmm.save(path)`: Saves the HMM model.
- `sentai.hmm.load(path)`: Loads a saved model.
- `sentai.hmm.info()`: Returns model metadata.
- `sentai.hmm.clear()`: Clears parameters and sequences.

Appendix U.2 Example

```
import sentai
```

```

sentai.imu.init()
sentai.hmm.init(3, 8)
seq = [sentai.hmm.from_imu(8) for _ in range(100)]
sentai.hmm.add_seq(seq)
sentai.hmm.train(20)
print(sentai.hmm.predict(sentai.hmm.from_imu(8)))

```

Appendix V Appendix: sentai.rl

The `sentai.rl` namespace collects three reinforcement-learning mechanisms with different complexity levels: tabular Q-learning, multi-armed bandits, and a compact DQN. It is intended for online adaptation of thresholds, policies, or configuration choices in small embedded experiments.

Appendix V.1 Functions

- `sentai.rl.q_init(n_states, n_actions)`: Initializes a tabular Q-table.
- `sentai.rl.q_update(s, a, r, s_next, alpha, gamma)`: Updates one Q-value.
- `sentai.rl.q_action(s, epsilon)`: Selects an action with an epsilon-greedy policy.
- `sentai.rl.q_value(s, a)`: Returns one Q-value.
- `sentai.rl.q_save(path)`: Saves the Q-table.
- `sentai.rl.q_load(path)`: Loads the Q-table.
- `sentai.rl.q_clear()`: Clears the Q-table.
- `sentai.rl.mab_init(n_arms)`: Initializes a multi-armed bandit.
- `sentai.rl.mab_pull(arm, reward)`: Reports the reward of a selected arm.
- `sentai.rl.mab_select(strategy)`: Selects the next arm.
- `sentai.rl.mab_stats()`: Returns per-arm statistics.
- `sentai.rl.mab_clear()`: Clears bandit statistics.
- `sentai.rl.dqn_init(state_dim, n_actions, hidden)`: Initializes the DQN.
- `sentai.rl.dqn_observe(state, action, reward, next_state, done)`: Adds one replay transition.
- `sentai.rl.dqn_action(state, epsilon)`: Selects an action from the DQN.
- `sentai.rl.dqn_train(batch_size)`: Trains on a replay batch.
- `sentai.rl.dqn_save(path)`: Saves DQN weights.
- `sentai.rl.dqn_load(path)`: Loads DQN weights.
- `sentai.rl.dqn_clear()`: Clears DQN state.
- `sentai.rl.info()`: Returns a summary of all three subsystems.

Appendix V.2 Example

```

import sentai

sentai.rl.mab_init(4)
for _ in range(20):
    arm = sentai.rl.mab_select(1)
    reward = 1.0 if arm == 0 else 0.2
    sentai.rl.mab_pull(arm, reward)
print(sentai.rl.mab_stats())

```

Appendix W Appendix: sentai.slam

The `sentai.slam` namespace implements a compact detection-based EKF-SLAM system. It estimates a planar robot pose and maintains a landmark map using object detections

or manually supplied observations. The interface is decoupled from the TPU, so it can consume detections from any compatible source.

Appendix W.1 Functions

- `sentai.slam.init(fov_h_deg, img_w, img_h, max_lm, baseline_m)`: Initializes the SLAM model.
- `sentai.slam.update(detections)`: Updates pose and landmarks from monocular detections.
- `sentai.slam.update_stereo(dets_left, dets_right)`: Updates from stereo detections.
- `sentai.slam.observe(class_id, bearing_rad, range_m)`: Adds a manual landmark observation.
- `sentai.slam.predict(dx, dy, dtheta)`: Applies an explicit motion prediction.
- `sentai.slam.pose()`: Returns the current estimated pose.
- `sentai.slam.landmarks()`: Returns the active landmark set.
- `sentai.slam.noise(sigma_v, sigma_w, sigma_b, sigma_r)`: Gets or sets noise parameters.
- `sentai.slam.imu_correct(pitch_deg, roll_deg)`: Adjusts bearing noise using IMU tilt.
- `sentai.slam.clear()`: Resets pose and landmarks.
- `sentai.slam.save(path)`: Saves the map and pose.
- `sentai.slam.load(path)`: Loads the map and pose.
- `sentai.slam.info()`: Returns SLAM metadata and statistics.

Appendix W.2 Example

```
import sentai

sentai.slam.init(66.0, 320, 320)
sentai.tpu.load('/models/yolov8n_320.tflite')
sentai.camera.init()

sentai.camera.to_tensor()
sentai.tpu.invoke()
dets = sentai.tpu.detect(0.3, 0.45, 50)
sentai.slam.update(dets)
print(sentai.slam.pose(), sentai.slam.landmarks())
```

Appendix X Glossary

Terms, acronyms and platform-specific identifiers used in the paper and its appendices. Grouped by domain.

Appendix X.1 Embedded-systems / RTOS

Term	Expansion / meaning
DMA	Direct Memory Access. Hardware engine that moves bytes without CPU involvement.

Term	Expansion / meaning
eDMA	“enhanced” DMA — the NXP i.MX RT1176 DMA0 peripheral used for the SDRAM→tensor memcpy.
ISR	Interrupt Service Routine. Runs in interrupt context with bounded, minimal work per NASA/JPL discipline (embedded.md §C).
MCU	Microcontroller Unit. Here: NXP i.MX RT1176.
RTOS	Real-Time Operating System. Here: FreeRTOS 10.x (CMSIS M7 build).
SDRAM	Synchronous DRAM. On-module, 16 MB on the SEMC bus at 166 MHz.
SEMC	Smart External Memory Controller. The RT1176 peripheral that drives the external SDRAM.
SDP	Serial Download Protocol. NXP’s ROM bootloader protocol; the board falls back to SDP if firmware fails to boot.
WDOG	Hardware watchdog (WDOG1). 30 s timeout on the 32 kHz clock, independent of CPU.
VBLANK	Vertical blanking interval. Period between the last line of one video frame and the first line of the next, during which no pixel data flows on the MIPI-CSI lane.
VSYNC / HSYNC	Vertical / horizontal sync pulses marking frame / line boundaries. In MIPI-CSI2, replaced by FS / LS short packets.

3276

Appendix X.2 Camera / imaging

3277

Term	Meaning
CSI	Camera Serial Interface. The RT1176 on-chip peripheral that reads pixel data.
CSI-2	MIPI Camera Serial Interface, version 2. The protocol spoken between the OV5640 and the RT1176 on our platform.
D-PHY	MIPI physical-layer specification that CSI-2 runs over.
FS / LS	Frame Start / Line Start — short packets in MIPI-CSI2 marking frame and line boundaries.
EOF	End-of-Frame — the interrupt fired when a DMA buffer has finished filling. Our flip-on-EOF fix runs inside this ISR.

Term	Meaning
MUX	Multiplexer. On this board, an analogue switch on the shared MIPI-CSI2 lane, GPIO-controlled; selects which of the two OV5640 sensors the CSI receiver is talking to.
PXP	Pixel Pipeline. The RT1176 2-D graphics accelerator used for colour conversion and resizing between the raw DMA buffer and the TPU input tensor.
OV5640	OmniVision 5-megapixel image sensor used as both cam0 and cam1 on this board. Dual instance, shared MIPI lane via MUX.
FSIN	Frame Sync Input pin on the OV5640, used for master/slave synchronisation between two sensors. Not wired on the SentAI board.
AEC / AGC	Auto Exposure Control / Auto Gain Control — the sensor's internal adaptive exposure loops. Take several frames to converge after a stream resume.
QXGA / 1080P / 720P / VGA / QVGA	Standard resolutions: 2592×1944, 1920×1080, 1280×720, 640×480, 320×240.

3278

Appendix X.3 AI inference

3279

Term	Meaning
EdgeTPU	Google-designed systolic-array AI accelerator, on-die on the Coral Dev Board Micro. Runs 8-bit quantised TFLite models at fixed clock; execution time scales with input pixel count and FLOPs.
TFLite	TensorFlow Lite. The on-device inference framework. “TFLite Micro” is the embedded variant we use via <code>sentai.tpu.*</code> .
YOLOv5 / YOLOv5-enhanced	Single-stage object-detection architecture family. The “enhanced” 1-class 512×512 model used throughout E15-E18 has a single upsample at P5/32 and a 1-class head producing [1, 1344, 6].
NMS	Non-Maximum Suppression. Post-processing step that deduplicates overlapping detections. Runs on the CPU (MicroPython-callable) after Invoke.

Term	Meaning
Anchor	A pre-defined bounding-box prior used by YOLO-style detectors. 1344 anchors in the 512×512 single-upsample model.
Mode 3 (kMax)	EdgeTPU performance mode requested by <code>sentai.tpu.load()</code> .
Invoke	Single forward pass of the TFLite model. Reported as <code>invoke_ms</code> in every CSV.
Quantisation (scale, zero_point)	8-bit integer representation of activations; $\text{real} = (\text{int} - \text{zero_point}) \times \text{scale}$.

3280

Appendix X.4 Firmware platform (SentAI-specific)

3281

Term	Meaning / where defined
sentai module	Root MicroPython namespace that exposes the hardware surface. Submodules: camera, tpu, pipeline, fs, rtos, diag, io, imu, mic, usb, link, mesh, crazy, tfl, aifes, kmeans, pca, anomaly, dtw, hmm, rl, slam.
PrepTask / InferTask	The two tasks inside the firmware parallel pipeline (see camera.md §PXP). PrepTask does grab + PXP + quant into a staging buffer; InferTask does memcpy-to-tensor + Invoke + NMS.
Staging buffer / tensor buffer	The two 512×512×3 buffers between the two pipeline tasks. The staging buffer is written by PrepTask; the tensor buffer is written by InferTask's memcpy (eDMA since the fix) and read by the TPU.
<code>sentai.camera.select(id)</code>	Arm a glitch-free flip to camera id; the CSI EOF ISR consumes the arm in VBLANK. cam_switch.md §“Fix B”.
<code>sentai.camera.switch_drain(n)</code>	Number of fresh ISR frames required after a MUX flip before the next captured frame is returned. Default 2; 1 is an experimentation hook.
<code>sentai.camera.ratio(a, b)</code>	Stateless auto-alternate scheduler: over any (a+b)-frame cycle, cam0 gets a frames and cam1 gets b. (0, 0) disables.
<code>sentai.diag.cam_stats()</code>	Returns a dict of persistent fault counters: <code>switch_ok_eof</code> , <code>switch_fallback</code> , <code>drain_timeout</code> , <code>grab_retry</code> , <code>grab_fatal</code> .

Term	Meaning / where defined
Session	A <code>/diags/sNNN_<name>/</code> folder on the LittleFS user partition, opened by <code>diag.begin(...)</code> and closed by <code>diag.end()</code> . Self-contained: CSV + descriptor text + manifest + summary + scene snapshots.
Manifest	<code>manifest.csv</code> inside a session: one row per experiment completed in that session.
Diag experiment	A function <code>e<N>_xxx(...)</code> inside the <code>diag</code> package; the numbered experiment classes referenced throughout this paper (E13, E14, E15, E16, E17, E18).

3282

Appendix X.5 Storage / USB

3283

Term	Meaning
LFS / LittleFS	Flash-friendly file system by ARM; used for the NAND “user” partition. All <code>/diags/</code> data lives here.
MSC	USB Mass Storage Class. Mode entered by <code>sentai.usb.drive(1)</code> ; exposes the raw LittleFS block device as <code>/dev/sda</code> on the host.
CDC-ACM	USB Communications Device Class — Abstract Control Model. The REPL transport (<code>/dev/ttyACM0</code>).
CDC-NCM	USB Communications Device Class — Network Control Model. The IP transport (<code>enxXXXX</code> on the host).
HTTP fast path vs slow path	See lfs.md §4.2. <code>/api/raw</code> reads go through a mutex-protected fast path; <code>/api/lfs</code> always queues to <code>lfs_task</code> (slow path) to avoid long <code>tcpip_thread</code> blocking.
“Bricked”	Board visible on USB as Google Coral ID <code>18d1:9307</code> . Means firmware never started; requires a manual button press to enter SDP mode. The camera-switch work was careful to keep USB CDC up before any risky code so the board is never in this state (see embedded.md §M).

3284

Appendix X.6 Measurement methodology

3285

Term	Meaning
Warm-up	Pre-measurement iteration(s) that exercise every stage on the path but are not recorded. See experiments/methodology.md §3.1.
Drop-first-sample	Convention that the first recorded iteration is discarded from statistics, because it straddles warm-up and steady state. See statistical_notes.md §2.
Sweep	One block of N iterations with a fixed alternation pattern. E18 runs three sweeps per session (A fixed cam0, B fixed cam1, C alternating).
Per-switch overhead	$C_total_mean = \max(A_total_mean, B_total_mean)$ in an E18 head-to-tail session. Table 3 of evaluation.md .
Directional asymmetry	$cam1_total_mean - cam0_total_mean$ within the alternating sweep of an E16/E18 session. Non-zero before Fix B; within noise after Fix B.
Scene snapshot	JPEG captured from each camera at the start and end of a session, archived next to the CSVs for offline scene-drift verification.
Fault counter	Persistent-since-boot counter of a degraded-path event (ISR arm not consumed, drain timeout, grab retry, grab fatal). Zero-valued post-run counter is a measurement-integrity check.

3286

Appendix X.7 Build / tooling

3287

Term	Meaning
<code>flashtool.py</code>	<code>scripts/flashtool.py</code> — NXP-provided tool; <code>-e sentai_runtime</code> flashes the example in persistent mode.
<code>_host_upload_repl.py</code>	<code>diag/_host_upload_repl.py</code> — our REPL-chunked uploader. Used instead of <code>HTTP /api/write/</code> because the latter hangs on this firmware.
<code>repl_run.py</code>	Simple REPL driver. Has a stale-prompt bug on long-running commands; for robust driving we use the inline <code>_send_line</code> pattern from <code>_host_upload_repl.py</code> .

Term	Meaning
QSTR	MicroPython's interned-string pool. Must be regenerated (with the <code>micropython-embed-package</code> make target) whenever a <code>MP_QSTR_*</code> symbol is added, renamed, or removed. See agent/agent.md §6.
<code>ticks_ms()</code>	<code>sentai.rtos.ticks_ms()</code> — 32-bit FreeRTOS tick counter exposed to MicroPython. Wraps every 49.7 days; all deltas are computed with unsigned subtraction so the wrap is harmless.
<code>_ticks()</code>	Module-local alias for <code>sentai.rtos.ticks_ms()</code> used throughout <code>diag/</code> .